CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS

CURSO DE ENGENHARIA DE COMPUTAÇÃO

IGOR LAMOIA QUEIROZ

INTERPRETADOR DINÂMICO PARA ENSINO DE PROGRAMAÇÃO:

Uma abordagem interativa com ambiente de desenvolvimento integrado (IDE) para personalização e execução de comandos definidos pelo usuário

Leopoldina

2026

IGOR LAMOIA QUEIROZ

INTERPRETADOR DINÂMICO PARA ENSINO DE PROGRAMAÇÃO:

Uma abordagem interativa com ambiente de desenvolvimento integrado (IDE) para personalização e execução de comandos definidos pelo usuário

Trabalho de Conclusão de Curso apresentado ao Curso de Engenharia de Computação do Centro Federal de Educação Tecnológica de Minas Gerais, do Campus Leopoldina, como parte do requisito para obtenção do título de Bacharel em Engenharia de Computação.

Orientador: Luís Augusto Mattos Mendes

Coorientador: Eduardo Gabriel Reis Miranda

Leopoldina

2026

AXXX Queiroz, Igor Lamoia.

2026 Interpretador Dinâmico para Ensino de Programação : Uma abordagem interativa com ambiente de desenvolvimento integrado (IDE) para personalização e execução de comandos definidos pelo usuário / Igor Lamoia Queiroz

Queiroz - Leopoldina : CEFET-MG, 2026.

x, 117 f. : il.

Orientador: Luís Augusto Mattos Mendes
Coorientador: Eduardo Gabriel Reis Miranda

TCC (Graduação) – Centro Federal de Educação Tecnológica de Minas Gerais, Unidade Leopoldina. Engenharia de Computação.

1. Interpretador dinâmico. 2. Ensino de programação. 3. Compiladores. 4. Ambiente de Desenvolvimento Integrado. I. Mendes, Luís Augusto Mattos. II. Miranda, Eduardo Gabriel Reis. III. Interpretador Dinâmico para Ensino de Programação.

CDU XXX.XX

Elaboração da ficha catalográfica pela bibliotecária Luzia Adriana Damasceno, CRB 6º 2305 / CEFET-MG campus Leopoldina.

IGOR LAMOIA QUEIROZ

INTERPRETADOR DINÂMICO PARA ENSINO DE PROGRAMAÇÃO:
Uma abordagem interativa com ambiente de desenvolvimento integrado (IDE) para personalização e execução de comandos definidos pelo usuário

Trabalho de conclusão de curso apresentado ao Curso de Engenharia de Computação do Centro Federal de Educação Tecnológica de Minas Gerais, do Campus Leopoldina, como parte do requisito para obtenção do título de Bacharel em Engenharia de Computação.

Banca Examinadora:

Prof. Me. Luís Augusto Mattos Mendes- Orientador
CEFET-MG

Prof. Dsc. Eduardo Gabriel Reis Miranda- Coorientador
CEFET-MG

Prof. Me. Matheus Ávila Moreira de Paula
CEFET-MG

Prof. Dsc. Gabriella Castro Barbosa Costa
CEFET-MG

(Assinaturas Digitais)

Leopoldina
2026

AGRADECIMENTOS

Agradeço à minha família pelo apoio incondicional, incentivo e paciência ao longo desta jornada. Sem vocês, nada disso seria possível. Meu sincero agradecimento também ao meu orientador, Luís Augusto Mattos Mendes, e ao meu coorientador, Eduardo Gabriel Reis Miranda, pela orientação, dedicação e ensinamentos valiosos que tornaram este trabalho possível. A todos que, de alguma forma, contribuíram para esta conquista, meu muito obrigado.

RESUMO

Este trabalho apresenta o projeto, a implementação e a integração de um Ambiente de Desenvolvimento Integrado (IDE) web a um núcleo de tradução e execução configurável para apoio ao ensino introdutório de programação. O projeto é desenvolvido em coautoria: o texto complementar trata do núcleo, enquanto este documento se concentra na interface, no assistente de personalização e nos fluxos educacionais. O qualificativo dinâmico refere-se à aplicação, a cada análise, do vocabulário e das variantes de linguagem selecionados pelo usuário entre as possibilidades implementadas. O desenvolvimento combina revisão bibliográfica, especificação de requisitos, modelagem da arquitetura, implementação incremental e testes automatizados. A IDE permite editar e executar código no navegador, inspecionar resultados de análise, depurar programas e administrar linguagens e atividades de turmas. A revisão técnica inclui testes da IDE e uma verificação delimitada de responsividade, navegação por teclado e acessibilidade automatizada. Também é proposta uma avaliação futura com tarefas, medidas e instrumentos de coleta. Os resultados apresentados descrevem o artefato e as verificações realizadas; não demonstram redução de carga cognitiva, melhoria de aprendizagem ou conformidade integral de acessibilidade. A aplicação do protocolo com estudantes e a correção das limitações identificadas constituem encaminhamentos futuros.

Palavras-chave: Interpretador Dinâmico; Educação em Programação; Ambiente Educacional Web; IDE Educacional; Personalização de Linguagens.

ABSTRACT

This work presents the design, implementation, and integration of a web Integrated Development Environment (IDE) with a configurable translation and execution core to support introductory programming education. The project is developed jointly: the companion document addresses the core, whereas this document focuses on the interface, the customization wizard, and educational workflows. The term dynamic refers to applying the vocabulary and language variants selected by the user from the implemented options at each analysis. Development combines a literature review, requirements specification, architecture modeling, incremental implementation, and automated tests. The IDE supports editing and running code in the browser, inspecting analysis results, debugging programs, and managing languages and classroom activities. Technical verification includes IDE tests and a limited assessment of responsive layout, keyboard navigation, and automated accessibility checks. A future evaluation is also proposed, specifying tasks, measures, and data collection instruments. The reported results describe the artifact and the checks performed; they do not establish reduced cognitive load, improved learning, or full accessibility conformance. Applying the protocol with students and addressing the identified limitations remain future work.

Keywords: Dynamic Interpreter; Programming Education; Web-Based Educational Environment; Educational IDE; Language Customization.

LISTA DE ILUSTRAÇÕES

Figura 2.1 – Arquitetura de um compilador moderno. 18
Figura 2.2 – Exemplificação de agrupamento de caracteres em lexemas. 20
Figura 3.1 – Casos de uso: acesso e ambiente de desenvolvimento. 44
Figura 3.2 – Casos de uso: participação e gestão pedagógica. 45
Figura 4.1 – Arquitetura da plataforma. 54
Figura 4.2 – IDE com a linguagem Didática em Português ativa e o terminal após uma execução. 64
Figura 4.3 – Sequência de *tokens* gerada para um programa escrito em Didática em Português. 67
Figura 4.4 – Área de resolução após o envio, com linguagem obrigatória e resultado dos casos de teste. 71
Figura 4.5 – Correção de uma submissão pelo professor. 72
Figura 4.6 – Adaptação do assistente na versão verificada em setembro de 2026. 74
Figura 4.7 – Passos 1 e 2 do tutorial do assistente de criação de linguagem. 78
Figura 4.8 – Passos 3 e 4 do tutorial do assistente de criação de linguagem. 79
Figura 4.9 – Diálogos de ajuda da IDE com a linguagem Didática em Português ativa. . 81
Figura 4.10 – Página Sobre. 82
Figura 4.11 – Página inicial da plataforma em suas duas variações de tema. 83
Figura 4.12 – Painel principal do aluno, com acesso às turmas, listas e exercícios. 84
Figura 4.13 – Detalhamento de uma turma do aluno. 85
Figura 4.14 – Lista de exercícios sob a ótica do aluno. 85
Figura 4.15 – Painel principal do professor. 86
Figura 4.16 – Detalhamento de uma turma do professor. 86
Figura 4.17 – Listas de exercícios vinculadas à turma. 87
Figura 4.18 – Preparo de uma lista de exercícios pelo professor: listagem das listas existentes e detalhe de uma lista. 88
Figura 4.19 – Conclusão da lista de exercícios: vinculação dos enunciados e publicação para a turma. 89

Figura 4.20 – Criação de enunciados pelo professor: listagem dos exercícios existentes e formulário de cadastro. 90
Figura 4.21 – Acompanhamento dos enunciados pelo professor: detalhamento do exercício e submissões recebidas. 91
Figura 4.22 – Etapas 1 e 2 do assistente de personalização da linguagem dinâmica. 92
Figura 4.23 – Etapa 3 do assistente, dedicada às palavras-chave de controle de fluxo. . . . 93
Figura 4.24 – Etapa 4 do assistente, dedicada à substituição de operadores por palavras. . 95
Figura 4.25 – Etapa 5: personalização dos literais booleanos. 96
Figura 4.26 – Etapa 6 do assistente: definição dos delimitadores de bloco e do terminador de instrução. 97
Figura 4.27 – Pré-visualização final da linguagem personalizada, exibida ao término do assistente. 98

LISTA DE TABELAS

Tabela 2.1 – Comparativo entre as ferramentas analisadas e a plataforma proposta, segundo os três eixos de análise. 40
Tabela 3.1 – Elementos da linguagem passíveis de redefinição pelo usuário. 43
Tabela 3.2 – Requisitos funcionais da plataforma. 47
Tabela 3.3 – Requisitos não funcionais da plataforma. 48
Tabela 3.4 – Cronograma de desenvolvimento do trabalho de conclusão de curso. 49
Tabela 4.1 – Configurações predefinidas disponíveis no seletor da IDE. 60
Tabela 4.2 – Ocorrências reportadas pelo axe no recorte do assistente. 75
Tabela 4.3 – Escopo dos casos automatizados do pacote E2E. 101
Tabela 4.4 – Blocos do instrumento PA-01, procedência e número de itens. 104

SUMÁRIO

1	**INTRODUÇÃO**	**13**
1.1	Justificativa	14
1.2	Objetivos	15
1.2.1	Objetivo Geral	15
1.2.2	Objetivos Específicos	15
1.3	Estrutura do Trabalho	16
2	**REFERENCIAL TEÓRICO E TRABALHOS RELACIONADOS**	**17**
2.1	Compiladores	17
2.1.1	Compiladores e seus Fundamentos	17
2.1.2	O Modelo de Análise e Síntese (Fases da Compilação)	18
2.1.3	A Análise Léxica e o Reconhecimento de *Tokens*	20
2.1.4	Estratégias de Análise Sintática: *Top-Down* e *Bottom-Up*	21
2.1.5	Análise Semântica e Verificação de Tipos	23
2.1.6	Geração de Código Intermediário	24
2.1.7	Otimização de Código e Eficiência Computacional	26
2.1.8	Geração de Código Objeto e Mapeamento de Arquitetura	27
2.1.9	Execução de Código de Máquina	28
2.2	Engenharia de Software	29
2.2.1	Desenvolvimento Iterativo e Incremental e *Extreme Programming*	29
2.2.2	Testes Automatizados e *Test-Driven Development* (TDD)	30
2.2.2.1	Testes de ponta a ponta no front-end	31
2.2.3	Arquitetura Limpa e Separação de Responsabilidades	32
2.2.4	Qualidade de Código e *Clean Code*	33
2.2.5	O *Framework* Next.js e sua Arquitetura	34
2.2.6	*Design* de Experiência do Usuário (UX) e Interface do Usuário (UI)	34
2.3	Softwares Educativos e Tecnologias de Apoio à Aprendizagem	35
2.4	Trabalhos Relacionados	36
2.4.1	Ambientes Voltados à Iniciação em Programação	36
2.4.2	Projetos Didáticos de Construção de Compiladores	38

2.4.3 Análise Crítica e o Diferencial da Solução Proposta 40
3 PROPOSTA 42
4 METODOLOGIA 50
4.1 Modelagem da Arquitetura da Plataforma 52
4.2 Interpretador 55
4.3 Customização Dinâmica de *Tokens* e Sintaxe 55
4.3.1 Linguagens predefinidas e identidade da configuração 59
4.4 Edição de Código e Sistema de Arquivos Virtual 62
4.5 Execução das Análises Léxica e Sintática a partir da IDE 63
4.6 Execução do Código Intermediário e *Feedback* no Terminal 67
4.7 Sistema de Correção e Avaliação Automatizada 69
4.7.1 Autoria de exercícios e acompanhamento de turmas 69
4.7.2 Resolução de exercícios e linguagem obrigatória 70
4.7.3 Correção de submissões pelo professor 71
4.8 Acessibilidade, Responsividade e Finalização de Fluxos 73
4.8.1 Verificação técnica do assistente 73
4.8.2 Internacionalização da interface 75
4.9 Recursos de Orientação ao Usuário 77
4.9.1 Tutoriais guiados 77
4.9.2 Exemplo da linguagem e regras do compilador 80
4.9.3 Página Sobre 80
4.10 Apresentação das Telas da Plataforma 80
4.11 Estratégia de Testes Automatizados 98
4.11.1 Testes de ponta a ponta da interface 100
4.11.2 Integração contínua e diagnóstico de falhas 101
4.11.3 Resultado da execução local 102
4.12 Proposta de Protocolo de Avaliação 103
4.12.1 Quem responde e o que é possível concluir 103
4.12.2 Composição do instrumento 104
4.12.3 Itens sobre a interface e a personalização 104
4.12.4 Escalas usadas e cálculo dos escores 106
4.12.5 Participantes, procedimento e questões éticas 107

4.12.6 Limites de interpretação e continuidade 107
4.12.7 Continuidade da verificação técnica 108
5 RESULTADOS DA AVALIAÇÃO 109
5.1 Composição do Painel 109
5.2 Assistente de Personalização da Linguagem 109
5.3 Uso do Ambiente Virtual de Aprendizagem 109
5.4 Usabilidade e Experiência de Uso 109
5.5 Potencial Pedagógico e Intenção de Uso 109
5.6 Riscos e Objeções Levantados pelo Painel 109
5.7 Respostas Abertas 109
5.8 Síntese e Confronto com os Objetivos 109
6 CONSIDERAÇÕES FINAIS 110
6.1 Retomada dos Objetivos 110
6.2 Limitações 111
6.3 Encaminhamentos 111
REFERÊNCIAS 113
A APÊNDICE A – PLANO DE TRABALHO: DIVISÃO DE RESPONSABILIDADES 116
A.1 Atividades Desenvolvidas em Regime Colaborativo 116
A.2 Eixos de Aprofundamento Individual 116
A.2.1 Igor Lamoia Queiroz (este documento) 116
A.2.2 Victor de Souza Vilela da Silva (documento complementar) 117

1 INTRODUÇÃO

O presente Trabalho de Conclusão de Curso foi desenvolvido em cooperação técnico-científica, em coautoria com o acadêmico Victor de Souza Vilela da Silva, do Curso de Engenharia de Computação do Centro Federal de Educação Tecnológica de Minas Gerais, Unidade Leopoldina. Esta pesquisa faz parte de um estudo complementar e interdependente intitulado *Interpretador Dinâmico para Ensino de Programação: Implementação de análise léxica e sintática, geração e interpretação de código intermediário em ambiente web*, de autoria do referido coautor e submetido ao mesmo curso desta Instituição. A divisão das atividades comuns e dos eixos individuais de aprofundamento é descrita no Apêndice A.

Embora este texto e o documento complementar acima identificado compartilhem a mesma base teórica geral e o mesmo objeto de estudo, isto é, uma plataforma educacional web formada por um interpretador dinâmico integrado a uma IDE com gramática configurável pelo usuário, as abordagens diferem em escopo e ênfase metodológica. Enquanto o documento complementar, de autoria de Victor de Souza Vilela da Silva, se concentra no núcleo de tradução, examinando as fases de análise léxica, análise sintática, geração de código intermediário e execução pelo interpretador, este texto trata do Ambiente de Desenvolvimento Integrado: sua arquitetura *front-end*, a criação e implementação do assistente de personalização da linguagem, a experiência de uso da plataforma e a preparação de uma avaliação futura com estudantes. Para uma compreensão completa do sistema proposto, recomenda-se a leitura dos dois documentos.

O ensino introdutório de programação envolve dificuldades de compreensão e aplicação de conceitos, discutidas no mapeamento sistemático de Souza, Batista e Barbosa (2016). Este trabalho enfoca uma dessas dificuldades: lidar simultaneamente com a construção do algoritmo e com as convenções de escrita da linguagem. Os experimentos de Stefik e Siebert (2013) indicam que escolhas de sintaxe afetam o desempenho de iniciantes, motivando investigar outras formas de apresentação dos comandos. Esse resultado não demonstra, por si só, que a personalização proposta neste projeto reduza reprovação, evasão ou carga cognitiva.

Neste trabalho, **dinâmico** designa a configuração recebida pelo núcleo a cada nova análise e execução: palavras-chave, operadores em forma de palavra, literais booleanos e delimitadores podem ser redefinidos, e o usuário pode selecionar modos de terminação de instruções, blocos, tipagem e arranjos já implementados. As escolhas são aplicadas sem recompilar a aplicação web; não permitem acrescentar produções gramaticais arbitrárias nem mudar a semântica das

operações durante a execução de um programa. A Tabela 3.1 delimita essas possibilidades. O núcleo compila o código-fonte para instruções intermediárias e as executa por interpretação; a IDE oferece a interface para configurar e acionar esse processo.

A pergunta que orienta esta pesquisa nasce dessa observação: **em que medida uma plataforma de programação cuja sintaxe possa ser redefinida pelo próprio estudante contribui para reduzir a barreira sintática enfrentada no primeiro contato com a programação, deslocando o foco da forma para o conteúdo do raciocínio computacional?**

Trabalha-se com a hipótese, ainda não submetida a verificação empírica, de que permitir ao estudante escolher quais palavras servirão como comandos, como os blocos serão delimitados, como os operadores serão escritos e se o terminador de instrução será obrigatório ou opcional possa reduzir a carga cognitiva exigida no aprendizado simultâneo da lógica e da forma. A expectativa é que o exercício de adaptar a linguagem ao próprio repertório converta a sintaxe, antes vivida como obstáculo, em objeto consciente de escolha pedagógica.

Cumpre delimitar, desde já, o alcance deste trabalho diante dessa pergunta. O que aqui se apresenta é a construção do artefato que torna a investigação possível e uma proposta de protocolo com que ela poderá ser conduzida. A aplicação desse protocolo junto a estudantes, que permitiria responder empiricamente à pergunta formulada, não integra o escopo deste documento e é registrada como encaminhamento futuro.

1.1 Justificativa

A justificativa parte de uma comparação delimitada entre ferramentas. O Portugol Studio utiliza comandos em português e oferece depuração e inspeção de variáveis (UNIVALI, 2026). O Quorum investiga decisões de linguagem com base em evidências e oferece recursos de acessibilidade (QUORUM, 2025). O Beecrowd apoia a prática por submissões avaliadas automaticamente (Beecrowd, 2026). Essas abordagens já tratam dificuldades de iniciantes, mas não têm como foco o mesmo assistente de configuração de vocabulário e de variantes sintáticas proposto aqui.

A contribuição deste trabalho é integrar esse assistente a uma IDE web com editor, terminal, visualização de *tokens* e código intermediário, depuração e recursos de gestão de exercícios. A comparação da Seção 2.4 explicita os critérios e os limites desse recorte, sem generalizar seus resultados para todos os ambientes educacionais. A efetividade pedagógica da

combinação permanece uma hipótese, cuja avaliação requer dados de uso e aprendizagem.

1.2 Objetivos

A seguir são apresentados o objetivo geral que orienta a investigação e os objetivos específicos que viabilizam o seu alcance.

1.2.1 Objetivo Geral

Projetar, implementar e integrar uma IDE web ao núcleo de tradução e execução configurável, oferecendo ao estudante iniciante recursos para personalizar a linguagem e experimentar algoritmos no navegador.

1.2.2 Objetivos Específicos

Para que o objetivo geral seja efetivamente alcançado, as seguintes metas operacionais foram fixadas:

- Especificar quais elementos da linguagem serão passíveis de redefinição pelo usuário, delimitando o conjunto configurável de palavras-chave, operadores, literais, delimitadores de bloco e terminador de instrução;
- Projetar e implementar a IDE web, contemplando o assistente de personalização da linguagem, a edição de código, a execução no próprio ambiente e a apresentação de *feedback* ao usuário;
- Integrar a IDE ao interpretador dinâmico, de modo que as análises léxica e sintática e a execução do código intermediário sejam acionadas a partir do código produzido no editor;
- Disponibilizar a plataforma em ambiente web, com os fluxos de uso concluídos e com atenção a requisitos de acessibilidade e responsividade;
- Propor um protocolo de avaliação da plataforma, estabelecendo os instrumentos de coleta, as métricas e os critérios que permitirão, em etapa posterior, aferir sua contribuição pedagógica.

1.3 Estrutura do Trabalho

O restante deste trabalho está organizado da seguinte maneira. O Capítulo 2 apresenta o referencial teórico que sustenta o projeto, reunindo os fundamentos de construção de compiladores, as práticas associadas ao desenvolvimento de aplicações web e as iniciativas educacionais relacionadas. O Capítulo 3, por sua vez, expõe a proposta do trabalho e o planejamento de suas duas fases. Por fim, o Capítulo 4 descreve a modelagem realizada e a implementação consolidada até o encerramento desta etapa, contemplando a arquitetura da plataforma, a personalização dinâmica da linguagem, a integração com o interpretador e a estratégia de testes adotada.

O Capítulo 6 retoma os objetivos, apresenta os resultados técnicos e as limitações e delimita os trabalhos futuros. A Seção 4.12 descreve o instrumento de avaliação, dirigido a um painel de especialistas e ainda não aplicado. O Capítulo 5 está reservado à análise dos dados.

2 REFERENCIAL TEÓRICO E TRABALHOS RELACIONADOS

Este capítulo apresenta o referencial teórico que fundamenta o desenvolvimento da plataforma. A Seção 2.1 discute a construção de compiladores e interpretadores, percorrendo as fases de análise léxica, sintática e semântica, a geração de código intermediário e objeto, as estratégias de otimização e a execução do código de máquina. A Seção 2.2 reúne as práticas de engenharia de software que orientaram o projeto: as metodologias ágeis de desenvolvimento, os testes automatizados, a separação de responsabilidades em camadas, as práticas de qualidade de código, a arquitetura do *framework* Next.js e os princípios de Experiência do Usuário (UX) e Interface do Usuário (UI). Em seguida, são apresentados os softwares educativos e as tecnologias de apoio à aprendizagem e, por fim, os trabalhos relacionados, cuja análise crítica fundamenta o diferencial da solução proposta.

2.1 Compiladores

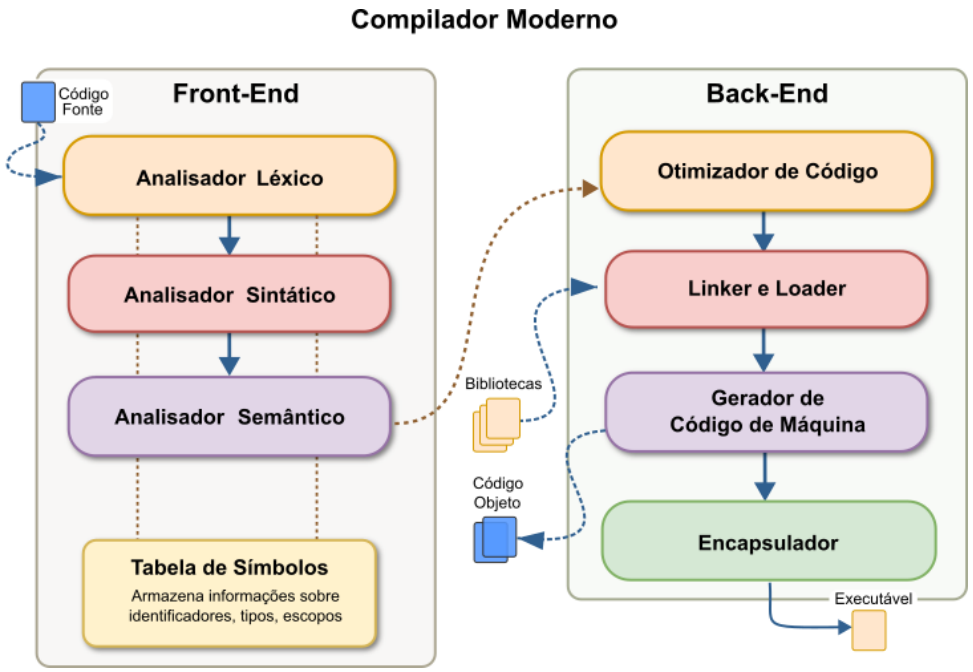
Esta seção percorre os fundamentos de compiladores que embasam o núcleo de tradução implementado neste trabalho, desde a definição de compilador e de suas fases até a geração de código intermediário.

2.1.1 Compiladores e seus Fundamentos

Um compilador é um programa cuja função é ler um código-fonte, escrito em uma linguagem específica, e traduzi-lo para um programa equivalente em uma linguagem-alvo. No modelo moderno de compilação, esse processo é estruturado em duas etapas principais e complementares: a análise e a síntese. A etapa de análise, comumente chamada de *front-end*, divide o programa original em suas partes e gera uma representação intermediária do código. Por outro lado, a etapa de síntese, conhecida como *back-end*, utiliza essa representação intermediária para construir e formatar o programa final desejado (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Para facilitar a compreensão dessa separação lógica e visualizar a evolução da transformação do código, observe a Figura 2.1.

Figura 2.1 – Arquitetura de um compilador moderno.



Fonte: Alcantara (2024).

De modo conceitual, a operação de um compilador ocorre por meio de fases sequenciais, nas quais cada etapa tem o papel de converter o programa de um nível de abstração para outro. O núcleo de análise é formado pelas fases iniciais, que englobam as análises léxica, sintática e semântica. A partir desse ponto, o fluxo do compilador avança para as etapas de síntese, que englobam a geração do código intermediário, a otimização desse código e, por fim, a geração do código-alvo (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

A tabela de símbolos é gerenciada ao longo de todas essas fases. Trata-se de uma estrutura de dados muito importante para o compilador, pois mapeia todos os identificadores usados no programa fonte. Ao coletar e armazenar informações sobre os atributos de cada identificador, essa tabela permite recuperar escopos rapidamente e realizar a validação semântica durante a tradução (AHO; SETHI; ULLMAN, 1995; PRICE; TOSCANI, 2001).

2.1.2 O Modelo de Análise e Síntese (Fases da Compilação)

De modo geral, a tradução é normalmente dividida em duas etapas principais: a análise e a síntese. No estágio de análise (*front-end*), o foco está em interpretar o programa original, dividindo-o em componentes menores para gerar uma abstração chamada representação intermediária. Por outro lado, o estágio de síntese (*back-end*) converte essa representação no

programa final pretendido. Essa distinção arquitetural entre *front-end* e *back-end* funciona como uma camada de abstração que simplifica a portabilidade, permitindo que o compilador suporte diversas linguagens e diferentes plataformas de hardware com maior agilidade (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Para lidar com a complexidade da tradução de linguagens de alto nível, o fluxo operacional é dividido em unidades lógicas menores, chamadas de fases. Cada fase é responsável por uma transformação específica na representação do código. Na prática, essas fases podem ser agrupadas em "passagens" (*passes*), estratégias que percorrem o código para otimizar a gerência de memória e o desempenho do sistema (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

As principais etapas que integram um compilador são detalhadas a seguir:

- **Análise Léxica (*Scanner*):** É a primeira etapa da compilação e processa o código-fonte caractere a caractere. Sua função é converter o fluxo de texto em uma sequência de unidades lógicas conhecidas como *tokens*, removendo elementos irrelevantes para o processamento, como comentários e espaços redundantes (COOPER; TORCZON, 2012; NYSTROM, 2021).

- **Análise Sintática (*Parser*):** Utiliza a sequência de *tokens* para validar a conformidade gramatical do programa em relação às regras da linguagem. O *parser* organiza esses elementos em estruturas hierárquicas (as árvores sintáticas ou de derivação), empregando métodos descendentes (*top-down*) ou redutivos (*bottom-up*) (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

- **Análise Semântica:** Esta fase vai além da forma gramatical para garantir que as instruções tenham coerência lógica e operacional. Uma atividade importante aqui é a checagem de tipos (*type checking*), que valida se a interação entre operadores, variáveis e funções respeita as definições da linguagem (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

- **Geração de Código Intermediário:** Concluídas as análises, o sistema produz uma versão do código que é independente da arquitetura final. O "código de três endereços" é um formato recorrente nesta etapa, servindo para reduzir a complexidade entre a abstração

do alto nível e o rigor das instruções de máquina (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

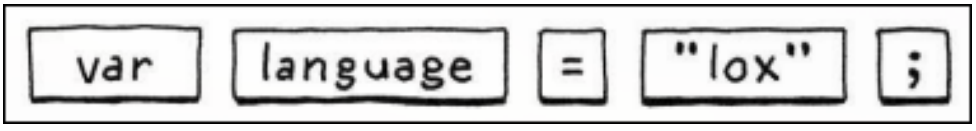
- **Otimização de Código:** Dedica-se ao refinamento do código intermediário, buscando torná-lo mais eficiente. O objetivo principal é reduzir o tempo de processamento e a ocupação de memória sem alterar o comportamento funcional do programa (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).
- **Geração de Código Objeto:** Marca o encerramento da fase de síntese, na qual a representação intermediária é mapeada para a linguagem alvo (seja *assembly* ou código de máquina). Esta etapa exige precisão na escolha das instruções nativas e cuidado na gestão dos registradores e endereços de memória do processador (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Durante todo o ciclo, dois componentes dão suporte contínuo: o gerenciador da Tabela de Símbolos e o Tratador de Erros. A tabela de símbolos centraliza o registro de identificadores e seus metadados (como tipo e escopo), garantindo acesso rápido às informações em qualquer fase. Já o módulo de erros monitora falhas léxicas, sintáticas e lógicas, aplicando rotinas de recuperação que permitem ao compilador prosseguir com a análise e identificar o maior número possível de inconsistências em uma única execução (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

2.1.3 A Análise Léxica e o Reconhecimento de *Tokens*

Conhecida tecnicamente como *scanner*, a análise léxica representa o estágio inicial do pipeline de compilação. Sua principal responsabilidade é percorrer o código-fonte caractere por caractere, processando essa entrada bruta para agrupar elementos em unidades com significado para a gramática da linguagem, chamadas de lexemas (conforme ilustrado na Figura 2.2) (COOPER; TORCZON, 2012; NYSTROM, 2021).

Figura 2.2 – Exemplificação de agrupamento de caracteres em lexemas.



Fonte: Nystrom (2021, p. 43).

Ao mesmo tempo em que organiza esses elementos, o analisador léxico realiza uma "limpeza" no fluxo de dados. Ele identifica e descarta componentes que, embora úteis ao programador, são irrelevantes para a tradução lógica, como comentários, tabulações e espaços em branco excedentes (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

É importante, contudo, não confundir os conceitos de lexema e *token*. O lexema refere-se à sequência literal de caracteres encontrada no texto original (a *string* propriamente dita, como "if", "soma" ou "42"). Já o *token* é um objeto abstrato produzido pelo compilador para representar esse lexema. O *token* carrega a categoria sintática da unidade (identificador, operador, palavra-chave) e, frequentemente, metadados adicionais, como o valor convertido e as coordenadas de linha e coluna, importantes para o rastreio de erros em fases posteriores (COOPER; TORCZON, 2012; NYSTROM, 2021).

Para que o reconhecimento desses padrões ocorra de forma rigorosa, a micro-sintaxe da linguagem é definida por meio de expressões regulares. Esse modelo matemático permite descrever, de maneira concisa, as regras de formação de cada categoria de *token*, definindo, por exemplo, quais combinações de símbolos formam um número real ou um nome de variável válido dentro daquela linguagem (PRICE; TOSCANI, 2001; COOPER; TORCZON, 2012).

Contudo, expressões regulares são ferramentas de especificação, não de execução. Para que o computador processe essas regras, elas são convertidas em autômatos finitos. Um autômato funciona como uma máquina de estados: ele consome a entrada caractere por caractere, transitando entre estados lógicos até atingir um "estado de aceitação". Quando isso ocorre, o lexema é validado e o *token* correspondente é enviado para o analisador sintático (PRICE; TOSCANI, 2001; COOPER; TORCZON, 2012).

A implementação baseada em autômatos garante que a análise léxica seja eficiente, operando em tempo linear em relação ao volume de código. Na prática, desenvolvedores utilizam ferramentas de geração automática, como o *Lex*, que transformam definições baseadas em expressões regulares diretamente em código otimizado de autômatos finitos determinísticos (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

2.1.4 Estratégias de Análise Sintática: *Top-Down* e *Bottom-Up*

A etapa de análise sintática, comumente chamada de *parsing*, tem a tarefa de validar se a sequência de *tokens* gerada previamente está de acordo com as regras gramaticais da linguagem

de origem. Para processar essa estrutura e gerar a árvore de derivação, o sistema utiliza algoritmos baseados em duas abordagens principais: a análise descendente (*top-down*), que atua da raiz para as folhas, e a análise ascendente (*bottom-up*), que opera no sentido inverso (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

I) Análise Sintática Descendente (*Top-Down*): Esta abordagem busca reconstruir a árvore sintática partindo do símbolo inicial (raiz) até alcançar os elementos terminais (folhas) que compõem o código de entrada. Durante esse fluxo, o analisador expande de forma sistemática o símbolo não-terminal posicionado mais à esquerda, utilizando as regras de produção para realizar o que se conhece tecnicamente como derivação à esquerda (*leftmost derivation*) (AHO; SETHI; ULLMAN, 1995; PRICE; TOSCANI, 2001).

Dentro dessa categoria, destacam-se os métodos recursivos descendentes e os preditivos baseados em tabelas, como o LL(1). O uso de analisadores por descida recursiva é frequente em projetos reais devido à sua legibilidade, simplicidade de implementação manual e eficiência no tratamento e diagnóstico de erros. Entretanto, para garantir um processamento linear e evitar o *backtracking* (retrocesso), a gramática precisa ser cuidadosamente preparada: ela não deve apresentar recursão à esquerda e precisa ser devidamente fatorada. Isso assegura que o analisador consiga decidir qual regra aplicar consultando apenas o próximo *token* da entrada (COOPER; TORCZON, 2012; NYSTROM, 2021).

II) Análise Sintática Ascendente ou Redutiva (*Bottom-Up*): Diferente do modelo anterior, a análise ascendente inicia o reconhecimento a partir dos *tokens* individuais (folhas) e progride estruturalmente até atingir o símbolo inicial da gramática (raiz) (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

O termo "redutiva" caracteriza bem o funcionamento deste algoritmo: o analisador identifica padrões na pilha que correspondem ao lado direito de uma regra gramatical (denominados *handles*) e os substitui (reduz) pelo respectivo símbolo não-terminal do lado esquerdo. Esse procedimento resulta em uma derivação à direita realizada de forma reversa (AHO; SETHI; ULLMAN, 1995; PRICE; TOSCANI, 2001).

Os analisadores da família LR (como SLR, LALR e o LR Canônico) representam o que há de mais robusto nesta categoria, operando por meio de um sistema de empilhamento e redução (*shift-reduce*) controlado por tabelas de estados. Uma vantagem significativa sobre a técnica descendente é a flexibilidade, pois os métodos *bottom-up* lidam nativamente com recursão à

esquerda e suportam quase todas as construções de gramáticas livres de contexto (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Contudo, a criação manual das tabelas de transição para esses analisadores envolve alta complexidade matemática. Por esse motivo, a prática comum não envolve a codificação manual do *parser*, mas sim o uso de ferramentas de geração automática, como o YACC, que traduzem a gramática formal diretamente em código funcional (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

2.1.5 Análise Semântica e Verificação de Tipos

Após a verificação gramatical feita pela análise sintática, o compilador entra na fase de análise semântica. Segundo Price e Toscani (2001), esta etapa tem como objetivo principal garantir que as estruturas e construções do programa, embora já validadas sintaticamente, façam sentido lógico para a posterior execução do código. Enquanto o *parser* (analisador sintático) assegura que a forma do código está correta, o analisador semântico vai além, inspecionando o que as peças do programa realmente significam de forma combinada (NYSTROM, 2021). Nesse cenário, uma de suas atividades fundamentais é a resolução de associações, na qual o compilador verifica todas as variáveis mencionadas e determina exatamente a qual declaração de escopo cada identificador se refere (NYSTROM, 2021).

Outro componente central da análise semântica é a verificação de tipos (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012). Nesta etapa, o compilador utiliza a estrutura hierárquica gerada previamente (a árvore de derivação) em conjunto com as informações coletadas na tabela de símbolos para checar se cada operador da linguagem está recebendo os operandos que lhe são legalmente permitidos (AHO; SETHI; ULLMAN, 1995). A análise semântica atua inferindo tipos para as expressões e detectando situações nas quais os valores são interpretados incorretamente (COOPER; TORCZON, 2012). Quando permitido pela especificação da linguagem, é também nesta fase que o compilador realiza conversões de tipo embutidas (coerções), como a transformação implícita de um valor inteiro para ponto flutuante antes de realizar uma adição entre tipos mistos (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Os erros semânticos, por sua vez, são falhas peculiares, pois ocorrem em programas que podem possuir uma estrutura gramatical perfeitamente bem-formada, mas que violam as

regras de significado, consistência ou contexto da linguagem de programação (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012). O tratamento de erros semânticos é focado em capturar inconsistências que incluem, principalmente:

- **Incompatibilidade de Tipos de Operandos:** Ocorre quando um operador é aplicado a um ou mais operandos cujos tipos de dados são conflitantes e não possuem regras de conversão implícita ou explícita (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012). Um exemplo prático na maioria das linguagens de programação fortemente tipadas é a tentativa inválida de se aplicar um operador aritmético binário a uma *string* (texto) e a um booleano, ou tentar adicionar um tipo de precisão dupla a um tipo complexo de forma irregular (COOPER; TORCZON, 2012).
- **Uso de Variáveis Não Declaradas ou Falhas de Escopo:** Este erro se manifesta quando o código faz referência a uma variável, função ou objeto que não foi previamente definido no programa, ou cuja declaração se encontra em um escopo léxico distinto e inacessível naquele momento (COOPER; TORCZON, 2012; NYSTROM, 2021). A utilização de uma variável local fora de seu bloco restrito, por exemplo, acarretará uma falha semântica de identificador não reconhecido ou indefinido (NYSTROM, 2021).
- **Inconsistência de Aridade e Estruturas:** Acontece comumente em construções que exigem conformidade de números, como o número incorreto de argumentos (COOPER; TORCZON, 2012). Exemplos incluem uma invocação de função ou chamada de procedimento na qual a quantidade de parâmetros passados difere da quantidade estipulada em sua definição original (COOPER; TORCZON, 2012; NYSTROM, 2021). Igualmente, configura-se um erro semântico referenciar um *array* informando um número de dimensões diferente do limite (ou *rank*) estipulado em sua declaração (COOPER; TORCZON, 2012).

2.1.6 Geração de Código Intermediário

A fase de geração de código intermediário funciona como um mediador essencial entre a análise (*front-end*) e a síntese (*back-end*) de um sistema de compilação. Uma vez validadas as estruturas e a semântica do programa original, o compilador elabora uma representação interna explícita, projetada para ser agnóstica tanto em relação à linguagem de origem quanto à arquitetura de destino. Essa estratégia de tradução em múltiplos estágios é fundamental para

decompor a complexidade de converter abstrações de alto nível em comandos de baixo nível de forma controlada. Além disso, essa camada de isolamento facilita a aplicação de técnicas de otimização e potencializa a portabilidade, visto que uma única representação interna pode ser mapeada para diferentes plataformas de *hardware* (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

Dentre os formatos de representação linear mais difundidos na literatura técnica, destaca-se o código de três endereços, que opera como um tipo de linguagem de montagem (*assembly*) abstrata e universal. Nessa convenção, operações lógicas e aritméticas complexas são fragmentadas em instruções elementares contendo, no máximo, três operandos. A distinção crucial entre este formato e o código objeto final é que a representação intermediária ignora restrições físicas, como a gestão específica de registradores e o mapeamento direto de endereços de memória (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Para exemplificar o conceito, considere uma expressão de atribuição como $A := X + Y * Z$. O compilador desmembra essa lógica em instruções de *pseudo-assembly* simplificadas, utilizando variáveis temporárias para armazenar resultados parciais. O fluxo resultante em código de três endereços seguiria a sequência (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012):

1. $T_1 := Y * Z$
2. $T_2 := X + T_1$
3. $A := T_2$

A automação desse processo de conversão é viabilizada pela técnica de tradução dirigida pela sintaxe. Esse método expande o formalismo das gramáticas livres de contexto, permitindo que atributos sejam associados aos símbolos gramaticais e que ações semânticas específicas sejam vinculadas às regras de produção da linguagem (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Na implementação prática, a emissão do código intermediário ocorre de forma síncrona com a análise sintática. No momento em que o *parser* identifica uma construção gramatical legítima, ele dispara a ação semântica correspondente. Esse procedimento processa os atributos herdados ou sintetizados dos nós da árvore sintática, permitindo que os fragmentos de código

sejam gerados de maneira ordenada e coerente com a lógica original do programa (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

2.1.7 Otimização de Código e Eficiência Computacional

Posicionada estrategicamente como um estágio de refinamento no *pipeline* de compilação, a otimização busca elevar a qualidade da representação interna do software. Esse aprimoramento vai além do ganho de velocidade e da economia de memória, abrangendo também a redução do consumo energético, um requisito crítico na computação contemporânea. Durante esta fase, o compilador atua substituindo construções genéricas por equivalentes semânticos mais ágeis. Um exemplo notório é o dobramento de constantes (*constant folding*), que antecipa operações entre valores fixos ainda em tempo de compilação, eliminando o custo desse processamento durante a execução do programa (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

A legitimidade de qualquer transformação de código está condicionada a dois critérios imperativos: a preservação semântica (segurança) e a eficácia computacional (rentabilidade). O pilar da segurança determina que a lógica original do algoritmo deve permanecer inalterada, assegurando que a otimização não introduza efeitos colaterais. Já a rentabilidade exige que a alteração proporcione uma melhoria de desempenho real e mensurável, justificando o esforço computacional e o tempo adicional dedicados pelo compilador ao processo (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

Quanto à abrangência das intervenções, a otimização é classificada em quatro níveis de profundidade:

- **Otimização Local:** Restringe-se à análise de blocos básicos, isto é, sequências de instruções sem desvios de controle. Neste nível, utilizam-se frequentemente Grafos Acíclicos Dirigidos (GADs) para detectar e remover subexpressões redundantes e gerir variáveis locais de forma eficiente (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

- **Otimização Regional:** Expande o foco para conjuntos de blocos básicos, concentrando esforços prioritariamente em laços de repetição (*loops*). Dado que a maior parte da carga de processamento ocorre nestes ciclos, melhorias como o deslocamento de código invariante

geram impactos substanciais na performance global (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

- **Otimização Global (Intraprocedural):** Analisa o contexto integral de uma função ou procedimento. Por meio de estudos minuciosos do fluxo de dados, o compilador identifica oportunidades de simplificação que seriam invisíveis sob uma inspeção puramente local ou limitada (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).
- **Otimização Interprocedural:** Atua no nível mais macro, transpondo as fronteiras entre diferentes módulos e sub-rotinas. Este escopo permite manobras agressivas como o *inlining*, onde o corpo de uma função é integrado diretamente ao ponto de chamada, eliminando a sobrecarga (*overhead*) associada ao desvio de execução e à gestão de pilhas de ativação (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Na prática, a maioria das rotinas de otimização é dividida em duas etapas: uma fase de análise, dedicada ao mapeamento de dependências e do fluxo de controle, seguida por uma fase de transformação sintática, que efetivamente reescreve o código. Entre as técnicas mais usuais, destacam-se a eliminação de código morto (instruções inacessíveis ou inúteis), a simplificação de redundâncias aritméticas e a movimentação estratégica de cálculos para fora de zonas de alta repetição (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

2.1.8 Geração de Código Objeto e Mapeamento de Arquitetura

A fase de geração de código objeto (ou geração de código de máquina) representa a culminação do processo de compilação, consolidando as atividades finais do *back-end*. O propósito central desta etapa é converter a representação intermediária otimizada na linguagem nativa da plataforma de destino. Para tanto, o compilador deve assegurar não apenas a fidelidade lógica em relação ao código original, mas também o aproveitamento máximo dos recursos de *hardware* para garantir uma execução de alta performance (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

Um dos obstáculos primordiais nesta fase reside na acentuada heterogeneidade das arquiteturas de hardware. Cada família de processadores, a exemplo das arquiteturas x86, ARM ou RISC-V, possui sua própria Arquitetura do Conjunto de Instruções (ISA - *Instruction Set Architecture*). Consequentemente, as operações de baixo nível e suas respectivas codificações

binárias variam drasticamente entre diferentes sistemas, exigindo um mapeamento preciso para cada caso (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

Para mitigar a complexidade de desenvolver um compilador do zero para cada novo processador, a engenharia de software moderna adota o princípio da *retargetabilidade* (redirecionamento). Essa modularidade é viabilizada pelo papel central da representação intermediária: enquanto o *front-end* opera de forma agnóstica em relação ao hardware, a adaptação para uma nova arquitetura exige apenas a substituição ou reconfiguração do módulo gerador de código (*back-end*) (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

A síntese do código para uma máquina específica fundamenta-se em duas atividades interdependentes:

- **Seleção de Instruções:** Atua como um mecanismo de reconhecimento de padrões que mapeia o fluxo do código intermediário para a sequência de comandos da ISA do processador que apresente o menor custo computacional possível (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).
- **Alocação de Registradores:** Gerencia o conjunto limitado de registradores físicos da Unidade Central de Processamento (UCP). Esta tarefa é crítica, pois o compilador deve decidir estrategicamente quais valores devem permanecer nesses componentes de altíssima velocidade e quais devem ser transferidos para a memória principal (*spilling*), visando minimizar os atrasos de acesso (AHO; SETHI; ULLMAN, 1995; COOPER; TORCZON, 2012).

2.1.9 Execução de Código de Máquina

Ao término do processo de síntese, o que emerge é uma sucessão binária, densa e linear de operações primitivas. Diferente das abstrações hierárquicas manipuladas nas etapas iniciais da compilação, como as árvores sintáticas, o código de máquina é desprovido de estruturas de alto nível. Trata-se de uma linguagem de baixíssimo nível, constituída por instruções rudimentares que ocupam apenas alguns *bytes* e ditam ações imediatas nos circuitos: movimentação de dados entre células de memória e registradores ou a execução de operações aritméticas básicas (COOPER; TORCZON, 2012; NYSTROM, 2021).

Para que essa massa de dados binários ganhe vida, o programa deve ser transferido para a memória principal. Neste estágio, o sistema operacional, auxiliado por um *software*

carregador (*loader*), prepara o ambiente de tempo de execução (*runtime*). Este processo envolve a segmentação da memória em áreas distintas, isolando o setor de código estruturado das regiões destinadas ao armazenamento de dados (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

A partir desse momento, o comando da execução passa à Unidade Central de Processamento (UCP). O "guia" dessa jornada é um registrador especializado da arquitetura, conhecido como Contador de Programa (*Program Counter* - PC) ou Ponteiro de Instrução (*Instruction Pointer* - IP), cuja função é monitorar rigorosamente o endereço da próxima instrução a ser processada (COOPER; TORCZON, 2012; PRICE; TOSCANI, 2001).

A execução ocorre por meio de um ciclo mecânico e ininterrupto de busca e execução. O processador percorre a fita de instruções, decodifica o código de operação (*opcode*) de cada entrada para extrair seu significado semântico e, em seguida, executa a tarefa correspondente. Nesse ecossistema de baixo nível, a complexidade dos laços de repetição e desvios condicionais das linguagens de alto nível é simplificada drasticamente. O controle do fluxo é resolvido exclusivamente através de operações de salto (*jumps* ou *branches*), que alteram o fluxo ao escrever um novo endereço de memória diretamente no Contador de Programa, desviando o "foco" da UCP de um ponto do binário para outro (COOPER; TORCZON, 2012; NYSTROM, 2021).

2.2 Engenharia de Software

Esta seção reúne as práticas de engenharia de software que orientaram a construção da plataforma: a organização do trabalho em ciclos iterativos, a estratégia de testes automatizados, a separação de responsabilidades em camadas, as diretrizes de qualidade de código, a arquitetura do *framework* adotado no *front-end* e os princípios de experiência e interface do usuário.

2.2.1 Desenvolvimento Iterativo e Incremental e *Extreme Programming*

O gerenciamento clássico de projetos de software, frequentemente baseado no modelo em cascata (*waterfall*), assume que o desenvolvimento pode ser tratado como uma sequência previsível e linear de atividades. No entanto, a engenharia de software lida com sistemas intrinsecamente complexos e com alto grau de incerteza, e o engessamento de requisitos em fases iniciais costuma resultar em desperdício e na entrega de produtos que não atendem às

necessidades reais. Em contraponto a essa visão, o desenvolvimento iterativo e incremental assume a imprevisibilidade natural do processo e organiza o trabalho em ciclos curtos, nos quais o sistema cresce por acréscimos funcionais sucessivos, cada um integrado e verificado antes do início do seguinte (ADAPTWORKS, 2012; BECK; ANDRES, 2004).

Entre os métodos ágeis, o *Extreme Programming* (XP) distingue-se por concentrar suas prescrições nas práticas técnicas de construção do software, e não em papéis ou cerimônias de gestão. O método parte do princípio de que o custo da mudança pode ser mantido baixo ao longo de todo o projeto, desde que o código permaneça simples, coberto por testes e continuamente refatorado. A mudança deixa, assim, de ser tratada como desvio a ser evitado e passa a ser incorporada ao fluxo normal de trabalho (BECK; ANDRES, 2004; WILDT et al., 2015).

As práticas que sustentam esse princípio incluem o planejamento incremental, no qual o escopo é detalhado apenas no momento em que será implementado; o *design* simples, que evita antecipar estruturas ainda não exigidas; o desenvolvimento guiado por testes, que estabelece o comportamento esperado antes da implementação; a refatoração contínua, que preserva a legibilidade do código à medida que ele cresce; a integração contínua, que reduz o custo de reunir contribuições paralelas; e a propriedade coletiva do código, segundo a qual qualquer integrante pode alterar qualquer parte do sistema, sustentada pela revisão mútua entre os desenvolvedores (BECK; ANDRES, 2004; WILDT et al., 2015).

Essa combinação torna o XP particularmente adequado a equipes reduzidas, nas quais não há massa crítica para instanciar papéis especializados: os mesmos desenvolvedores acumulam as atribuições de levantamento de requisitos, projeto, implementação, teste e revisão. Nesse cenário, as práticas técnicas assumidas pelo método ocupam o lugar dos mecanismos de coordenação que métodos orientados a processo delegariam a papéis distintos (WILDT et al., 2015). As subseções seguintes detalham três dessas práticas que estruturaram diretamente a construção da plataforma: os testes automatizados e o *Test-Driven Development*, a separação de responsabilidades em camadas e as diretrizes de *Clean Code*.

2.2.2 Testes Automatizados e *Test-Driven Development* (TDD)

A garantia da qualidade de um software exige validação contínua de seus comportamentos. Em projetos modernos, a automação de testes reduz a dependência de verificação manual e fornece uma rede de segurança para alterações, refatorações e evolução

do código. Essa prevenção contra a quebra de comportamentos já consolidados é conhecida como teste de regressão; além de validar funcionalidades, os testes automatizados funcionam como documentação executável das regras de negócio do sistema (ANICHE, 2012; SILVEIRA et al., 2012).

Nesse contexto, o TDD inverte o ciclo tradicional de implementação, exigindo que o desenvolvedor escreva um teste automatizado antes da funcionalidade. O ciclo vermelho-verde-refatora consiste em criar primeiro um teste que falha, escrever o código mínimo para fazê-lo passar e, por fim, refatorar a solução mantendo o comportamento validado (ANICHE, 2012; MARTIN, 2011).

Mais do que uma técnica de verificação, o TDD também influencia o *design* de software. Ao escrever o teste primeiro, o desenvolvedor precisa pensar na interface pública, nas dependências e na responsabilidade de cada componente; dificuldades excessivas na escrita dos testes podem indicar acoplamento elevado, baixa coesão ou problemas de arquitetura (MARTIN, 2019; ANICHE, 2012).

A estratégia de testes de um sistema costuma ser distribuída em diferentes níveis de granularidade, arranjo frequentemente ilustrado pela chamada pirâmide de testes. A base é composta pelos testes de unidade, rápidos e isolados, que verificam o comportamento de pequenas partes do código. O nível intermediário abriga os testes de integração, responsáveis por garantir a correta comunicação entre componentes e serviços de infraestrutura. No topo situam-se os testes de ponta a ponta (*end-to-end*), que validam o sistema inteiro a partir da interface do usuário, ao custo de execuções mais lentas e frágeis. A recomendação decorrente é concentrar o maior volume de casos na base e reservar os níveis superiores para os fluxos mais críticos (ANICHE, 2012; SILVEIRA et al., 2012).

2.2.2.1 Testes de ponta a ponta no front-end

Os testes de ponta a ponta (E2E) complementam as verificações isoladas ao exercitar jornadas que atravessam diferentes partes da aplicação. Em uma interface web, permitem observar se navegação, componentes, serviços e persistência produzem conjuntamente o resultado esperado. Como essas verificações envolvem mais dependências e tendem a apresentar maior custo de execução e manutenção, sua adoção deve priorizar jornadas relevantes e preservar os testes de menor granularidade. Essa recomendação fundamenta a inclusão de E2E como parte de

uma estratégia de regressão, sem pressupor que todos os comportamentos devam ser verificados pela interface (VOCKE, 2018).

O Playwright Test é uma ferramenta de automação de testes de aplicações web, com executor, asserções e suporte aos navegadores Chromium, Firefox e WebKit. Sua documentação apresenta a execução em diferentes navegadores e ambientes como possibilidades de configuração; usar a ferramenta não implica, por si só, que um projeto tenha exercitado todas essas combinações (Microsoft, 2026c).

A documentação recomenda verificar comportamentos visíveis ao usuário, manter testes independentes e preferir localizadores baseados em papéis, nomes acessíveis e contratos explícitos de identificação. As asserções que aguardam uma condição, como a visibilidade de uma mensagem, ajudam a sincronizar o teste com a interface. Essas práticas orientam a escrita dos cenários e reduzem a dependência de detalhes incidentais da estrutura visual (Microsoft, 2026a).

Outra parte da estratégia é executar a suíte no processo de integração contínua, com instalação das dependências e dos navegadores e disponibilização dos relatórios de execução. A documentação do Playwright exemplifica essa configuração no GitHub Actions para eventos de envio de alterações e abertura ou atualização de solicitações de integração. Assim, a verificação passa a ser repetível no fluxo de desenvolvimento, além de permanecer disponível localmente (Microsoft, 2026b).

2.2.3 Arquitetura Limpa e Separação de Responsabilidades

O projeto arquitetural de um sistema deve separar responsabilidades em camadas com papéis independentes e bem definidos. A interface de usuário no *front-end* concentra-se na experiência de uso e na integração visual, enquanto regras de negócio, persistência e serviços de apoio devem permanecer desacoplados de detalhes periféricos de entrega. Essa separação reduz dependências indevidas e facilita testes, manutenção e evolução do software (MARTIN, 2019; SILVEIRA et al., 2012).

No contexto da plataforma, a separação entre elementos de interface, regras de customização da linguagem, execução do código e persistência de dados contribui para que cada parte possa evoluir sem comprometer as demais. Essa organização também favorece o uso de padrões de projeto e abstrações voltadas à redução de acoplamento entre os módulos (MARTIN,

2019). Os padrões efetivamente adotados na construção da plataforma, bem como os pontos do código em que foram aplicados, são enumerados na Seção 4.1.

2.2.4 Qualidade de Código e *Clean Code*

A construção de um software de maneira ágil e sustentável esbarra invariavelmente na qualidade de sua base de código. O código-fonte é a linguagem formal na qual os requisitos de um sistema são expressos e detalhados de modo que a máquina possa executá-los. No entanto, sob a pressão de prazos, é comum que as equipes produzam códigos desorganizados, resultando em uma degradação estrutural contínua que, a longo prazo, faz a produtividade despencar, podendo até mesmo inviabilizar a evolução do projeto (MARTIN, 2009; ADAPTWORKS, 2012).

Para combater essa entropia, a adoção das práticas de Código Limpo torna-se essencial. A escrita de um código limpo exige disciplina, o uso consciente de padrões e uma sensibilidade aguçada para reconhecer a desorganização e aplicar refatorações. Um código limpo deve ser elegante, simples, direto e, sobretudo, legível para os seres humanos. O profissionalismo no desenvolvimento de software pressupõe que o código não deve apenas funcionar, mas deve ser fácil de entender, alterar e estender por outros desenvolvedores (MARTIN, 2009).

Dentre as diretrizes estruturais básicas e inegociáveis para a manutenção dessa clareza na base de código, destacam-se (MARTIN, 2009):

- **Nomes Significativos:** A nomenclatura de variáveis, funções, parâmetros e classes deve revelar claramente a sua intenção e responder ao porquê de sua existência e ao que o componente faz. Deve-se evitar o uso de siglas obscuras, codificações desnecessárias ou trocadilhos que exijam mapeamentos mentais, facilitando assim a leitura fluida e a busca no código (MARTIN, 2009).

- **Funções Pequenas e Focadas:** Uma função deve ser extremamente concisa e projetada para fazer estritamente uma única coisa, mantendo um único nível de abstração. Funções extensas que realizam múltiplas tarefas, agrupam várias seções lógicas ou que misturam tratamento de erros com a lógica de negócio violam a organização e dificultam o entendimento e a testabilidade (MARTIN, 2009).

- **Uso Consciente de Comentários:** O uso de comentários costuma compensar a

incapacidade do programador de se expressar claramente através do próprio código. Eles não devem ser utilizados para encobrir ou justificar um código ruim ou confuso. A energia da equipe deve ser direcionada para tornar o código tão claro e autoexplicativo que elimine a necessidade de anotações redundantes (MARTIN, 2009).

- **Eliminação de Duplicidade e Refatoração contínua:** A repetição de código é o grande inimigo de um sistema bem desenvolvido, pois representa trabalho, risco e complexidade desnecessária. Deve-se melhorar o *design* do código de forma contínua, aplicando refatorações que tornem o sistema mais enxuto e compreensível, sem alterar o seu comportamento externo (MARTIN, 2009; ADAPTWORKS, 2012).

2.2.5 O *Framework* Next.js e sua Arquitetura

O Next.js é um *framework* baseado em React que oferece roteamento por arquivos e renderização de páginas. No *Pages Router*, adotado neste projeto, os arquivos sob `pages` definem as páginas, enquanto os arquivos sob `pages/api` implementam rotas HTTP executadas no servidor. Esse arranjo permite combinar componentes interativos no navegador com operações de servidor em uma mesma aplicação (Vercel, 2026b; Vercel, 2026a).

As rotas de API são executadas no ambiente Node.js padrão. No projeto, elas atendem, entre outros recursos, à validação de submissões, enquanto a análise e a execução interativa do código do editor ocorrem no navegador. A Seção 4.1 mostra essa distribuição. Os conceitos de *App Router* e *Server Actions* não são necessários para descrever a implementação examinada.

2.2.6 *Design* de Experiência do Usuário (UX) e Interface do Usuário (UI)

A experiência do usuário (UX) abrange a interação com um produto e seus serviços; a interface do usuário (UI) corresponde aos elementos pelos quais essa interação acontece. Usabilidade é uma qualidade da interface, relacionada à facilidade de aprendizagem e de uso, e constitui parte da experiência mais ampla (NORMAN; NIELSEN, 1998).

Neste projeto, essas noções orientam a organização da personalização em etapas, a apresentação de exemplos antes de salvar uma linguagem, a indicação de configurações inválidas e a exposição do estado de execução no terminal. A existência desses recursos pode ser inspecionada no código e na interface. Sua compreensão por iniciantes, entretanto, requer

observar tarefas de uso. A Seção 4.12 lista as decisões de interface e os itens que perguntam sobre elas. A aparência visual, sozinha, não demonstra usabilidade nem aprendizagem.

Acessibilidade da interface

A WAI-ARIA define papéis, estados e propriedades para comunicar a semântica de componentes às tecnologias assistivas. Por exemplo, `role="dialog"` identifica um diálogo; `aria-expanded` informa o estado de expansão; e `aria-labelledby` associa um elemento a seu rótulo (W3C, 2023). Esses atributos não implementam, por si só, os comportamentos de teclado. O guia de práticas ARIA recomenda verificar a interação e a compatibilidade entre navegador e tecnologia assistiva (World Wide Web Consortium, 2026).

As WCAG 2.2 fornecem critérios de sucesso que permitem organizar a verificação de acessibilidade do conteúdo web. No recorte deste trabalho, interessam operação por teclado (2.1.1), ausência de bloqueio de teclado (2.1.2), foco visível (2.4.7), contraste mínimo (1.4.3), reorganização do conteúdo (1.4.10), identificação de erros (3.3.1) e nome, papel e valor dos componentes (4.1.2) (World Wide Web Consortium, 2024). A adoção de Radix UI fornece mecanismos de semântica e foco nos componentes primitivos, mas a composição final continua dependendo de rótulos, estilos e interações definidos pela aplicação (Radix, 2026). A Seção 4.8 delimita a verificação realizada e as lacunas restantes.

2.3 Softwares Educativos e Tecnologias de Apoio à Aprendizagem

Neste trabalho, uma plataforma educativa é entendida como um conjunto de recursos para organizar atividades e apoiar a experimentação de algoritmos. As ferramentas examinadas na seção seguinte concretizam esse apoio de formas distintas: o Portugol Studio expõe a execução do programa (UNIVALI, 2026), enquanto o Beecrowd oferece submissões confrontadas com casos de teste (Beecrowd, 2026).

Na plataforma proposta, a mediação pedagógica é representada pela criação de turmas, publicação de listas e exercícios, definição de entradas e saídas esperadas e acompanhamento das submissões pelo professor. A personalização da linguagem complementa essas atividades. Esses recursos estabelecem possibilidades de uso; ganhos de compreensão, engajamento ou autonomia precisam ser investigados com estudantes e não são inferidos da presença das funcionalidades.

2.4 Trabalhos Relacionados

Esta seção tem como finalidade situar a plataforma desenvolvida neste trabalho no conjunto de soluções tecnológicas voltadas ao ensino introdutório de programação. Embora o objetivo do projeto seja apoiar a aprendizagem de programação, e não o ensino de construção de compiladores, sua própria natureza, a de uma IDE acoplada a um interpretador de gramática configurável, posiciona a proposta na fronteira entre dois universos de ferramentas: de um lado, os ambientes voltados à iniciação em programação; de outro, os projetos didáticos de construção de compiladores e de analisadores léxicos e sintáticos, cujas decisões técnicas e pedagógicas são pertinentes por dialogarem diretamente com o núcleo de tradução que sustenta esta plataforma. Por essa razão, são examinadas cinco iniciativas selecionadas intencionalmente, organizadas nas duas subseções seguintes conforme o grupo a que pertencem e observando, em cada uma delas, as escolhas de interface, os recursos pedagógicos e as estratégias de interação com o estudante. Ao final, apresenta-se uma análise comparativa que evidencia as lacunas restantes e o diferencial da abordagem proposta, que se situa justamente entre esses dois grupos.

A seleção é intencional, não sistemática: inclui ambientes de iniciação com interface ou avaliação de exercícios e projetos de construção de tradutores com documentação técnica disponível. Portugol e Quorum permitem discutir vocabulário e acessibilidade; Beecrowd, o ciclo de submissão; Laila, a edição de gramáticas em uma interface web; e *chibicc*, a implementação incremental. O quadro compara apenas esse conjunto e não demonstra ineditismo frente a todas as ferramentas existentes.

2.4.1 Ambientes Voltados à Iniciação em Programação

Reúnem-se aqui as ferramentas cujo público-alvo é o estudante em primeiro contato com a programação e cujo propósito declarado é reduzir o atrito desse contato inicial. Cada uma é examinada segundo os três eixos anunciados no início desta seção: as escolhas de interface, os recursos pedagógicos e as estratégias de interação com o estudante.

Portugol Studio e Portugol Webstudio

Escolhas de interface. Criado pelo Laboratório de Inovação Tecnológica na Educação (LITE) da Universidade do Vale do Itajaí, o Portugol Studio é distribuído como um ambiente de desenvolvimento integrado de código aberto, originalmente para instalação local. O Portugol Webstudio é outro projeto que oferece uma IDE online para Portugol, com inspiração no Portugol

Studio e implementação de execução no navegador (GADELHA, 2026). A composição da tela reproduz a organização convencional de uma IDE profissional, com editor central, painel de saída e área de inspeção, de modo que a familiaridade adquirida seja transferível a ferramentas de mercado (UNIVALI, 2026).

Recursos pedagógicos. O recurso central é a própria linguagem: construções sintáticas inspiradas em C e PHP, porém com palavras-chave integralmente traduzidas para o português brasileiro, o que evita que o aprendiz lide com dois idiomas ao mesmo tempo durante o estudo das estruturas básicas de controle de fluxo e de dados. A esse núcleo somam-se um depurador com execução passo a passo, um inspetor de variáveis sincronizado com a linha em execução e uma árvore que representa a estrutura sintática do código produzido, recursos que tornam observável o que normalmente permanece implícito para o iniciante (UNIVALI, 2026).

Estratégias de interação. A interação apoia-se na visualização do programa em funcionamento: o estudante acompanha, linha a linha, a alteração dos valores em memória, o que aproxima a execução mental do algoritmo de sua execução efetiva. O suporte nativo a bibliotecas gráficas e sonoras amplia essa estratégia ao permitir projetos lúdicos, como pequenos jogos, cuja saída visível fornece retorno imediato sobre o efeito do código escrito (UNIVALI, 2026).

Quorum: linguagem de programação baseada em evidências

Escolhas de interface. O ecossistema disponibiliza o Quorum Studio, IDE concebida desde o início para operar com leitores de tela, e não adaptada a eles posteriormente. O ambiente funciona tanto em modo *offline*, com geração de *bytecode* para a máquina virtual Java, quanto em modo executável no navegador, o que permite adotá-lo em contextos com restrições de instalação (QUORUM, 2025; LADNER, 2019).

Recursos pedagógicos. O traço distintivo do projeto é submeter as decisões de sintaxe e de semântica a verificação empírica, em vez de herdá-las por imitação ou familiaridade histórica. Os experimentos de Stefik e Siebert (2013) compararam linguagens consolidadas no mercado a uma linguagem de controle cujas palavras-chave foram geradas aleatoriamente, recurso metodológico que isolou o efeito da forma sintática sobre o desempenho de novatos. Os resultados mostram que escolhas sintáticas podem afetar o desempenho de iniciantes; não autorizam generalizar que toda substituição de símbolo por palavra reduza erros (STEFIK; SIEBERT, 2013).

Estratégias de interação. A acessibilidade orienta o projeto, que oferece formas de interação compatíveis com diferentes necessidades dos usuários. O ecossistema estende essa interação a domínios de saída variados — áudio, gráficos e robótica —, oferecendo ao estudante formas de perceber o resultado do programa que não dependem exclusivamente da leitura visual de texto (QUORUM, 2025; LADNER, 2019).

Beecrowd: prática de algoritmos e avaliação automatizada

Escolhas de interface. Antes denominada URI Online Judge, a Beecrowd é uma plataforma inteiramente web, organizada em torno de um catálogo de problemas navegável por tema e por nível de dificuldade. O fluxo considerado nesta comparação é a submissão de código a um juiz de problemas, com retorno de veredito; a documentação consultada orienta a preparação de entradas e saídas e a interpretação dos resultados (Beecrowd, 2026).

Recursos pedagógicos. O acervo contempla milhares de problemas categorizados, abrangendo desde estruturas básicas de seleção e repetição até tópicos avançados como grafos e programação dinâmica, e admite submissões em diversas linguagens consolidadas no mercado, incluindo C++, Python, Java e JavaScript. A graduação de dificuldade é o principal recurso didático, pois permite ao estudante progredir por incrementos e ao professor selecionar conjuntos coerentes com o momento da disciplina (Beecrowd, 2026).

Estratégias de interação. A dinâmica articula-se em torno de um ciclo curto de submissão e *feedback*: o estudante lê o enunciado, redige uma solução e a envia ao sistema, que a executa contra um conjunto fechado de casos de teste e retorna um veredito como "*Accepted*", "*Wrong Answer*" ou "*Presentation Error*". Cabe observar que esse retorno é binário quanto ao resultado e não localiza o erro no código. Combinado a elementos de gamificação como *rankings* públicos, fóruns e contagem de submissões aceitas, o ciclo sustenta um regime de prática deliberada que tende a manter o estudante em posição ativa (Beecrowd, 2026).

2.4.2 Projetos Didáticos de Construção de Compiladores

Reúnem-se aqui as iniciativas voltadas ao ensino da construção de tradutores. Embora se dirijam a um público mais avançado, suas decisões técnicas e pedagógicas dialogam diretamente com o núcleo de tradução que sustenta a plataforma proposta, e por isso são examinadas segundo os mesmos três eixos.

Laila: gerador de analisadores léxicos e sintáticos *online*

Escolhas de interface. A ferramenta oferece um ambiente de desenvolvimento inteiramente executado no navegador, construído em JavaScript com o *framework* VueJS e apoiado na biblioteca CodeMirror, responsável pelo destaque de sintaxe e pela sinalização *inline* de inconsistências. A camada de *back-end* é uma API REST em Python que coordena a execução de FLEX e Bison no servidor. Essa escolha responde a um problema concreto de interface com a disciplina: a instalação coordenada desses geradores junto a uma cadeia compatível do GCC consome parte significativa da carga horária de laboratório, sobretudo em sistemas operacionais que não derivam do Linux (SILVA; PEREIRA; LIMA, 2021).

Recursos pedagógicos. O recurso didático central é a possibilidade de o estudante especificar gramáticas, redigir analisadores léxicos e sintáticos e executar os experimentos sem qualquer dependência local, concentrando o tempo de aula no objeto de estudo e não em sua configuração. O servidor captura comportamentos anômalos, como laços infinitos ou erros de compilação, e devolve os diagnósticos correspondentes ao cliente (SILVA; PEREIRA; LIMA, 2021).

Estratégias de interação. A interação organiza-se em um ciclo curto entre edição, submissão e retorno do resultado. O estudo relatado reúne 17 respostas de alunos e ex-alunos da disciplina de Compiladores do IFCE, Campus Aracati, e apresenta percepções favoráveis sobre o uso da ferramenta. Esses relatos não constituem uma medida controlada de ganho de aprendizagem (SILVA; PEREIRA; LIMA, 2021).

***chibicc*: abordagem incremental para o ensino de compiladores**

Escolhas de interface. Este é o caso em que a análise por eixos revela a maior distância em relação às demais ferramentas: o *chibicc* não possui interface própria. Trata-se de um repositório de código-fonte, cuja fruição pressupõe editor, cadeia de compilação e sistema de controle de versão já instalados e dominados pelo estudante. Toda a mediação recai sobre o histórico de *commits* do projeto (UEYAMA, 2020).

Recursos pedagógicos. O traço pedagógico mais relevante está na organização incremental. Em vez de expor o aprendiz, desde o princípio, a uma arquitetura completa com múltiplos passes e estruturas internas elaboradas, o projeto propõe uma trajetória gradual em que cada etapa adiciona uma única capacidade gramatical ou semântica ao compilador. O ponto de partida é um tradutor mínimo que reconhece apenas um número como entrada válida; sobre essa base, são integrados, em iterações sucessivas, suporte a expressões, declarações, controle de

fluxo, ponteiros, estruturas e demais elementos do C11. Apesar do enfoque didático, o resultado cobre parte expressiva da especificação C11, sendo capaz de compilar, sem adaptações, projetos de complexidade considerável, como o sistema de controle de versão Git e o gerenciador de banco de dados SQLite (UEYAMA, 2020).

Estratégias de interação. Não há interface gráfica de execução assistida: a interação consiste na leitura do código de cada etapa e em sua reimplementação pelo estudante, que verifica o próprio progresso executando manualmente a suíte de testes do projeto. A estratégia favorece a observação direta da relação entre escolhas de implementação e comportamento do compilador, mas transfere integralmente ao aprendiz a responsabilidade pela regulação do próprio ritmo (UEYAMA, 2020).

2.4.3 Análise Crítica e o Diferencial da Solução Proposta

A Tabela 2.1 sintetiza as principais características das ferramentas examinadas frente à plataforma proposta, evidenciando, em especial, a personalização da sintaxe pelo próprio estudante como traço distintivo da abordagem.

Tabela 2.1 – Comparativo entre as ferramentas analisadas e a plataforma proposta, segundo os três eixos de análise.

Ferramenta	**Escolhas de interface**	**Recursos pedagógicos**	**Estratégias de interação**	**Personalização da sintaxe pelo estudante**
Portugol Studio e Webstudio	IDE própria, em versão local e em versão executada no navegador	Linguagem com palavras-chave em português; depurador, inspetor de variáveis e árvore sintática	Acompanhamento visual da execução; projetos gráficos e sonoros	Não
Quorum	IDE própria concebida para leitores de tela; execução local ou no navegador	Sintaxe definida por evidência empírica; operadores expressos por palavras	Leitura por síntese de voz; saídas em áudio, gráficos e robótica	Não
Beecrowd	Plataforma web organizada em problemas e submissões	Acervo graduado por tema e dificuldade; múltiplas linguagens de mercado	Ciclo submissão–veredito, sem localização do erro; gamificação	Não

Continua na próxima página.

Tabela 2.1 – continuação

Ferramenta	**Escolhas de interface**	**Recursos pedagógicos**	**Estratégias de interação**	**Personalização da sintaxe pelo estudante**
Laila	IDE web com destaque de sintaxe e diagnóstico *inline*	Dispensa a instalação de FLEX, Bison e GCC, liberando o tempo de laboratório	Ciclo curto entre edição, execução e diagnóstico	Sim, mas como objeto de estudo: o aluno escreve gramáticas formais
chibicc	Sem interface própria; repositório de código-fonte	Trajetória incremental, uma capacidade da linguagem por etapa	Leitura e reimplementação do código; verificação pela suíte de testes	Não
Plataforma proposta	IDE web própria, com terminal integrado e sistema de arquivos	Exposição das fases de tradução (*tokens* e código intermediário); avaliação automatizada de exercícios	Execução e depuração passo a passo; erros relatados no vocabulário escolhido pelo próprio estudante	Sim: palavras-chave, delimitadores de bloco, operadores, literais booleanos e terminador de instrução

Fonte: elaborado pelo autor.

No conjunto comparado, Portugol, Quorum e Beecrowd priorizam a prática em linguagens e ambientes definidos por seus desenvolvedores. Laila permite escrever especificações léxicas e sintáticas, e *chibicc* permite estudar e modificar a implementação do compilador, exigindo conhecimentos próprios desse domínio. A IDE proposta oferece um assistente para configurar alternativas já suportadas pelo núcleo, dispensando a escrita de produções gramaticais ou alterações no código do tradutor.

A hipótese pedagógica é que o assistente favoreça a experimentação com o vocabulário e as convenções da linguagem. O instrumento da Seção 4.12 coleta a opinião de especialistas sobre a plausibilidade dessa hipótese. Confirmá-la depende do estudo com estudantes registrado ali como etapa seguinte. O comparativo identifica diferenças funcionais, não superioridade pedagógica da plataforma.

3 PROPOSTA

A finalidade deste capítulo é delimitar o que se pretende construir ao longo das duas fases do Trabalho de Conclusão de Curso e, em seguida, apresentar o planejamento de atividades derivado da pré-proposta. A descrição parte da plataforma vista como produto educacional, atravessa as macroatividades técnicas necessárias à sua materialização e culmina na exposição do cronograma e do regime de trabalho em dupla adotado.

O artefato que orienta este trabalho não é apenas um compilador ou um editor de código, mas sim uma plataforma educacional web cuja unidade de valor é a possibilidade de o estudante personalizar a própria linguagem que utilizará para aprender programação. Em termos concretos, oferece-se ao usuário a possibilidade de redefinir o vocabulário das palavras-chave, escolher os delimitadores de bloco que melhor representem sua noção de escopo, substituir operadores lógicos e relacionais por palavras de uso natural, reescrever os literais booleanos no idioma de sua preferência e ajustar as regras de finalização de instrução. Cada uma dessas dimensões responde, em alguma medida, a uma fonte conhecida de atrito sintático no contato inicial com linguagens consolidadas no mercado, sendo a integração desses recursos na interface o recorte da proposta. A Tabela 3.1 detalha cada um desses elementos, confrontando a forma canônica herdada da família C com um exemplo de personalização e com as restrições verificadas pela plataforma no momento da configuração.

Além das substituições lexicais, a configuração `GrammarConfig` seleciona alternativas de análise e execução: terminador obrigatório ou opcional por fim de linha; blocos delimitados ou por indentação; modo tipado ou não tipado; e arranjos fixos ou dinâmicos. Essas variantes são combinadas pelo módulo `compiler-config.ts`; não são novas produções fornecidas pelo usuário. No modo por indentação, delimitadores textuais deixam de ser o mecanismo de delimitação de blocos.

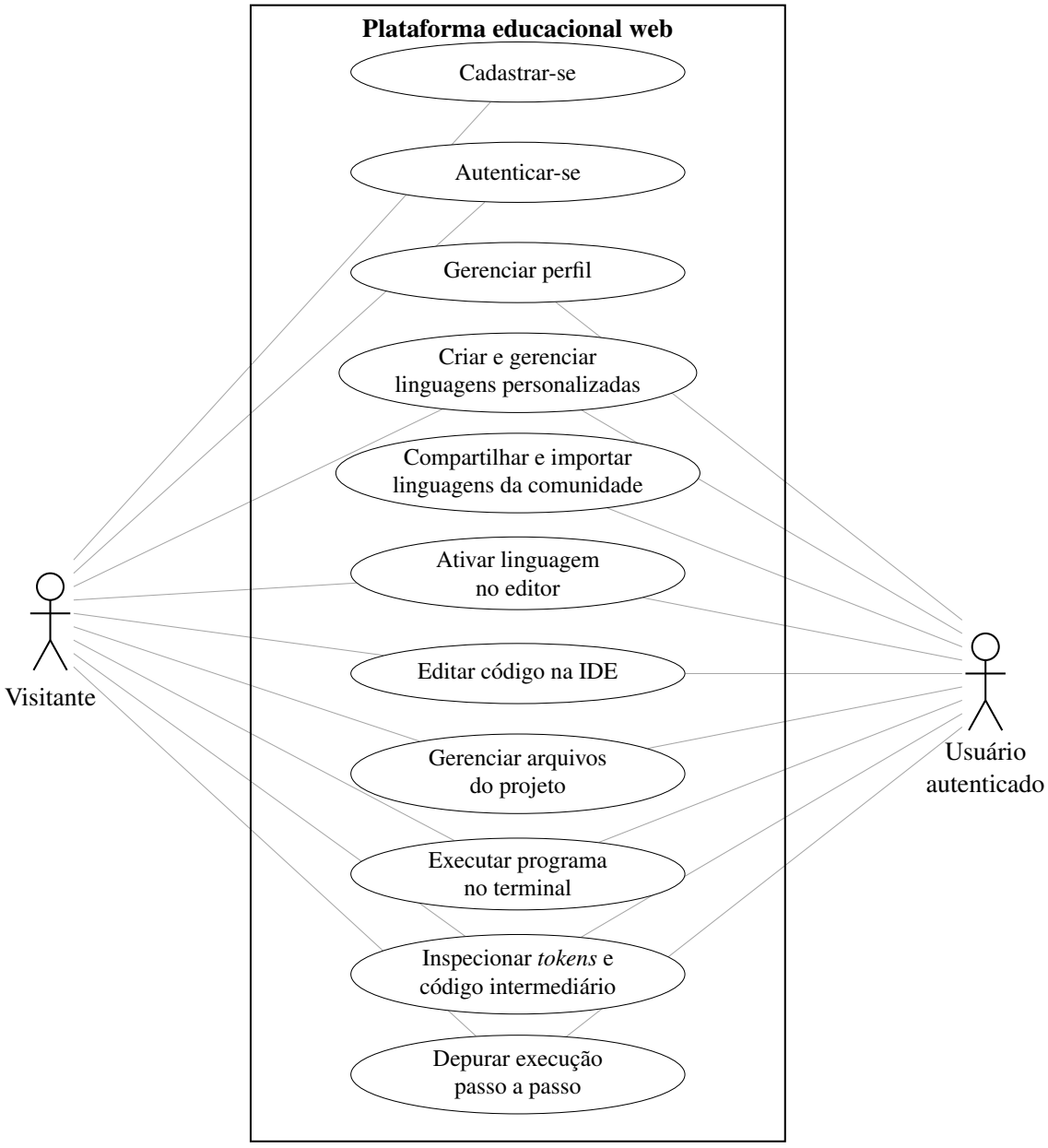
Os fluxos de uso da IDE e de gestão pedagógica especificados neste trabalho, com os respectivos atores, são apresentados nas Figuras 3.1 e 3.2. As duas figuras distinguem quatro atores: o visitante, que pode usar a IDE e a personalização local, além de cadastrar-se e autenticar-se; o usuário autenticado, que acrescenta persistência remota e compartilhamento de linguagens; e os papéis de aluno e de professor, que dele herdam e acrescentam, respectivamente, as funcionalidades de participação em turmas e de gestão pedagógica.

Tabela 3.1 – Elementos da linguagem passíveis de redefinição pelo usuário.

Elemento	**Forma canônica**	**Exemplo de personalização**	**Restrições verificadas**
Palavras-chave de tipo	`int`, `float`, `bool`, `string`	`inteiro`, `real`, `logico`, `texto`	Devem ter forma de identificador e não colidir entre si nem com os demais elementos configurados
Controle de fluxo	`if`, `else`, `for`, `while`, `break`, `continue`, `return`	`se`, `senao`, `para`, `enquanto`, `pare`, `siga`, `retorne`	Mesmas restrições das palavras-chave de tipo
Entrada e saída	`print`, `scan`	`escreva`, `leia`	Mesmas restrições das palavras-chave de tipo
Operadores lógicos	`&&`, `||`, `!`	`e`, `ou`, `nao`	Forma de identificador; sem duplicatas; não podem reusar palavra reservada nem colidir com delimitadores de bloco
Operadores relacionais	`==`, `!=`, `>`, `>=`, `<`, `<=`	`igual`, `diferente`, `maior`, `maiorIgual`, `menor`, `menorIgual`	Mesmas restrições dos operadores lógicos
Literais booleanos	`true`, `false`	`verdadeiro`, `falso`	Forma de identificador; sem duplicatas; não podem conflitar com operadores em forma de palavra nem com delimitadores
Delimitadores de bloco	`{` e `}`	`inicio` e `fim`, ou indentação	No modo delimitado, o par alternativo deve ter identificadores distintos e sem colisões; no modo por indentação, o recuo determina os blocos
Terminador de instrução	`;`	`ponto` ou terminação por fim de linha	Quando um lexema alternativo for definido, não pode ser vazio, conter espaços, reusar o ponto e vírgula ou caracteres de operadores fixos, nem colidir com palavras já definidas na linguagem

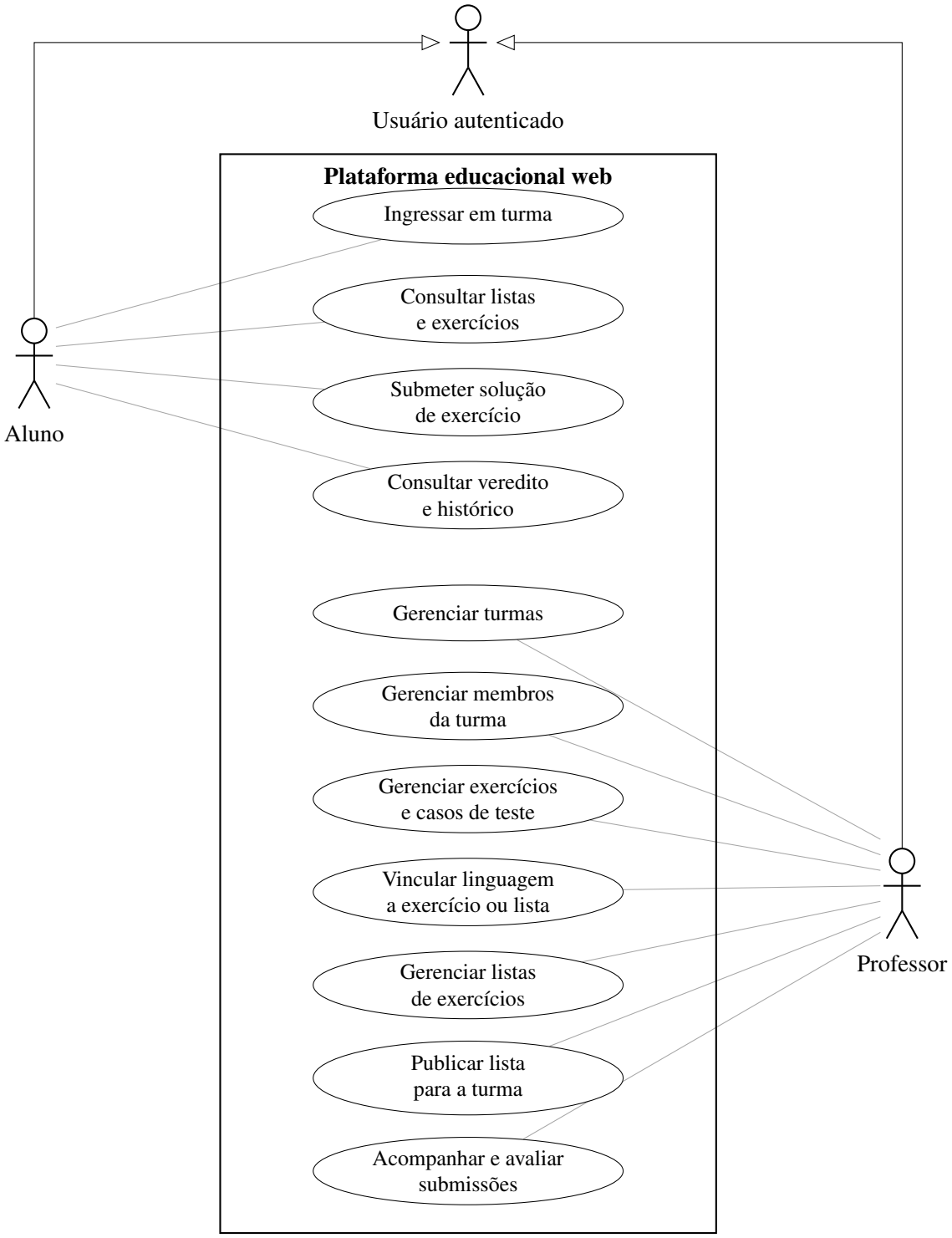
Fonte: elaborado pelo autor, a partir das funções de validação do módulo `config.ts` do pacote `compiler`.

Figura 3.1 – Casos de uso: acesso e ambiente de desenvolvimento.



Fonte: elaborado pelo autor.

Figura 3.2 – Casos de uso: participação e gestão pedagógica.



Fonte: elaborado pelo autor.

Os atores dos diagramas representam capacidades de interação, e não uma reprodução literal da enumeração de papéis do banco. No recorte da interface, o perfil administrador compartilha os recursos pedagógicos do professor; o perfil comunidade utiliza os recursos de linguagens do usuário autenticado. O papel interno de sistema não representa uma pessoa operando a IDE. A gestão de linguagens pelo visitante refere-se ao estado local; publicação e persistência remota exigem autenticação.

A partir desses casos de uso, consolidaram-se os requisitos que orientaram a construção da plataforma. A Tabela 3.2 relaciona os requisitos funcionais, isto é, o que o sistema deve fazer; a Tabela 3.3, os requisitos não funcionais, que estabelecem as qualidades esperadas da solução e os critérios sob os quais ela será verificada.

Conforme definido na pré-proposta, a metodologia foi organizada em seis macroetapas complementares. Essas etapas não devem ser entendidas como blocos completamente isolados, pois o desenvolvimento de uma plataforma interativa exige revisões sucessivas entre interface, interpretador e experiência de uso. Ainda assim, elas delimitam com clareza o percurso técnico e acadêmico do projeto:

- **Revisão bibliográfica:** levantamento das bases conceituais sobre interpretadores, compiladores e ferramentas educacionais relacionadas, com o objetivo de sustentar tanto as decisões de projeto quanto a posterior análise comparativa;
- **Definição de requisitos e arquitetura:** consolidação dos elementos configuráveis da linguagem, da modelagem das camadas da plataforma e dos contratos de comunicação entre o núcleo do interpretador e a IDE;
- **Desenvolvimento da plataforma:** implementação da aplicação web, contemplando os fluxos de edição, navegação, personalização, autenticação e uso educacional da IDE;
- **Implementação da análise dinâmica léxica e sintática:** integração entre o código escrito na IDE e as fases de reconhecimento de *tokens* e validação sintática com suporte às regras configuradas pelo usuário;
- **Geração e interpretação do código intermediário:** disponibilização, no ambiente da IDE, da execução do código intermediário e do retorno de avisos, erros e resultados ao usuário;

Tabela 3.2 – Requisitos funcionais da plataforma.

ID	Descrição
RF01	Permitir o cadastro e a autenticação de usuários, distinguindo aluno, professor, administrador e comunidade
RF02	Permitir ao usuário autenticado consultar e editar os dados de seu perfil
RF03	Oferecer um assistente guiado, organizado em etapas independentes, para a criação de uma linguagem personalizada
RF04	Permitir a redefinição dos elementos da linguagem relacionados na Tabela 3.1
RF05	Validar preventivamente cada configuração, impedindo colisões entre palavras-chave, operadores, literais, delimitadores e terminador de instrução
RF06	Exibir, em tempo real, a pré-visualização de um programa exemplo sob a configuração corrente
RF07	Persistir localmente a customização ativa e remotamente as linguagens nomeadas pelo usuário
RF08	Permitir publicar uma linguagem no catálogo comunitário e importar linguagens publicadas por terceiros
RF09	Oferecer edição de código com realce de sintaxe e sugestão automática ajustados à linguagem ativa
RF10	Manter um sistema de arquivos virtual que permita criar, renomear, remover e alternar entre arquivos
RF11	Executar o programa produzido no editor, com entrada e saída conectadas ao terminal integrado
RF12	Exibir ao usuário a tabela de *tokens* reconhecidos e o código intermediário gerado
RF13	Oferecer execução passo a passo, destacando no editor a instrução em processamento
RF14	Relatar erros léxicos, sintáticos, semânticos e de execução no vocabulário da linguagem ativa e no idioma selecionado
RF15	Permitir ao professor criar e gerir turmas, e ao aluno ingressar em uma turma
RF16	Permitir ao professor criar e gerir exercícios, com casos de teste de entrada e saída esperada
RF17	Permitir vincular uma linguagem obrigatória a um exercício ou a uma lista, conforme a política de linguagem definida
RF18	Permitir organizar exercícios em listas e publicá-las para uma turma
RF19	Avaliar automaticamente a submissão do aluno contra os casos de teste do exercício
RF20	Permitir ao professor acompanhar as submissões da turma e atribuir nota

Fonte: elaborado pelo autor.

- **Integração, testes e preparação da avaliação:** execução de testes técnicos e elaboração de tarefas, instrumentos e medidas para uma avaliação futura. As atividades de documentação acompanham as etapas técnicas e culminam na redação dos resultados e das limitações.

A distribuição temporal dessas atividades ao longo dos doze meses que compreendem o TCC I e o TCC II é apresentada na Tabela 3.4, preservando as marcações temporais da pré-proposta e ajustando a descrição da avaliação ao escopo atual. A tabela expressa planejamento, não comprovação de execução. Cada coluna numerada corresponde a um mês corrido das duas etapas e cada célula marcada com “X” sinaliza a previsão de execução da

Tabela 3.3 – Requisitos não funcionais da plataforma.

ID	Descrição
RNF01	**Acessibilidade:** os componentes de interface devem observar as diretrizes WAI-ARIA (W3C, 2023), com navegação por teclado, leitura por tecnologias assistivas e gerenciamento de foco
RNF02	**Responsividade:** as composições de tela devem reorganizar-se conforme a faixa de resolução do dispositivo
RNF03	**Internacionalização:** toda cadeia visível ao usuário deve estar externalizada, com suporte aos idiomas *pt-BR*, *pt-PT*, *es* e *en*
RNF04	**Autonomia de execução:** edição, análise léxica e sintática, geração de código intermediário e interpretação devem permanecer funcionais na ausência de conexão com o serviço de persistência
RNF05	**Latência:** a execução do núcleo de tradução deve ocorrer sem ciclo de requisição e resposta a servidor remoto
RNF06	**Consistência visual:** os temas claro e escuro devem manter paridade de contraste e legibilidade, inclusive no editor
RNF07	**Testabilidade:** as construções críticas devem ser cobertas por testes automatizados, executados localmente e em uma futura etapa de testes do fluxo de integração
RNF08	**Segregação de acesso:** as rotas e operações restritas devem verificar o papel do usuário antes de permitir o acesso

Fonte: elaborado pelo autor.

respectiva atividade no período indicado.

Convém retomar, neste ponto, o regime de trabalho em dupla já apresentado na introdução (Capítulo 1), dada sua influência direta sobre o escopo do projeto e sobre os capítulos subsequentes. A pré-proposta aprovada para este Trabalho de Conclusão de Curso prevê a **participação conjunta em todas as etapas** do desenvolvimento, mas com funções complementares. Em respeito a esse compromisso, as decisões de arquitetura, os ciclos de revisão de código e as etapas de integração foram conduzidos de modo colaborativo pelos dois integrantes da dupla. Cabe ressaltar que, por se tratar de uma equipe reduzida, não houve especialização de papéis: ambos os integrantes acumularam as atribuições de levantamento de requisitos, projeto, implementação, teste e revisão. Adotaram-se, de forma pragmática, as práticas técnicas do *Extreme Programming* discutidas no referencial teórico, sem ciclos de duração fixa: partiu-se de um conjunto mínimo de requisitos e o sistema avançou por acréscimos funcionais, incorporando as melhorias identificadas durante a própria implementação (BECK; ANDRES, 2004; WILDT et al., 2015).

A partir dessa base comum, a divisão complementar de eixos de aprofundamento manifesta-se nos capítulos de redação acadêmica e nas evoluções incrementais subsequentes. No presente documento, o autor concentra a investigação na plataforma e na customização de comandos: arquitetura da aplicação web, concepção do assistente de personalização, experiência

Tabela 3.4 – Cronograma de desenvolvimento do trabalho de conclusão de curso.

Atividade	**TCC I**						**TCC II**					
	01	**02**	**03**	**04**	**05**	**06**	**07**	**08**	**09**	**10**	**11**	**12**
Revisão Bibliográfica	X	X	X	X								
Escrita da Introdução e Revisão de Literatura		X	X	X								
Predefinição de escopo e Especificação dos comandos dinâmicos			X	X								
Documentação dos procedimentos de desenvolvimento			X		X	X						
Arquitetura e Modelagem da Plataforma			X	X								
Customização dinâmica de *tokens* e sintaxe				X	X							
Execução da Análise Léxica e Sintática com o código da IDE					X	X						
Execução do código intermediário no Terminal da IDE com *feedback* (avisos e erros)						X	X	X				
Finalização de fluxos, acessibilidade e responsividade						X	X	X				
Testes técnicos e preparação do protocolo de avaliação									X	X	X	
Escrita do TCC (Resultados, Discussão e Conclusão)									X	X	X	X

Fonte: elaborado pelo autor.

de uso da IDE, execução do código no terminal integrado e procedimentos de coleta de *feedback* junto aos estudantes. O detalhamento aprofundado da evolução do núcleo do interpretador, incluindo a geração de código intermediário e sua integração técnica ao restante da plataforma, é desenvolvido em paralelo pelo estudante Victor de Souza Vilela da Silva. Essa divisão de eixos não fragmenta o produto: o que se distribui é a profundidade analítica de cada autor sobre partes específicas do mesmo artefato, conduzido pelos dois em todas as etapas. Tal arranjo justifica-se pela amplitude do escopo, que, individualmente, tenderia a resultar em um interpretador simplificado ou em uma plataforma incompleta, restrita a um terminal.

O planejamento do TCC I concentrou a arquitetura e a integração inicial. Esta revisão para o TCC II documenta a implementação disponível, os testes executados e a proposta de protocolo. A coleta com estudantes permanece futura. O próximo capítulo apresenta esse estado técnico e distingue as verificações realizadas das avaliações ainda previstas.

4 METODOLOGIA

Este capítulo apresenta a abordagem metodológica adotada para o desenvolvimento da plataforma web proposta neste trabalho, detalhando a arquitetura do Ambiente de Desenvolvimento Integrado (IDE), as estratégias empregadas para a customização dinâmica de *tokens* e sintaxe, os mecanismos de integração com o núcleo do interpretador e os fluxos de execução, avaliação e *feedback* ao usuário. O capítulo registra o estado técnico da plataforma durante a revisão para o TCC II, a partir da implementação e dos testes disponíveis, contemplando a revisão bibliográfica, a especificação do escopo dos comandos dinâmicos, a modelagem da arquitetura da plataforma, a implementação da customização interativa da linguagem, a execução das análises léxica e sintática a partir do código produzido na IDE, a apresentação dos resultados intermediários no terminal embutido e a finalização dos fluxos principais, com atenção à acessibilidade e à responsividade.

A descrição da implementação foi conferida inicialmente em 16 de setembro de 2026, na revisão `b4372bf`. Em 19 de setembro, a análise foi atualizada para a revisão `3ffae01`, incorporando os testes de ponta a ponta do front-end e o fluxo de integração contínua. As verificações de interface e as capturas anteriores conservam suas datas e seu escopo original; os resultados da nova execução de testes são identificados separadamente na Seção 4.11 (QUEIROZ; SILVA, 2026c; QUEIROZ; SILVA, 2026a; QUEIROZ; SILVA, 2026b). Em 19 de outubro, a descrição foi conferida de novo na revisão `04adff2`, que trouxe os tutoriais guiados, a página Sobre, o diálogo de regras do compilador e a correção de submissões dentro da IDE (QUEIROZ, 2026a).

O desenvolvimento foi conduzido de forma incremental e iterativa, orientado pelas práticas técnicas do *Extreme Programming* discutidas na Seção 2.2.1. Convém explicitar de que modo essas práticas foram efetivamente instanciadas em uma equipe de dois integrantes, uma vez que o método foi adotado de forma parcial e deliberada. Não se estabeleceram ciclos de duração fixa: o trabalho partiu de um conjunto mínimo de requisitos e avançou por acréscimos funcionais, e sempre que uma melhoria não prevista se mostrava pertinente durante a implementação, ela era incorporada naquele momento, em vez de ser postergada para um planejamento posterior. Essa incorporação contínua da mudança, e não a cadência fixa de entregas, é o traço do método que efetivamente governou o projeto (BECK; ANDRES, 2004; WILDT et al., 2015).

Quanto à divisão de trabalho, este TCC concentra a responsabilidade de Igor pelo

front-end, enquanto o texto complementar de Victor trata do núcleo de tradução e execução. A colaboração técnica manteve a propriedade coletiva do código: ambos os autores atuaram no levantamento de requisitos, no projeto, na implementação, no teste e na revisão, em regime de propriedade coletiva do código, no qual qualquer integrante alterava qualquer parte do sistema, com alterações registradas no histórico do repositório (WILDT et al., 2015). As demais práticas do método foram adotadas nos termos descritos ao longo deste capítulo: o uso de testes automatizados, detalhado na Seção 4.11; o versionamento e a configuração de implantação; o *design* simples e a refatoração contínua, que orientaram a organização em camadas descrita na Seção 4.1.

O restante deste capítulo está organizado da seguinte maneira. A Seção 4.1 descreve a modelagem da arquitetura e a divisão em camadas; a Seção 4.2 situa o núcleo de tradução compartilhado entre os dois textos; a Seção 4.3 detalha o mecanismo de customização dinâmica da linguagem, funcionalidade central da plataforma. Em seguida, a Seção 4.4 trata da edição de código e do sistema de arquivos virtual; a Seção 4.5 e a Seção 4.6 descrevem, respectivamente, o acionamento das análises a partir da IDE e a execução do código intermediário com retorno ao usuário; e a Seção 4.7 apresenta o subsistema de avaliação automatizada. Por fim, a Seção 4.8 apresenta a verificação da interface, a Seção 4.9 descreve os recursos de ajuda ao usuário, a Seção 4.10 reúne as telas, a Seção 4.11 expõe os testes e a Seção 4.12 descreve o instrumento de avaliação, cuja análise ficou reservada ao Capítulo 5.

Em razão da elevada quantidade de telas, componentes interativos e estados compartilhados característicos de uma IDE web, cada fluxo da plataforma, desde a autenticação do usuário até a submissão de exercícios — o conjunto completo deles está sistematizado nas Figuras 3.1 e 3.2 —, foi tratado como uma unidade autônoma de entrega, implementada e revista ao longo dos incrementos. Essa abordagem foi sustentada por testes automatizados unitários e de integração escritos com o *framework* Vitest, relacionados à discussão de Desenvolvimento Guiado por Testes de Aniche (2012) e Martin (2011), permitindo a verificação contínua de regressões diante das frequentes mudanças impostas pela natureza altamente interativa da plataforma.

4.1 Modelagem da Arquitetura da Plataforma

A organização da plataforma foi concebida sob a ótica da Separação de Responsabilidades defendida por Martin (2019), dividindo o projeto em três responsabilidades principais: o pacote `compiler`, responsável pelo núcleo do interpretador; o pacote `ide`, responsável pela camada de apresentação e por toda a interação com o usuário; e o serviço de *back-end*, responsável pela persistência das entidades de domínio do Ambiente Virtual de Aprendizagem. Os pacotes `compiler` e `ide` são versionados em um único repositório no formato *monorepo*, viabilizando o consumo direto do compilador como dependência local da IDE, sem qualquer empacotamento intermediário e sem comunicação remota entre as duas camadas. Esse consumo elimina a requisição remota na análise interativa e permite compartilhar a implementação entre navegador e servidor; não foi medida a redução de latência.

O *front-end* foi implementado com o *framework* Next.js, em sua arquitetura de *Pages Router* e sobre o *Node.js Runtime* padrão — nenhuma rota declara `runtime = 'edge'`, de modo que o *Edge Runtime* discutido no referencial teórico não é utilizado neste projeto, opção coerente com a necessidade de acesso pleno às APIs do Node.js pelas rotas de validação de submissões —, sobre a biblioteca React 19, utilizando TypeScript em todo o código-fonte para verificar tipos durante o desenvolvimento. A estilização foi conduzida com Tailwind CSS em sua quarta versão, complementada por componentes das bibliotecas *shadcn/ui* e *Radix UI*, que fornecem primitivas de semântica e interação, sem garantir a acessibilidade da composição final (Radix, 2026). O gerenciamento de estado assíncrono foi delegado à biblioteca *TanStack React Query*, responsável pela orquestração de cache, sincronização e revalidação dos dados consumidos das rotas HTTP, enquanto a validação de formulários foi modelada com a biblioteca *Zod*, integrada ao *React Hook Form*.

A camada de *back-end* foi implementada em Python sobre o *framework* FastAPI, escolhido pelas características de alto desempenho, validação automática via *Pydantic* e geração automatizada de documentação OpenAPI discutidas por FASTAPI (2026). A persistência foi sustentada pelo ORM SQLAlchemy assíncrono, com versionamento de *schema* via Alembic, isolando o modelo de dados das regras de negócio em conformidade com os princípios de Arquitetura Limpa apresentados por Martin (2019). Essa separação mantém o modelo de persistência confinado a uma camada própria, de modo que o núcleo de compilação e a interface web não dependam diretamente do banco de dados nem de suas particularidades. Não se trata de

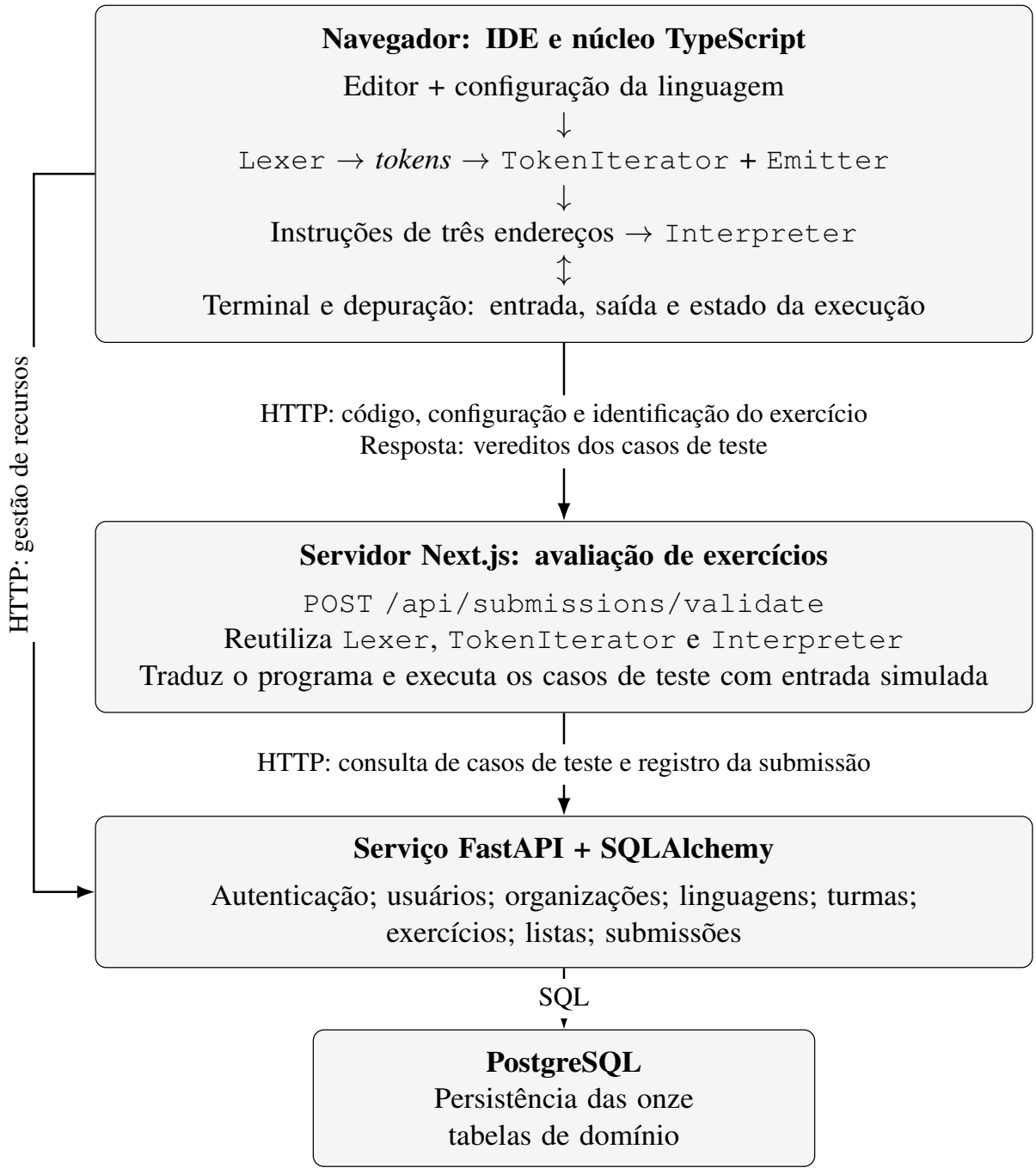
minimizar a importância da persistência, mas de restringir seu alcance: mudanças no esquema ou no mecanismo de armazenamento não devem propagar-se para as regras de tradução nem para a camada de apresentação.

O fluxo geral da Figura 4.1 distingue dois ambientes de execução. No navegador, os *hooks* `useLexerAnalyse` e `useIntermediatorCode` e o terminal instanciam o núcleo diretamente para analisar, traduzir e executar o programa do editor. No servidor Node.js do Next.js, a rota `/api/submissions/validate` reutiliza as mesmas classes para avaliar soluções de exercícios. O serviço FastAPI fornece autenticação e recursos persistentes. Após o carregamento da aplicação, os recursos locais não exigem uma chamada a esse serviço para cada análise; isso não equivale a garantir o carregamento inicial da plataforma sem conexão.

A organização interna do código da IDE seguiu a topologia clássica de aplicações Next.js, com diretórios dedicados para páginas (`pages`), componentes reutilizáveis (`components`), *contexts* de estado global (`contexts`), *hooks* personalizados (`hooks`), entidades de domínio (`entities`), bibliotecas de apoio (`lib`) e composições de tela (`views`). Essa segmentação é um primeiro passo na direção da coesão funcional discutida por Silveira et al. (2012): a organização em diretórios, isoladamente, não a garante, mas torna explícita a responsabilidade atribuída a cada módulo e facilita que ela seja preservada ao longo da evolução do código, favorecendo a legibilidade e a testabilidade do sistema.

Cabe explicitar quais padrões de projeto foram efetivamente empregados nessa organização. Na camada de apresentação, adota-se o padrão *Provider* da biblioteca React: cada preocupação transversal da aplicação (autenticação, editor, customização da linguagem, terminal, tema, notificações e erros de execução) é isolada em um *context* próprio, sob o diretório `contexts`, que expõe estado e operações aos componentes descendentes sem exigir a propagação manual de propriedades. Os *hooks* personalizados, por sua vez, operam como fachadas sobre esses *contexts* e sobre o pacote `compiler`, oferecendo aos componentes uma interface reduzida e estável, independente da forma como o núcleo de tradução é instanciado. A organização interna do núcleo de tradução é detalhada no trabalho complementar; neste texto, o foco recai nos contratos consumidos pelos componentes da IDE.

Figura 4.1 – Arquitetura da plataforma: camadas, módulos internos e fluxos de comunicação.



Fonte: elaborado pelo autor.

4.2 Interpretador

O interpretador constitui o núcleo de tradução comum a este projeto e foi desenvolvido de forma colaborativa pelos dois autores, Victor de Souza Vilela da Silva e Igor Lamoia Queiroz, conforme a divisão de responsabilidades registrada no Apêndice A. Implementado em TypeScript e consumido pela IDE como dependência local, conforme descrito na Seção 4.1, o núcleo encadeia as fases de análise léxica, análise sintática, geração de código intermediário em três endereços e execução pelo interpretador, recebendo dinamicamente a configuração da linguagem definida pelo usuário (palavras-chave, operadores em forma de palavra, literais booleanos, delimitadores de bloco e terminador de instrução).

Por se tratar da base técnica compartilhada entre os dois textos, o detalhamento interno de cada fase do núcleo, incluindo as estratégias de varredura léxica, a análise sintática por descida recursiva, a geração de código de três endereços e o funcionamento do interpretador, é objeto do trabalho complementar de autoria de Victor de Souza Vilela da Silva. Este texto concentra-se na forma como a IDE aciona e integra esse núcleo, detalhada nas seções seguintes.

4.3 Customização Dinâmica de *Tokens* e Sintaxe

A funcionalidade central da plataforma, que consiste na possibilidade de o usuário personalizar dinamicamente a sintaxe e o vocabulário da linguagem, foi materializada em um conjunto coordenado de componentes, *contexts* e utilitários. O elemento central dessa funcionalidade é o `KeywordContext`, um *context* do React que mantém, em memória e na camada de armazenamento local do navegador, o estado completo da customização ativa, incluindo o mapeamento de palavras-chave personalizadas, os operadores em forma de palavra, os literais booleanos, os delimitadores de bloco e o terminador de instrução.

O contrato técnico que sustenta a customização é definido pelo módulo `config.ts` do pacote `compiler`, que declara, por meio de tipos TypeScript, todos os elementos da linguagem passíveis de personalização. O Trecho de Código 4.1 apresenta esses tipos, que funcionam como a interface formal entre a camada de interface, responsável pela coleta das preferências do usuário, e o núcleo do tradutor, responsável pela aplicação da configuração no momento da varredura léxica.

A definição de quais elementos da linguagem seriam exibidos como configuráveis foi guiada pelos achados empíricos de Stefik e Siebert (2013), segundo os quais a sintaxe

```typescript
export type KeywordMap = Record<string, number>;

export type OperatorWordMap = {
  logical_or?: string;      logical_and?: string;
  logical_not?: string;     less?: string;
  less_equal?: string;      greater?: string;
  greater_equal?: string;   equal_equal?: string;
  not_equal?: string;
};

export type BooleanLiteralMap = {
  true?: string;
  false?: string;
};

export type LexerBlockDelimiters = {
  open: string;
  close: string;
};

export type LexerConfig = {
  customKeywords?: KeywordMap;
  operatorWordMap?: OperatorWordMap;
  booleanLiteralMap?: BooleanLiteralMap;
  statementTerminatorLexeme?: string;
  blockDelimiters?: LexerBlockDelimiters;
  locale?: string;
  indentationBlock?: boolean;
  tabWidth?: number;
};
```

Listing 4.1 – Tipos que descrevem a configuração customizável do lexer.

tradicional derivada da família C atua como uma barreira significativa para estudantes iniciantes. Foram disponibilizadas para customização as palavras-chave de declarações de tipo (`int`, `float`, `bool`, `string`), de controle de fluxo (`if`, `else`, `for`, `while`, `break`, `continue`, `return`, `switch`, `case`), as operações de entrada e saída (`print`, `scan`), os operadores lógicos (`&&`, `||`, `!`) e relacionais (==, `!=`, >, >=, <, <=), os literais booleanos (`true`, `false`), os delimitadores de bloco (chaves ou indentação) e o terminador de instrução. Essas opções abrangem precisamente os elementos que oneram a carga cognitiva inicial discutidos por Stefik e Siebert (2013), mantendo intactos os aspectos estruturais responsáveis pela consistência das demais fases do compilador.

A consolidação do estado da customização no `KeywordContext` é exposta às demais camadas da aplicação por meio do utilitário `buildLexerConfig`, apresentado no Trecho

de Código 4.2. Esse utilitário traduz o estado em memória, contendo os mapeamentos de palavras-chave, os operadores, os literais booleanos, os delimitadores de bloco e o terminador de instrução, em um descritor compatível com o tipo `LexerConfig` do pacote `compiler`, atuando como a fronteira efetiva entre o estado da interface e a invocação do tradutor.

```
const buildKeywordMap = useCallback((): Record<string, number> =>
    {
  const map: Record<string, number> = {};
  for (const m of customization.mappings) {
    map[m.custom] = m.tokenId;
  }
  return map;
}, [customization.mappings]);

const buildLexerConfig = useCallback((): IDECompilerConfigPayload
    => {
  return buildLexerConfigFromCustomization(customization);
}, [customization]);
```

Listing 4.2 – Utilitário `buildLexerConfig` do `KeywordContext`, que materializa a configuração corrente em um descritor consumível pelo lexer.

A interface de customização foi implementada como um assistente progressivo no formato *wizard*, materializado no componente `keyword-customizer`, que conduz o usuário por etapas independentes, cada uma responsável por um aspecto específico da personalização. O fluxo é controlado pelo `wizard-model`, que mantém o estado de navegação e valida cada passo antes de permitir o avanço. A declaração formal das etapas do assistente é apresentada no Trecho de Código 4.3, no qual cada elemento do arranjo `WIZARD_STEPS` associa um identificador, um título exibido na barra de progresso, uma descrição textual e um ícone semântico, oferecendo ao usuário um mapa visual contínuo do fluxo de personalização.

A cada alteração proposta pelo usuário, o componente `preview-builder` produz, em tempo real, um trecho de código exemplificativo que reflete a customização corrente, oferecendo uma visualização imediata do impacto das mudanças sobre a aparência dos programas. O Trecho de Código 4.4 ilustra a função `buildDelimiterSnippet`, responsável por interpolar, em um molde fixo, as palavras-chave e os delimitadores efetivamente escolhidos pelo usuário, produzindo um trecho ilustrativo com a configuração corrente. Essa pré-visualização não executa a análise sintática do trecho gerado e, portanto, não comprova que o exemplo seja um programa completo aceito pelo núcleo.

A pré-visualização permite inspecionar a escrita resultante antes de salvar a configuração.

```
export const WIZARD_STEPS = [
  { id: "identity",  title: "Identidade",
    description: "Escolha o ponto de partida da linguagem.",
    icon: "fingerprint" },
  { id: "IO",        title: "Entrada/Saida",
    description: "Defina saida, leitura e a linguagem das
       variaveis.",
    icon: "cable" },
  { id: "types",     title: "Tipagem",
    description: "Defina leitura, escrita e o vocabulario usado
       para declarar variaveis.",
    icon: "book-open-text" },
  { id: "structure", title: "Estrutura",
    description: "Configure blocos, delimitadores e o fim das
       instrucoes.",
    icon: "blocks" },
  { id: "rules",     title: "Operadores",
    description: "Ajuste os operadores, literais booleanos e
       regras de decisao.",
    icon: "sigma" },
  { id: "flow",      title: "Fluxo",
    description: "Customize o vocabulario de controle.",
    icon: "route" },
  { id: "review",    title: "Revisao",
    description: "Confira o resumo final antes de salvar.",
    icon: "clipboard-check" },
] as const;
```

Listing 4.3 – Declaração do arranjo `WIZARD_STEPS`, que organiza as etapas do assistente de personalização.

```
export function buildDelimiterSnippet(
  draft: StoredKeywordCustomization,
): string {
  const print = getKeyword(draft, "print");
  const scan = getKeyword(draft, "scan");
  const lineEnding = buildLineEnding(draft);
  const functionKeyword = getKeyword(draft, "function");
  const baseCodeSnippet = `${functionKeyword} main()`;

  const open = draft.blockDelimiters.open.trim() || "{";
  const close = draft.blockDelimiters.close.trim() || "}";

  return `${baseCodeSnippet}${open}\n\t${print}("Ola Mundo!")${
     lineEnding}
  ${scan}(nome)${lineEnding}\n\t${print}("Me chamo:", nome)\n${
     close}`;
}
```

Listing 4.4 – Função `buildDelimiterSnippet`, que gera o *preview* dinâmico exibido durante a personalização.

Um dos itens do instrumento descrito na Seção 4.12 pergunta se ela é útil. Saber se ela funciona com iniciantes exigiria o estudo com estudantes registrado ali como etapa seguinte.

A validação das customizações é centralizada em um conjunto de funções utilitárias dedicadas, hospedadas no diretório `contexts/keyword/keyword-validator.ts`. A função `validateCustomKeyword` garante que palavras-chave personalizadas atendam às restrições léxicas elementares e não colidam entre si; `validateBlockDelimiters` verifica a consistência dos delimitadores de bloco. Toda a verificação ocorre de forma preventiva, no momento da alteração pelo usuário, impedindo que configurações inválidas sejam persistidas ou propagadas ao analisador léxico, em conformidade com os princípios de detecção precoce de erros discutidos por Cooper e Torczon (2012).

O Trecho de Código 4.5 ilustra a estratégia de verificação aplicada aos delimitadores de bloco. A função encadeia as restrições elementares (ambos os delimitadores preenchidos, ausência de espaços, distinção entre o delimitador de abertura e o de fechamento e ausência de colisão com palavras reservadas), devolvendo, em caso de violação, a mensagem internacionalizada correspondente. Até setembro, a segunda verificação exigia forma de identificador, com `WORD_REGEX`. Ela foi trocada por `DELIMITER_REGEX`, que recusa apenas espaços, para que a predefinição livre do assistente pudesse informar as próprias chaves como delimitadores. Com isso a interface aceita também delimitadores como `<:` e `:>`, que o analisador léxico do núcleo não reconhece como um único *token*; a limitação está descrita no texto complementar. Quando todas as restrições são atendidas, a função retorna `null`, sinalizando à interface que a configuração pode ser persistida.

4.3.1 Linguagens predefinidas e identidade da configuração

A IDE oferece quatro linguagens prontas no seletor, definidas em `lib/default-languages.ts`: Portugol, Python-like, C-like e Ruby-like. Portugol é a configuração inicial quando não há uma configuração salva aplicável. Essas opções combinam vocabulário e modos já implementados no núcleo; seus nomes indicam inspiração sintática, sem equivalência com as linguagens de propósito geral. A Tabela 4.1 resume os modos escolhidos por cada opção.

O *hook* `useLanguageChoices` reúne essas quatro opções com as linguagens locais do visitante ou com o acervo remoto do usuário autenticado. Selecionar uma opção predefinida aplica sua configuração e grava a escolha no armazenamento local, sem criar uma linguagem

```ts
export function validateBlockDelimiters(
  value: BlockDelimiters,
): string | null {
  const open = value.open.trim();
  const close = value.close.trim();

  if (!open && !close) return null;
  if (!open || !close) {
    return ui.validation_fill_delimiters;
  }

  if (!DELIMITER_REGEX.test(open) || !DELIMITER_REGEX.test(close)
     ) {
    return ui.validation_invalid_delimiter_format;
  }

  if (open === close) {
    return ui.validation_delimiters_must_differ;
  }

  if (
    ORIGINAL_KEYWORDS.includes(open) ||
    ORIGINAL_KEYWORDS.includes(close)
  ) {
    return ui.validation_delimiters_reserved;
  }

  return null;
}
```

Listing 4.5 – Função `validateBlockDelimiters`, responsável pela verificação preventiva dos delimitadores de bloco.

Tabela 4.1 – Configurações predefinidas disponíveis no seletor da IDE.

Configuração	**Blocos**	**Tipagem e arranjos**	**Terminador**
Portugol	`inicio`/`fim`	Tipos explícitos; arranjos fixos	Ponto e vírgula obrigatório
Python-like	Indentação	Sem declaração explícita de tipo; arranjos dinâmicos	Opcional no fim da linha
C-like	Chaves	Tipos explícitos; arranjos fixos	Ponto e vírgula obrigatório
Ruby-like	`do`/`end`	Sem declaração explícita de tipo; arranjos dinâmicos	Opcional no fim da linha

Fonte: elaborado pelo autor a partir de `default-languages.ts`.

no banco. Para uma linguagem remota, o *hook* recupera o detalhe e solicita a atualização da linguagem ativa pela API. As predefinições do seletor são distintas dos pontos de partida do assistente, definidos em `wizard-model.ts`, que incluem também as opções livre e inspirada no vocabulário mineiro.

As palavras canônicas para declarações sem tipo explícito passaram a ser `variable` e `function`. Os termos `variavel` e `funcao` continuam possíveis como personalizações; a normalização de configurações armazenadas reconhece as chaves antigas e as converte para as atuais, preservando os lexemas escolhidos. O reconhecimento de identificadores e a validação dos nomes personalizados passaram a admitir também `ç` e `Ç`, além das letras latinas sem acento e do sublinhado; dígitos são aceitos após o primeiro caractere. Isso não corresponde a aceitar qualquer caractere Unicode.

O assistente mantém nome e descrição nos campos `name` e `description`. A edição de uma linguagem existente usa a rota `/language-creator/[id]`, cujo carregamento inicial ocorre no servidor por `getServerSideProps`: o identificador é validado, a credencial do *cookie* é encaminhada à API e os dados retornados inicializam o formulário. A criação sem identificador continua disponível sem autenticação, com persistência local. Essa distinção evita tratar a edição remota como uma simples reabertura de um rascunho local.

A persistência da customização foi modelada em duas camadas complementares. As configurações ativas no editor são salvas localmente, no *localStorage* do navegador, por meio do utilitário `keyword-language-storage`, garantindo que o estado da personalização sobreviva ao recarregamento da página e à navegação entre rotas da aplicação. Linguagens nomeadas e exportadas pelo usuário, por sua vez, são persistidas no *back-end* via API, viabilizando o compartilhamento entre dispositivos e a vinculação de linguagens específicas a exercícios oferecidos pelo professor. O utilitário `language-creator-navigation` orquestra as transições entre as etapas de criação, gravação e ativação de linguagens, mantendo coesão entre o estado em memória, o estado no *localStorage* e o estado remoto.

Por fim, o impacto visual imediato da customização sobre o editor Monaco é mediado pela função `updateJavaMMKeywords`. Essa função reconstrói dinamicamente a gramática léxica registrada no editor cada vez que o `KeywordContext` sinaliza uma alteração na customização, refletindo de imediato o realce de sintaxe, a sugestão automática de palavras-chave e a documentação contextual exibida pelo Monaco. O Trecho de Código 4.6 apresenta sua

assinatura, evidenciando o desacoplamento entre o módulo de personalização e o módulo de edição: a função recebe a instância do Monaco, o conjunto atualizado de mapeamentos e as opções de linguagem, delegando à função interna `registerJavaMMLanguage` o registro completo do dialeto sob o identificador `java-`. Essa abordagem evita a necessidade de reinicializar o editor a cada mudança e preserva a instância do editor durante a personalização.

```
export const JAVAMM_LANGUAGE_ID = "java--";

export type JavaMMKeywordMapping = {
  original: string;
  custom: string;
  tokenId: number;
};

export function updateJavaMMKeywords(
  monaco: typeof monacoEditor,
  keywordMappings: JavaMMKeywordMapping[],
  options: JavaMMLanguageOptions = {},
) {
  registerJavaMMLanguage(monaco, keywordMappings, options);
}
```

Listing 4.6 – Função `updateJavaMMKeywords`, ponto de integração entre o estado da customização e o Monaco Editor.

4.4 Edição de Código e Sistema de Arquivos Virtual

A experiência de edição foi construída em torno do *Microsoft Monaco Editor*, o mesmo motor de edição utilizado pelo Visual Studio Code, integrado à aplicação por meio do pacote `@monaco-editor/loader`. A linguagem personalizada do projeto foi registrada sob o identificador `java-`, e dois temas próprios, `editor-glass-dark` e `editor-glass-light`, foram modelados a partir do módulo `EditorThemes.ts` para harmonizar visualmente o editor com o restante da identidade da plataforma, em ambas as variações de tema claro e escuro.

O gerenciamento do editor foi encapsulado no `EditorContext`, que abriga a referência da instância Monaco, o código-fonte corrente, o nome do arquivo ativo, os marcadores de linha utilizados para destacar erros e o conjunto de configurações de exibição. A interação entre o usuário e o editor foi enriquecida pelo *hook* `useDebugger`, responsável por sinalizar visualmente, durante a execução passo a passo, qual instrução do código intermediário está

sendo processada pelo interpretador, em consonância com o requisito pedagógico de tornar visível o processo de tradução.

A persistência local dos arquivos do usuário foi modelada pelo *hook* `useFileSystem`, que implementa um sistema de arquivos virtual sustentado pelo *localStorage* do navegador. Essa abordagem permite que múltiplos arquivos sejam criados, renomeados, removidos e alternados sem qualquer dependência de servidor remoto, permitindo trabalhar com documentos locais após o carregamento da aplicação, sem comprovar funcionamento integral *offline*. O *hook* `useEditorWithFileSystem` combina o estado do editor com o sistema de arquivos virtual, garantindo a sincronização bidirecional entre o conteúdo exibido no Monaco e o conteúdo persistido localmente. Complementarmente, os *hooks* `useExplorer` e `useSearch` oferecem, no painel lateral, navegação por estrutura de arquivos e busca textual sobre todos os arquivos do projeto, replicando as funcionalidades essenciais de uma IDE profissional.

O painel lateral da IDE, implementado em `SidebarPanel`, reúne as visões de arquivos, busca, linguagem e depuração em uma composição cuja largura pode ser ajustada por arraste. O componente mantém a largura durante sua montagem e aplica um limite mínimo conforme a largura da janela. O ajuste usa eventos de ponteiro; sua presença não demonstra uma operação equivalente por teclado. A revisão também adotou cores derivadas do tema em formulários, painéis e tabelas, cujos resultados de contraste no recorte testado são apresentados na Seção 4.8.

A Figura 4.2 mostra a IDE na área de resolução de exercícios, com a linguagem Didática em Português ativa. No painel da IDE, à esquerda fica o explorador do sistema de arquivos virtual; ao centro, o editor com o arquivo `main.?`, em que `numero_inteiro`, `leia`, `se` e `inicio` recebem o destaque de palavra reservada; abaixo, o terminal com uma execução do programa. As capturas desta seção e das seguintes foram feitas em 20 de outubro de 2026, na execução local da plataforma, com as contas de demonstração do *seed* e uma lista criada para o registro, travada na linguagem predefinida Didática em Português.

4.5 Execução das Análises Léxica e Sintática a partir da IDE

A invocação das análises do compilador a partir da IDE foi modelada por meio de *hooks* dedicados que encapsulam toda a interação entre a camada de interface e o núcleo de tradução. Diferentemente da estratégia adotada por ferramentas como a plataforma Laila, discutida no

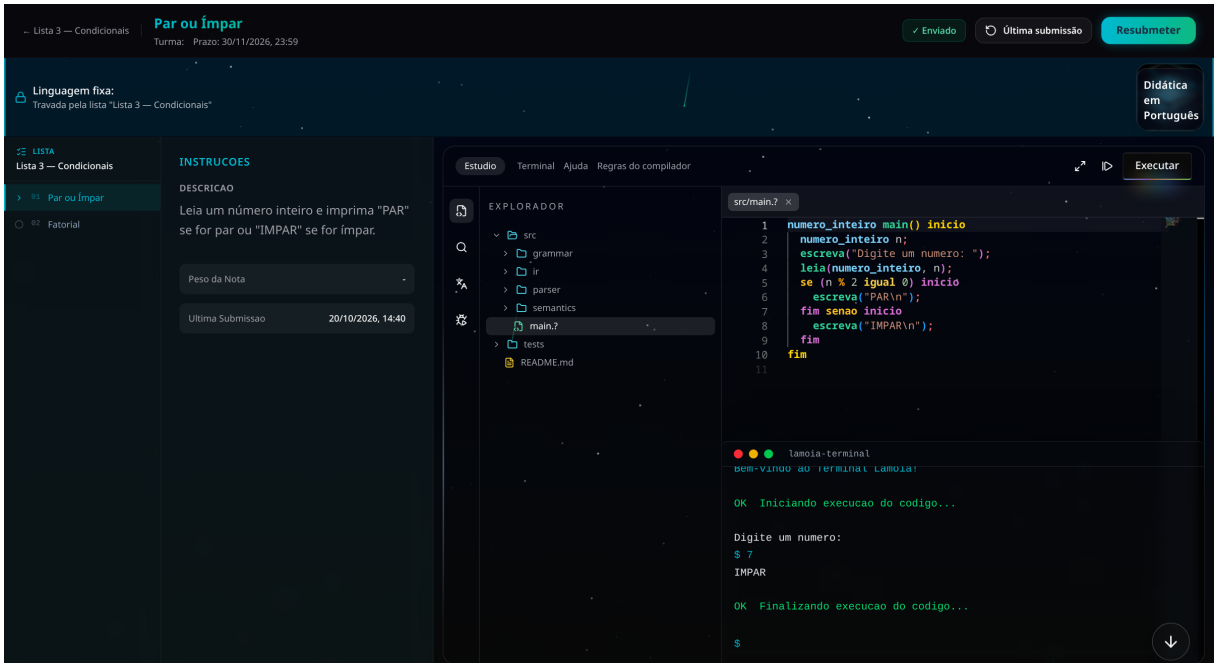


Figura 4.2 – IDE com a linguagem Didática em Português ativa e o terminal após uma execução.

Fonte: elaborado pelo autor.

referencial teórico, na qual o código submetido pelo usuário é encaminhado a um serviço remoto para processamento, neste trabalho a execução das análises foi modelada inteiramente no navegador, sem intermediação de rotas HTTP. Essa decisão arquitetural foi viabilizada pela organização *monorepo* descrita na Seção 4.1, que permite à IDE consumir o pacote `compiler` como dependência local e instanciar suas classes diretamente no contexto de execução do cliente.

O *hook* `useLexerAnalyse` concentra a orquestração da fase léxica. A cada solicitação do usuário, o *hook* recupera o código-fonte corrente, consulta o `KeywordContext` para obter a configuração ativa da linguagem dinâmica, normaliza o mapeamento efetivo de palavras-chave por meio do utilitário `buildEffectiveKeywordMap` e instancia uma nova classe `Lexer` do pacote `compiler`, passando como parâmetros o mapa de palavras-chave personalizadas, os operadores em forma de palavra, os literais booleanos, o terminador de instrução, os delimitadores de bloco e o *locale* ativo. A varredura é então executada pelo método `scanTokens`, e a coleção de *tokens* resultante, juntamente com eventuais avisos e mensagens informativas coletadas pelo sistema de *issues*, é devolvida diretamente ao componente que originou a invocação, sem qualquer ida ao servidor.

O Trecho de Código 4.7 evidencia essa instanciação direta. Observa-se a importação do símbolo `Lexer` a partir do pacote local `@ts-compilator-for-java/compiler`, viabilizada pela organização em *monorepo*, e a construção do analisador com a configuração

derivada do `KeywordContext`, sem qualquer chamada HTTP intermediária.

```
import { Lexer } from "@ts-compilator-for-java/compiler/src/lexer";

async function runLexerAnalyse(
  input: RunLexerAnalyseInput,
): Promise<TLexerAnalyseData> {
  try {
    const effectiveKeywordMap = buildEffectiveKeywordMap(input.keywordMap);
    const lexer = new Lexer(input.sourceCode, {
      customKeywords: effectiveKeywordMap,
      operatorWordMap: input.operatorWordMap,
      booleanLiteralMap: input.booleanLiteralMap,
      statementTerminatorLexeme: input.statementTerminatorLexeme,
      blockDelimiters: input.blockDelimiters,
      locale: input.locale,
      indentationBlock: input.indentationBlock,
    });
    const tokens = lexer.scanTokens();
    return {
      message: "Lexical Analysis completed",
      tokens,
      warnings: lexer.warnings,
      infos: lexer.infos,
      error: null,
    };
  } catch (error) {
    if (error instanceof IssueError) throw error;
    throw new Error((error as Error).message || "Codigo nao suportado");
  }
}
```

Listing 4.7 – Função `runLexerAnalyse`, que instancia o lexer diretamente no cliente.

Os *tokens* obtidos por essa execução local são renderizados na IDE como cartões interativos pelo componente `token-card`, oferecendo ao usuário uma visualização direta do resultado da fase léxica, incluindo o lexema original, sua categoria e sua posição no código-fonte. Essa exposição direta atende ao propósito pedagógico de tornar visíveis as etapas internas do processo de tradução, característica diferencial da plataforma em relação às ferramentas analisadas no referencial teórico. Eventuais erros lançados pelo lexer, encapsulados pela classe `IssueError`, são interceptados pelo *hook* e materializados tanto no terminal quanto no editor, por meio dos utilitários `showLineIssues` e do mecanismo de marcadores discutido na Seção 4.6.

O *hook* `useIntermediatorCode` segue uma abordagem análoga. Após reaproveitar a tabela de *tokens* produzida pela fase léxica, o *hook* instancia um `TokenIterator` e aciona o analisador sintático diretamente no cliente, integrado a um `Emitter` responsável pela geração das instruções de código intermediário em três endereços. A lista de instruções resultante é igualmente exibida na IDE em um painel dedicado, permitindo ao estudante inspecionar a tradução do código-fonte para uma forma de representação intermediária. Essa execução também ocorre integralmente no navegador, sem qualquer requisição HTTP intermediária.

A escolha por executar as análises léxica e sintática diretamente no cliente, em vez de delegá-las ao servidor por meio de rotas internas do Next.js, atende a três objetivos correlacionados. Primeiro, minimiza a latência percebida pelo usuário, ao eliminar o ciclo de requisição e resposta exigido por uma arquitetura cliente-servidor convencional. Segundo, permite acionar edição, análise e interpretação sem uma requisição ao serviço de persistência, após o carregamento da aplicação. Terceiro, simplifica a manutenção do código, uma vez que a lógica do compilador é consumida como dependência direta, sem necessidade de empacotamento, serialização ou conversão de tipos entre cliente e servidor. As rotas HTTP do servidor Next.js, situadas sob o caminho `/api`, atendem à validação automatizada de submissões e a recursos como busca de imagens de linguagem, conforme detalhado na Seção 4.7.

Por fim, o *hook* `use-api-queries` consolida, por meio do *TanStack React Query*, todas as consultas relacionadas a turmas, exercícios, listas de exercícios e submissões, oferecendo cache automático, revalidação em segundo plano e tratamento padronizado de erros para toda a aplicação. Essas consultas, distintamente das análises locais discutidas anteriormente, envolvem comunicação remota com o serviço de persistência, mantendo claras as fronteiras entre a execução local do compilador e o consumo de recursos do Ambiente Virtual de Aprendizagem.

A Figura 4.3 mostra o início da sequência de *tokens* produzida para o mesmo programa. Cada cartão traz a categoria do *token*, seu código numérico, a linha, a coluna e o lexema lido. O lexema `numero_inteiro` recebe o código 21, o mesmo de `int`, e `inicio` é classificado como abre chaves: o analisador reconhece as palavras configuradas e as associa aos *tokens* canônicos da linguagem.

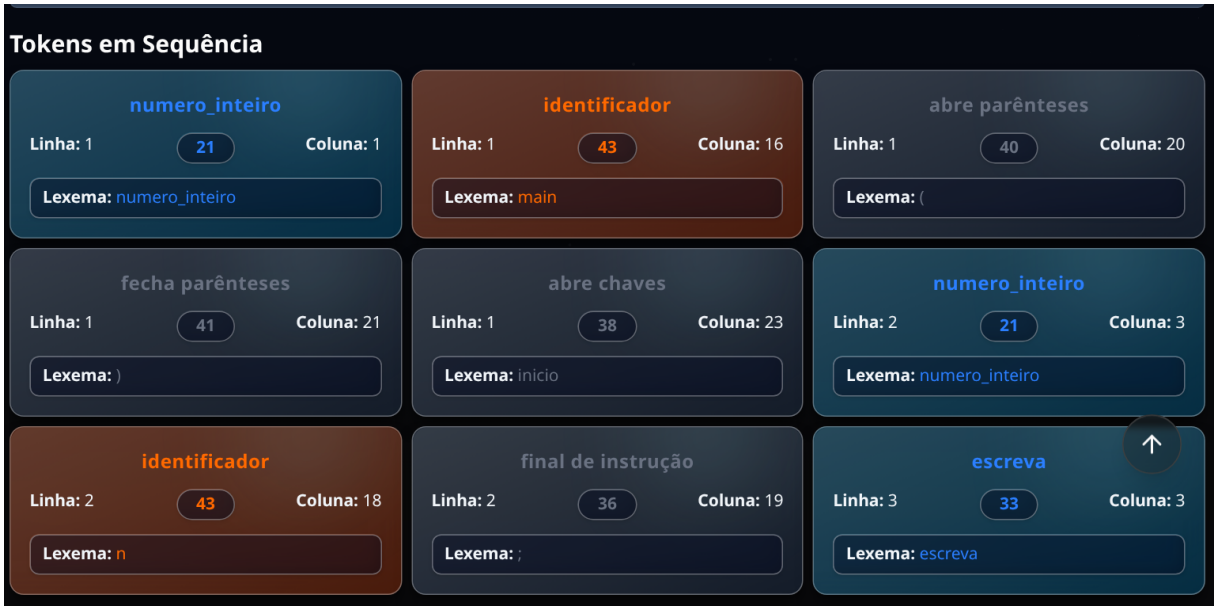


Figura 4.3 – Sequência de *tokens* gerada para um programa escrito em Didática em Português.

Fonte: elaborado pelo autor.

4.6 Execução do Código Intermediário e *Feedback* no Terminal

O componente `terminal`, hospedado em `components/terminal`, foi modelado como o canal primário de *feedback* interativo entre o programa em execução e o usuário. Sua arquitetura interna é composta por três peças: o componente `body`, que renderiza o histórico de saídas e o prompt de entrada; o componente `header`, que apresenta os controles de execução e o estado corrente; e o *hook* `useKeyboardShortcuts`, que mapeia atalhos teclados às operações de limpeza, foco e interrupção da execução. O terminal é controlado pelo `TerminalContext`, responsável por gerenciar seu estado de abertura, fechamento e visibilidade na composição da tela.

A execução dos programas foi modelada de modo que as operações `print` e `scan` produzidas pelo interpretador sejam diretamente conectadas ao terminal embutido. O `Interpreter`, instanciado no navegador durante a execução interativa e no servidor Next.js durante a validação de submissões, recebe funções de retorno (`stdout` e `stdin`) cujo comportamento varia conforme o modo de execução: em modo interativo, as funções são associadas ao terminal da IDE; em modo de validação automatizada de exercícios, são conectadas a *buffers* controlados que capturam a saída para comparação com o resultado esperado. Esse desacoplamento entre o motor de execução e a camada de apresentação atende aos princípios de baixo acoplamento discutidos por Martin (2019) e foi inspirado nas abstrações de fluxos de entrada e saída descritas

por Cooper e Torczon (2012).

Na depuração, a função `appendOutputLines` acumula os fragmentos de saída mantendo o estado da linha aberta. Fragmentos sem quebra de linha são concatenados, enquanto as sequências de fim de linha são normalizadas. Assim, a chegada de um novo fragmento durante um passo de execução não cria, por si só, uma linha adicional no terminal.

O núcleo compartilhado também passou a oferecer igualdade estrita (===) e expressão condicional ternária (`condicao ? valorA : valorB`). Incompatibilidades de atribuição e retorno podem gerar aviso na análise e erro durante a execução. A interface consome esses diagnósticos pelos mesmos canais de avisos e erros usados pelas demais construções. A igualdade estrita e os símbolos do ternário não integram os campos de operadores em forma de palavra do assistente atual.

No exemplo da Figura 4.2, o programa escreve “Digite um numero:”, recebe 7 pelo terminal, imprime `IMPAR` e o terminal registra o início e o fim da execução.

O sistema de *feedback* de erros foi modelado em duas categorias complementares. Os erros estáticos detectados durante a compilação, como falhas léxicas, sintáticas e semânticas, são reportados pelo módulo `issue` do pacote `compiler`, que classifica cada ocorrência como `IssueError` (impede a tradução), `IssueWarning` (alerta não fatal) ou `IssueInfo` (informação contextual). Essas mensagens são interceptadas pela IDE e materializadas tanto no terminal, em formato textual, quanto no editor, por meio de marcadores de linha sublinhados, em conformidade com as práticas de relato de erros discutidas por Aho, Sethi e Ullman (1995). Os erros dinâmicos, detectados pelo interpretador em tempo de execução (como divisões por zero, conversões impossíveis durante a leitura de entrada ou acessos a posições inválidas de *arrays*), são encapsulados pela classe `RuntimeError` e gerenciados pelo `RuntimeErrorContext`, que coordena a exibição contextualizada no terminal e o destaque visual no editor.

Para garantir que mensagens de erro sejam pedagogicamente úteis, mesmo quando a linguagem corrente está fortemente personalizada, todas as mensagens passam pelo módulo de internacionalização `i18n` do pacote `compiler`, que oferece traduções para os idiomas *pt-BR*, *pt-PT*, *es* e *en*. Adicionalmente, as mensagens são adaptadas ao vocabulário corrente da customização: termos como “palavra reservada esperada” são substituídos pela palavra-chave efetivamente esperada na configuração ativa, evitando a exposição ao usuário de termos canônicos que tenham sido propositalmente ocultados pela personalização.

4.7 Sistema de Correção e Avaliação Automatizada

A plataforma incorpora um subsistema de avaliação automatizada relacionado ao ciclo de submissão e veredito documentado pelo Beecrowd (Beecrowd, 2026), materializado pela rota `/api/submissions/validate`. A modelagem do fluxo de submissão de exercícios partiu da estrutura conceitual desses juízes, mas foi adaptada ao contexto da linguagem dinâmica: enquanto os juízes tradicionais exigem que o aluno se adapte à sintaxe rígida de uma linguagem comercial, a plataforma proposta permite que cada exercício carregue sua própria configuração da linguagem, podendo, inclusive, restringir o aluno a um vocabulário específico definido pelo professor.

Entre as entidades persistidas estão `User`, `Class`, `Exercise`, `ExerciseList`, `Submission` e `Language`. A linguagem armazena a configuração do vocabulário e das variantes gramaticais; não se resume à imagem utilizada como identificação visual. Exercícios e listas podem definir uma política aberta ou vincular uma linguagem obrigatória, e a submissão registra uma cópia da configuração utilizada. O FastAPI e o SQLAlchemy mantêm esses registros, enquanto o *TanStack React Query* gerencia consultas e atualização de dados na interface.

Na avaliação automática, a rota Next.js recebe código e configuração, normaliza as opções, executa a análise e gera as instruções intermediárias uma vez. Em seguida, consulta os casos de teste, executa cada caso com entrada simulada, coleta a saída e a compara à esperada após normalização de quebras de linha e remoção de espaços finais. A resposta reúne os vereditos; fora do modo de simulação, o resultado é encaminhado ao FastAPI para registro. A limitação temporal implementada por uma promessa concorrente não constitui isolamento de processo nem demonstra contenção de todo programa que bloqueie a execução.

4.7.1 Autoria de exercícios e acompanhamento de turmas

A criação de exercícios envia enunciado, política de linguagem e casos de teste na mesma requisição. A edição completa usa `PUT /exercises/{id}`, substituindo também a coleção de casos de teste, que recebe índices de ordenação conforme a sequência enviada. O serviço verifica se o usuário é o professor responsável e valida a política de linguagem. A atualização parcial por `PATCH` continua disponível para os campos do exercício. Na interface, o diálogo de edição reúne os campos existentes, permite editar, acrescentar e remover casos de teste e atualiza

as consultas após a gravação.

A listagem de turmas passou a devolver, além dos dados da turma, um resumo do professor e as contagens de membros e listas publicadas. Os cartões do painel utilizam esse contrato para apresentar o contexto da turma. Esses resumos são calculados a partir dos relacionamentos existentes e não acrescentam tabelas ao modelo de dados.

4.7.2 Resolução de exercícios e linguagem obrigatória

A área de resolução foi decomposta em componentes de cabeçalho, instruções, navegação da lista, painel de submissão e composição da IDE, no diretório `components/exercise-workspace`. O editor recebe um escopo de armazenamento por exercício e pode iniciar com o código da última submissão disponível. O cabeçalho distingue envio e reenvio, apresenta a nota existente e, quando há uma publicação com prazo vencido, substitui o botão de envio por uma indicação de prazo encerrado. Essa regra de interface não substitui a validação no serviço.

O painel de submissão separa erros de tradução, avisos e resultados dos casos de teste, incluindo quantos passaram e o total executado. A navegação lateral permite alternar entre os itens de uma lista e indicar as submissões disponíveis nesse contexto. Esses recursos dão continuidade ao trabalho no exercício, sem alterar a divisão entre execução local e avaliação no servidor.

Quando o exercício ou a lista exige uma linguagem, a API resolve a precedência entre a restrição do exercício, a da lista e o modo livre. O `KeywordContext` aplica a configuração recebida como uma sobreposição temporária, com nome, descrição e imagem da linguagem obrigatória. O seletor passa a apresentar apenas essa opção e bloqueia a troca; o estado temporário não é gravado como a preferência pessoal no armazenamento local. Ao encerrar a sobreposição, a configuração anterior é restaurada. O aviso de linguagem obrigatória identifica se a restrição veio do exercício ou da lista.

A Figura 4.4 mostra a área de resolução depois de um envio. A faixa superior informa que a linguagem está travada pela lista e exibe o cartão da Didática em Português. O painel de submissão indica que um dos dois casos de teste passou; o caso que falhou está expandido, com a entrada 4, a saída esperada `PAR` e a saída obtida `IMPAR`. O erro vem do próprio programa enviado, que usa divisão (`n / 2`) onde deveria usar o resto (`n % 2`).

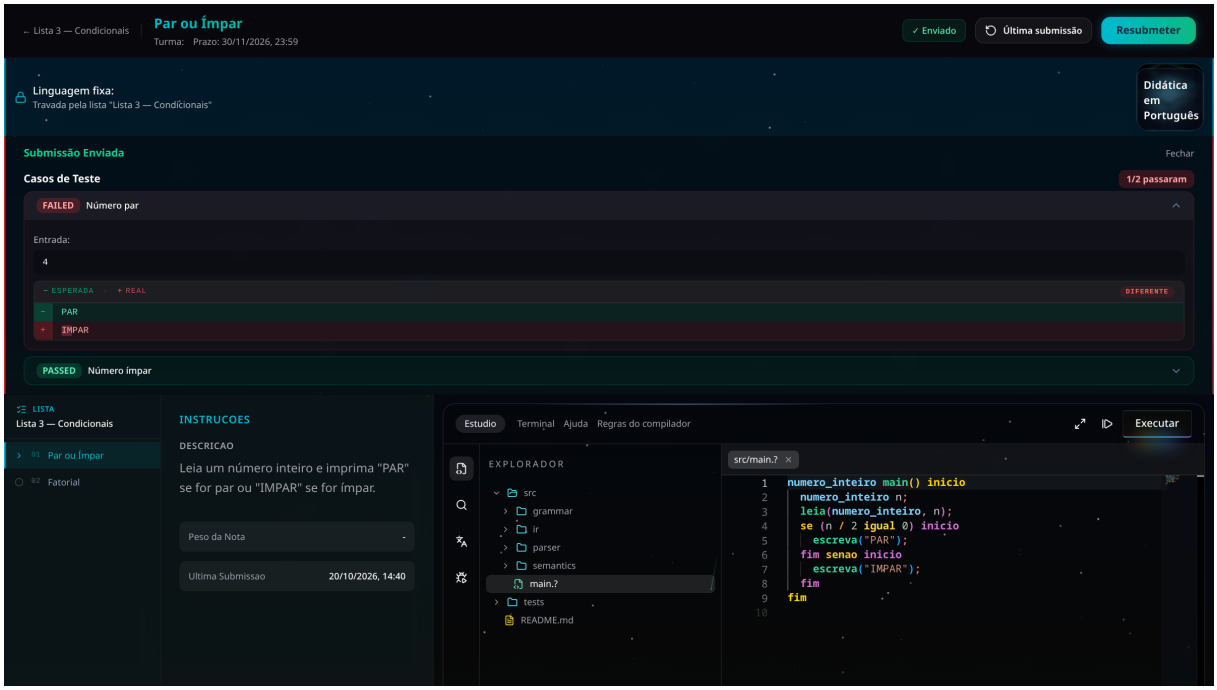


Figura 4.4 – Área de resolução após o envio, com linguagem obrigatória e resultado dos casos de teste.

Fonte: elaborado pelo autor.

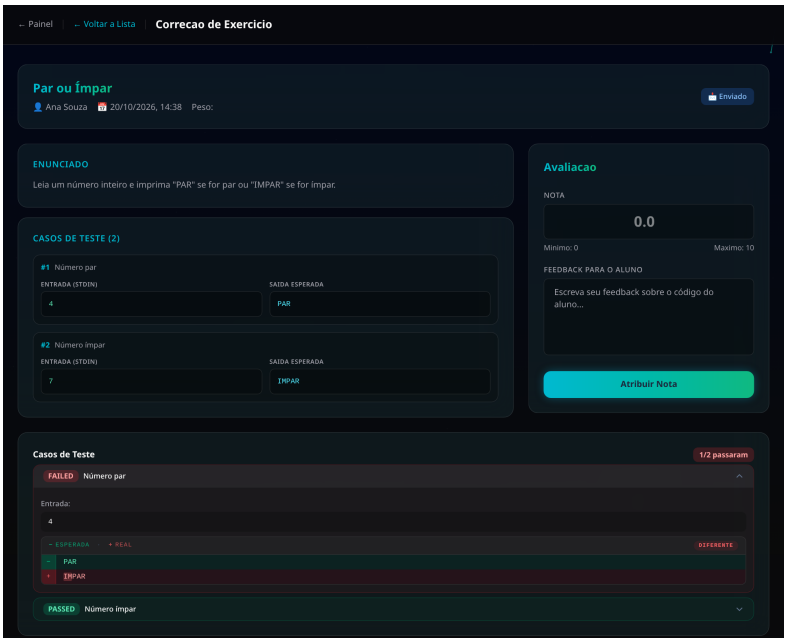
4.7.3 Correção de submissões pelo professor

A página de uma submissão, `/submissions/[id]`, reúne o enunciado e os casos de teste do exercício, o painel de nota e comentário e, abaixo deles, uma IDE completa carregada com a tentativa do aluno. O componente `SubmissionIdePanel` abre o editor com o código salvo em `code_snapshot` e aplica a configuração de `language_snapshot` como sobreposição temporária no `KeywordContext`, o mesmo mecanismo usado pela linguagem obrigatória. O professor lê e executa o programa no vocabulário que o aluno usou, sem trocar a própria linguagem ativa, que é restaurada ao sair da página. O editor usa um escopo de armazenamento próprio para cada submissão, e um botão devolve o código ao estado enviado depois de alterações feitas durante a correção.

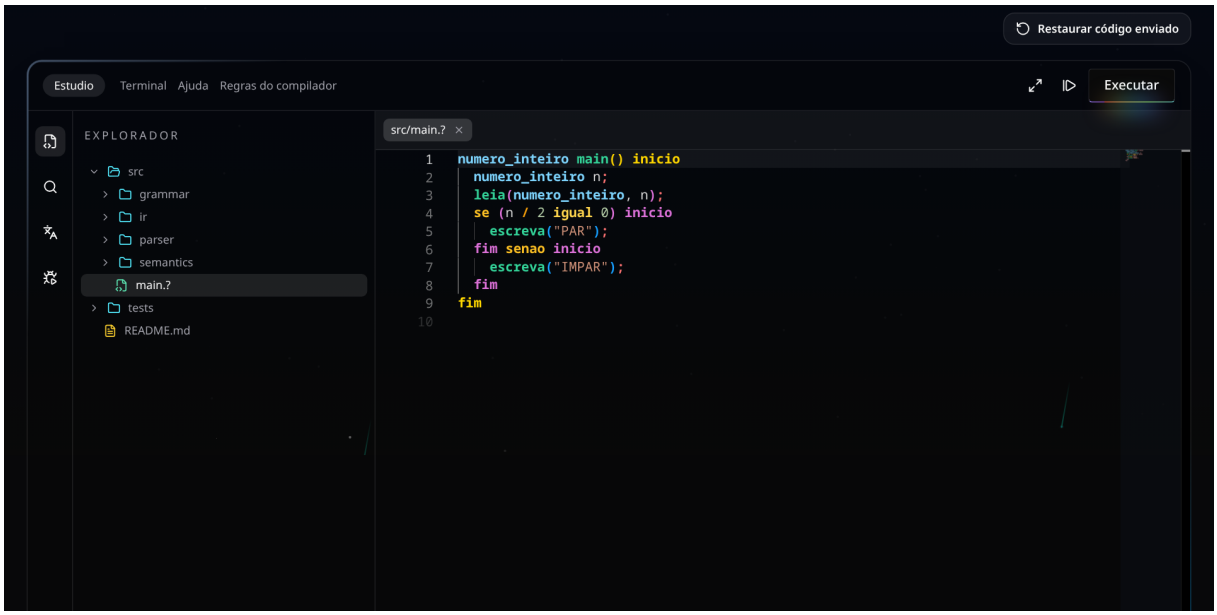
Acima da IDE aparece o resultado de cada caso de teste registrado no envio: entrada, saída esperada, saída obtida e se o caso passou. Esses dados vêm do campo `test_case_results`, acrescentado à tabela de submissões em outubro. Antes dessa mudança, a rota de validação devolvia os resultados por caso ao aluno, mas eles não eram gravados, e o professor via apenas o estado e a nota da submissão. Os resultados mostrados são os da validação feita no envio; executar o código de novo na IDE da correção não os altera.

A Figura 4.5 mostra a mesma submissão aberta pelo professor. Na parte superior ficam o

enunciado, os casos de teste cadastrados, o painel de nota e comentário e o resultado registrado no envio, com a diferença entre a saída esperada e a obtida (Figura 4.5a). Abaixo, a IDE abre com o código do aluno e com o destaque da linguagem usada no envio; acima dela fica o botão que restaura o código enviado (Figura 4.5b).



(a) Enunciado, casos de teste, avaliação e resultado registrado.



(b) IDE carregada com o código e a linguagem do envio.

Figura 4.5 – Correção de uma submissão pelo professor.

Fonte: elaborado pelo autor.

4.8 Acessibilidade, Responsividade e Finalização de Fluxos

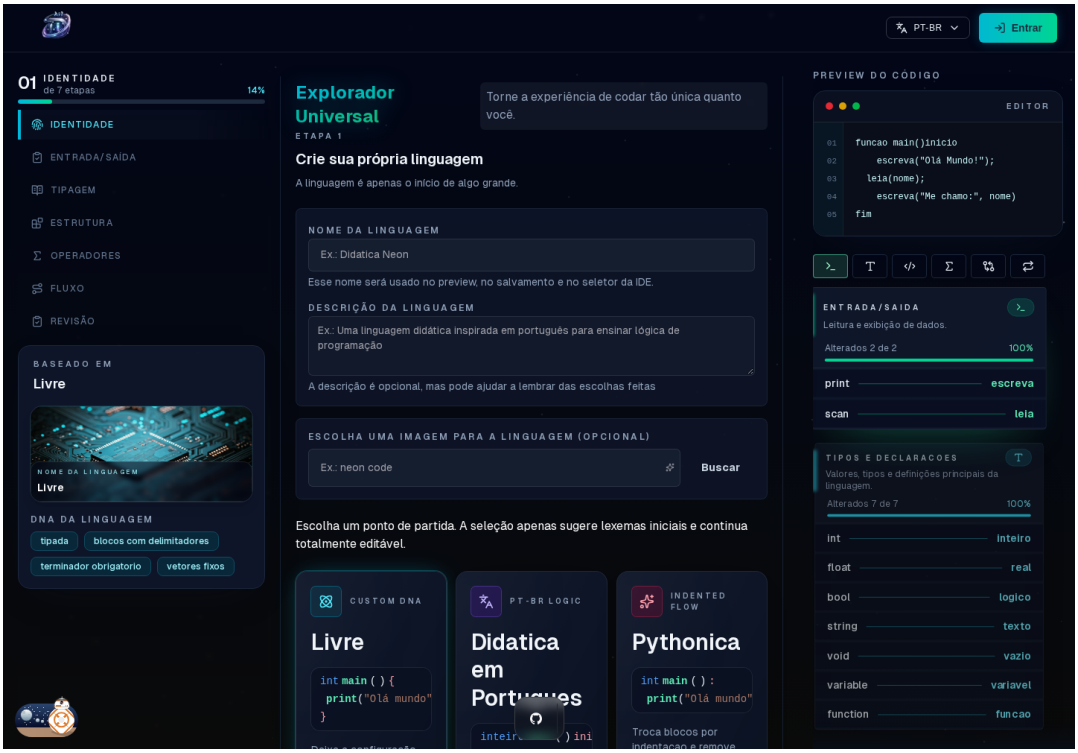
A acessibilidade foi examinada por inspeção de componentes e por uma verificação técnica delimitada, apresentada a seguir. O módulo `ui/dialog.tsx` encapsula as primitivas de diálogo, título e descrição do Radix. O componente `WizardStepper` usa botões nativos e mantém os nomes das etapas com a classe `sr-only` quando os rótulos deixam de ser visíveis em telas estreitas. Entretanto, a indicação de etapa ativa nesse componente é visual, sem atributo `aria-current`. Essas evidências mostram recursos e lacunas da implementação; não permitem declarar conformidade global com WAI-ARIA ou WCAG.

A adaptação visual usa utilidades responsivas do Tailwind CSS. No assistente, abaixo do limite `lg`, a lista de etapas assume disposição horizontal, os botões tornam-se circulares e o cartão da linguagem-base é ocultado. Acima desse limite, as etapas ficam na lateral. O *hook* `useBreakpoint` também disponibiliza consultas por largura, mas não é o único mecanismo de adaptação da aplicação. Na IDE, a composição usa `flex-col` e `sm:flex-row` para reorganizar os painéis. A organização visual deve ser verificada em cada fluxo, pois a existência dessas classes não demonstra, isoladamente, ausência de cortes ou de rolagem indevida.

4.8.1 Verificação técnica do assistente

Em 16 de setembro de 2026, foi verificada a rota local `/language-creator`, sem autenticação, em Chromium 153.0.8010.12, usando Playwright para interação e axe-core 4.13.0 para verificações automatizadas. Foram combinadas duas larguras (1440 e 390 pixels CSS) com os temas claro e escuro, em quatro cenários. O portal de ferramentas de desenvolvimento do Next.js foi ocultado e excluído da análise. A verificação abrangeu a etapa inicial do assistente e a navegação para entrada/saída; não incluiu contas, turmas, submissões ou leitor de tela.

Nos quatro cenários, cinco acionamentos de Tab alcançaram o botão da primeira etapa; Tab seguido de Enter ativou a segunda etapa. A lista de etapas apresentou direção vertical em 1440 pixels e horizontal em 390 pixels. A largura de rolagem do documento coincidiu com a largura da janela nos dois tamanhos. Esses resultados caracterizam o estado testado, não todos os componentes e estados da plataforma. A Figura 4.6 apresenta as capturas correspondentes no tema escuro.



(a) Janela de 1440 por 1000 pixels CSS: etapas na lateral e conteúdo em colunas.



(b) Janela de 390 por 844 pixels CSS: etapas em linha e cartões empilhados.

Figura 4.6 – Adaptação do assistente na versão verificada em setembro de 2026.

Fonte: capturas locais da aplicação, sem autenticação.

A análise automatizada selecionou as regras A e AA disponíveis no axe para WCAG 2.0, 2.1 e 2.2. A Tabela 4.2 resume as ocorrências reportadas por regra e cenário. Uma ocorrência corresponde a um elemento identificado pela ferramenta; as contagens de cenários distintos não devem ser somadas como problemas únicos. O relatório também contém itens inconclusivos, sobretudo de contraste, que requerem inspeção manual.

Tabela 4.2 – Ocorrências reportadas pelo axe no recorte do assistente.

Regra	**Claro** 1440	**Claro** 390	**Escuro** 1440	**Escuro** 390
Botão sem nome acessível	1	1	1	1
Contraste insuficiente	2	14	5	5
Título de documento ausente	1	1	1	1
Link sem nome acessível	0	0	0	0
Elementos interativos aninhados	6	6	6	6
Região rolável sem foco	3	0	3	0
Alvo de interação pequeno	1	1	1	1

Fonte: relatório automatizado da revisão de 16 de setembro de 2026.

Esses achados impedem tratar a acessibilidade como requisito concluído. A aprovação da transição por teclado mostra apenas que os dois passos testados são acionáveis nesse contexto. Permanecem necessárias a correção e a inspeção dos problemas reportados, a verificação do foco visível e dos diálogos, a análise de outras páginas e testes com tecnologias assistivas, conforme a proposta da Seção 4.12. O *script* e o relatório estruturado da verificação acompanham este texto no repositório, nos diretórios `scripts` e `verificacao`.

Os temas claro e escuro são administrados pelo `ThemeContext`, que aplica a classe `dark` e persiste a preferência no navegador, com alternância dos temas do Monaco. As Figuras 4.11a e 4.11b ilustram essa escolha; a aparência de uma captura não substitui a medição de contraste. As notificações são centralizadas no `ToastContext`.

4.8.2 Internacionalização da interface

O requisito RNF03 pede que toda cadeia visível ao usuário esteja externalizada, com suporte a *pt-BR*, *pt-PT*, *es* e *en*. A configuração do Next.js declara esses quatro locais, e os textos ficam no diretório `i18n/locales`, com um subdiretório por idioma. Cada idioma tem oito arquivos, separados por área da aplicação: `ui`, `wizard`, `toast`, `ide`, `footer`, `language-documentation`, `language-sample` e `compiler-rules`. Em *pt-BR*, somam 1.690 linhas.

Os componentes obtêm o texto de duas formas. A maioria chama a função `t`, exportada por `i18n`, passando o local corrente e a chave desejada. O assistente de personalização usa o *hook* `useWizardTranslation`, que lê o local pelo roteador do Next.js e acrescenta o prefixo `wizard.` às chaves, evitando repetir esse prefixo em cada etapa. As mensagens de erro do compilador seguem caminho próprio, descrito na Seção 4.6: são traduzidas pelo módulo `i18n` do pacote `compiler` e adaptadas ao vocabulário da linguagem ativa.

Dois conteúdos didáticos acompanham o idioma escolhido. O primeiro são os arquivos iniciais da IDE, declarados em `defaultFiles.ts`: a área de trabalho não começa vazia, mas com uma árvore que reproduz a estrutura de um tradutor, em que cada arquivo traz um texto comentado sobre a fase correspondente. Ao trocar de idioma esses arquivos são reescritos, com uma exceção: o arquivo do programa do próprio estudante é preservado, para que a troca não descarte o trabalho já feito. O segundo é a amostra de linguagem exibida no painel lateral, montada pela função `buildHelloWorldSample` a partir da personalização ativa; os identificadores e as cadeias do exemplo também vêm do pacote de idioma, de modo que um exemplo em português não apresente nomes de variáveis em inglês.

A cobertura, porém, é parcial. Na revisão de 22 de setembro, 56 dos 180 componentes da interface usavam o mecanismo de tradução. Na versão de 19 de outubro são 98 de 187, contados da mesma forma: arquivos `.tsx` de `components`, `views` e `pages` que importam `i18n`. Nesse intervalo passaram a ser traduzidos as listas e o catálogo de exercícios, as telas de submissão e de linguagens, o cadastro e a entrada no sistema. Continuam com cadeias fixas a área de resolução de exercícios, o aviso de linguagem obrigatória e o painel de resultados dos casos de teste. Em `components/exercise-workspace`, por exemplo, a mensagem "Submissão enviada com sucesso!" ainda está escrita em português no código. Boa parte dos 89 componentes restantes não exibe texto ao usuário, como as primitivas de `components/ui` e os fundos animados, de modo que a contagem não mede diretamente quanto texto falta traduzir. O requisito RNF03 ainda não pode ser considerado cumprido.

O `AuthContext` recupera o perfil autenticado e o utilitário `auth-cookies` mantém o token em um *cookie* acessível por JavaScript. A interface distingue os papéis administrador, professor, aluno e comunidade, e o modelo de dados inclui ainda um papel interno de sistema. A restrição visual de páginas não substitui as verificações de autorização no serviço. Esta revisão descreve a implementação e não constitui auditoria de segurança.

4.9 Recursos de Orientação ao Usuário

A revisão de outubro acrescentou recursos que explicam a plataforma a quem a usa pela primeira vez: tutoriais guiados, um diálogo com um exemplo da linguagem ativa, um diálogo com as regras do compilador e uma página sobre o projeto. Nenhum deles altera a linguagem configurada ou a execução dos programas.

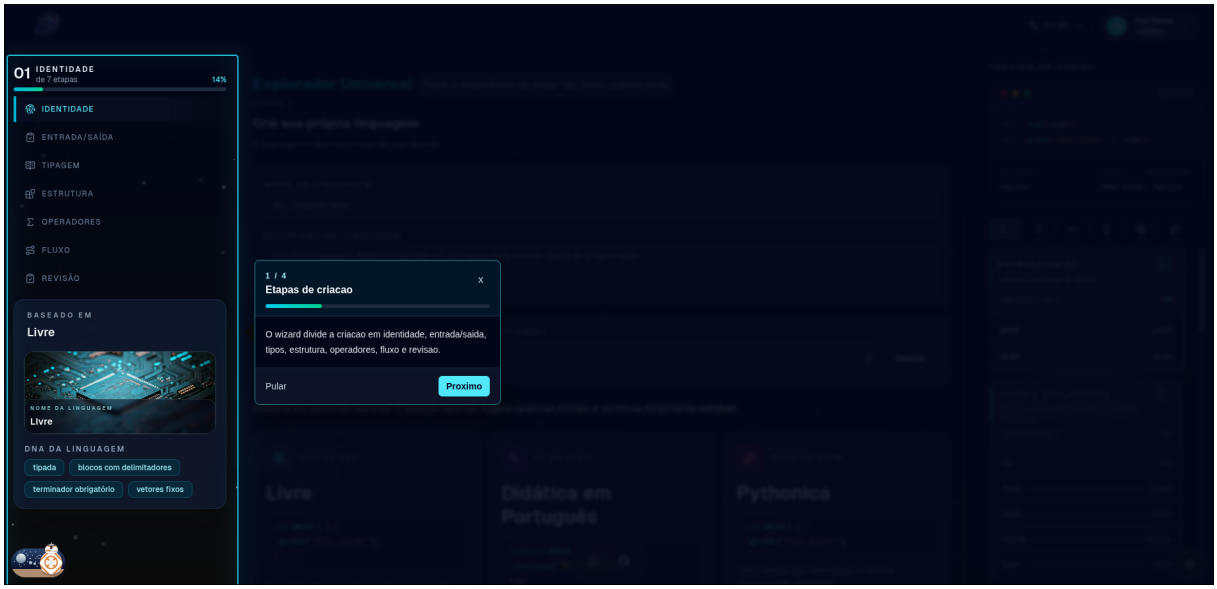
4.9.1 Tutoriais guiados

Os tutoriais usam a biblioteca *React Joyride* e estão declarados em `lib/tutoriais.ts`. São cinco roteiros, cada um ligado a uma tela: a IDE, com nove passos; o assistente de criação de linguagem, com quatro; a criação de exercícios, com cinco; e os painéis do professor e do aluno, com três e dois passos. Cada passo aponta para um elemento marcado com o atributo `data-tour`, e o título e o texto de cada passo vêm dos arquivos de idioma, nos quatro locais.

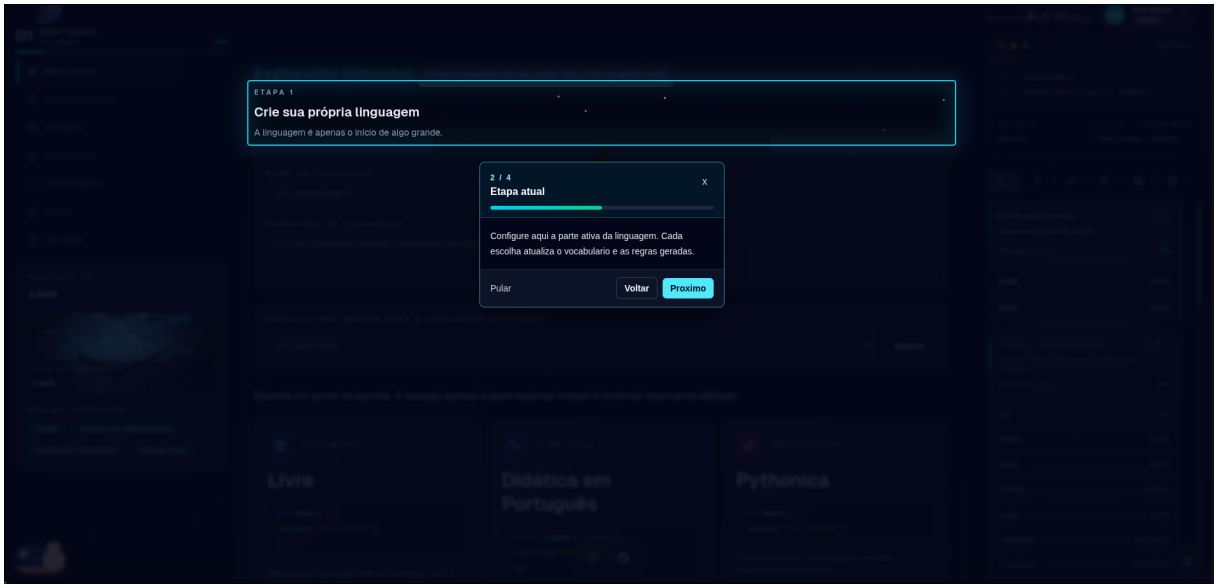
O tutorial não abre sozinho. O rodapé mostra um botão de ajuda quando a rota corrente tem um roteiro associado, e o roteiro começa quando o usuário clica nele. A escolha depende também do perfil: em `/exercises` o roteiro só aparece para professores, e em `/dashboard` professor e aluno recebem roteiros diferentes. Durante o tutorial, a tecla `Enter` avança e `Esc` encerra. Se o elemento de um passo não estiver na tela, o passo é pulado.

As Figuras 4.7 e 4.8 mostram os quatro passos do roteiro do assistente de criação de linguagem. Em cada passo, o restante da tela fica escurecido e o elemento indicado aparece destacado, ao lado de um balão com o número do passo, uma barra de progresso, o título e o texto. O balão tem os botões “Pular” e “Proximo”; “Voltar” aparece a partir do segundo passo, e no último passo “Proximo” dá lugar a “Concluir”. O primeiro passo aponta para a lista das sete etapas do assistente (Figura 4.7a), o segundo para o formulário da etapa corrente (Figura 4.7b), o terceiro para a prévia do código (Figura 4.8a) e o quarto para o rodapé com os botões de navegação do assistente (Figura 4.8b). Os textos em português aparecem sem acentos, como em “criacao” e “Proximo”, porque estão gravados assim no arquivo de idioma `pt-BR`.

Ao terminar ou pular um roteiro, a conclusão é gravada no armazenamento local do navegador, em uma chave com o prefixo `di:tutorial:`. Na versão examinada, esse registro não é lido em nenhum outro ponto da interface, de modo que concluir um tutorial não muda o que o usuário vê depois.



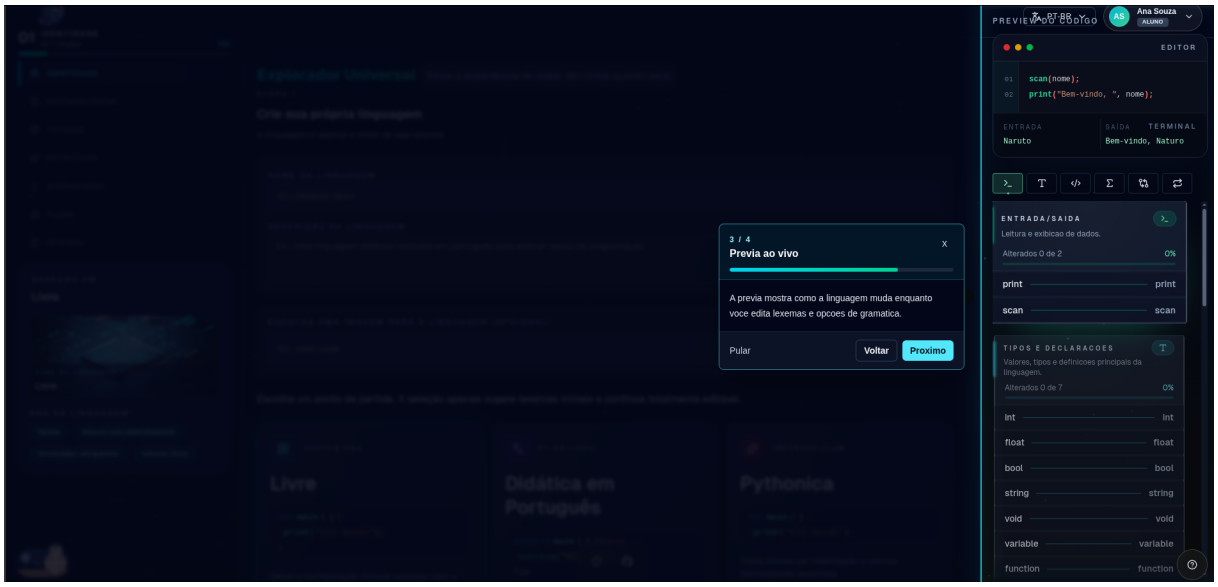
(a) Passo 1: etapas de criação.



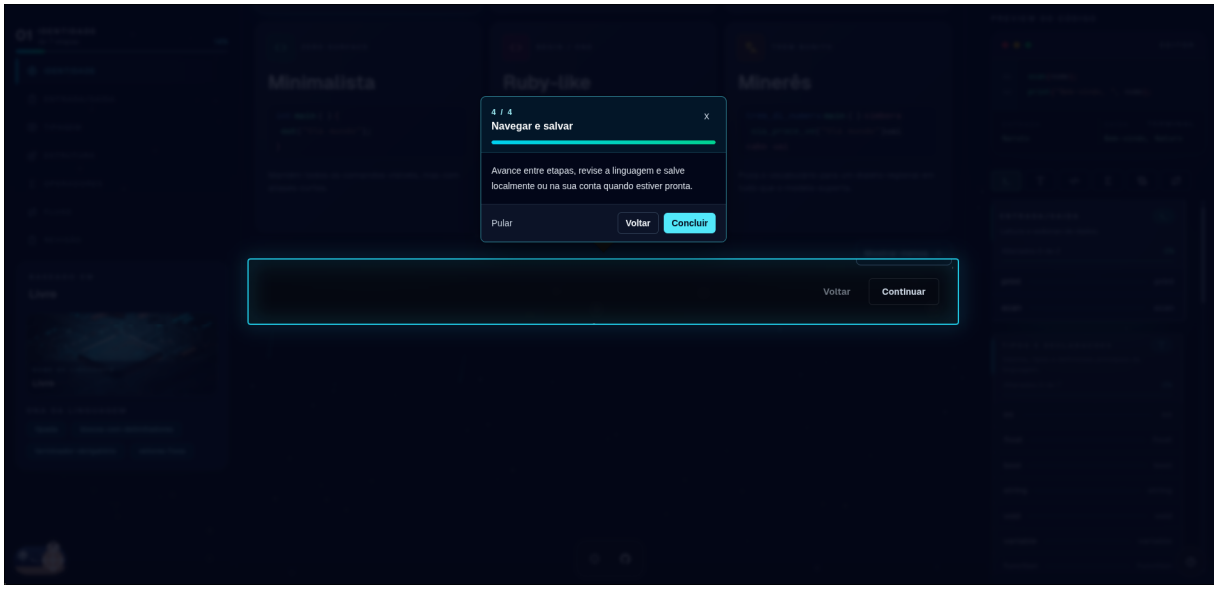
(b) Passo 2: etapa atual.

Figura 4.7 – Passos 1 e 2 do tutorial do assistente de criação de linguagem.

Fonte: elaborado pelo autor.



(a) Passo 3: prévia ao vivo.



(b) Passo 4: navegar e salvar.

Figura 4.8 – Passos 3 e 4 do tutorial do assistente de criação de linguagem.

Fonte: elaborado pelo autor.

4.9.2 Exemplo da linguagem e regras do compilador

O menu da IDE ganhou dois botões ao lado do botão do terminal: Ajuda e Regras do compilador. Os três ficam ocultos em telas mais estreitas que o limite `md` do Tailwind CSS. Ajuda abre um diálogo com o exemplo da linguagem ativa, montado com os lexemas da personalização corrente, o mesmo exemplo descrito na Subseção 4.8.2. Pelo diálogo, o usuário pode substituir o arquivo selecionado no editor por esse código.

Regras do compilador abre um diálogo com sete grupos de regras: formato do programa; blocos e finais de instrução; entrada, saída e atualizações; modo tipado; arranjos; literais de arranjo; e modo sem tipo. Os textos e os trechos de código vêm do arquivo `compiler-rules` de cada idioma e usam sempre as palavras canônicas, como `int`, `print` e `if`, qualquer que seja a linguagem ativa. Um estudante que configurou `escreva` no lugar de `print` lê a regra escrita com `print`. O pacote `compiler` tem um resumo próprio das regras, em `language-rules.ts`, que o diálogo não usa; o texto complementar registra essa duplicação.

A Figura 4.9 mostra os dois diálogos abertos na IDE de correção, com a Didática em Português aplicada. O exemplo da linguagem usa `numero_inteiro`, `se`, `inicio` e `fim` (Figura 4.9a). O diálogo de regras, aberto na mesma tela, apresenta os trechos com `int`, `print`, `if` e chaves (Figura 4.9b).

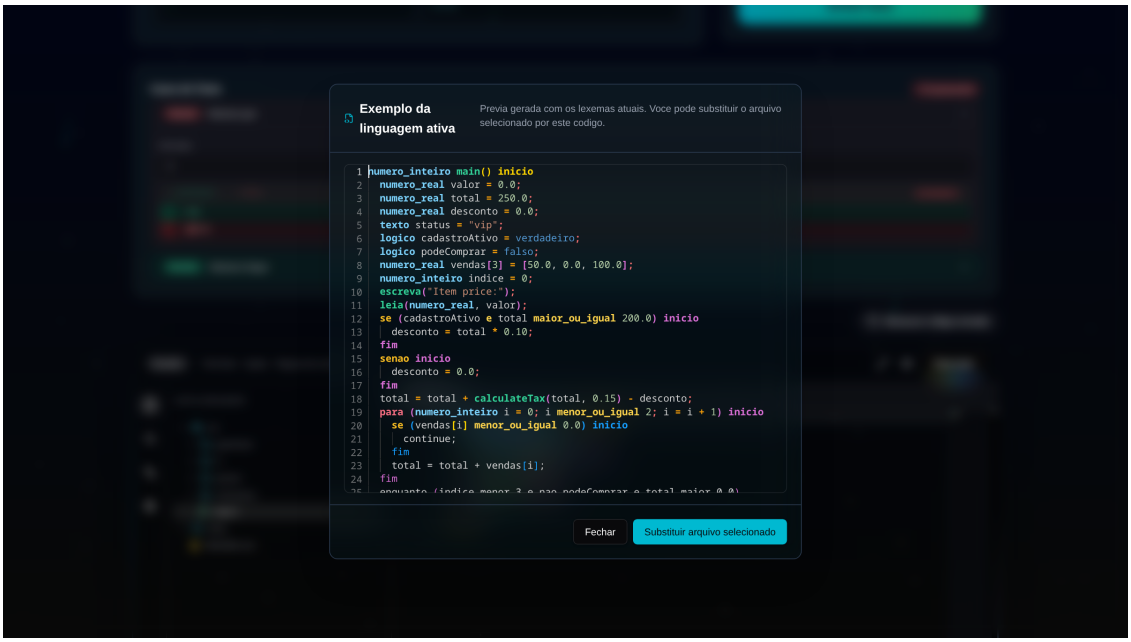
4.9.3 Página Sobre

A página `/about`, aberta pelo ícone de informação do rodapé, apresenta o projeto como trabalho de conclusão de curso. Ela traz os autores, a área, quatro destaques da plataforma e uma galeria com quatro capturas de tela: a IDE, o painel de *tokens*, a criação de linguagem e o painel do professor. Como as demais telas novas, a página obtém seus textos dos arquivos de idioma.

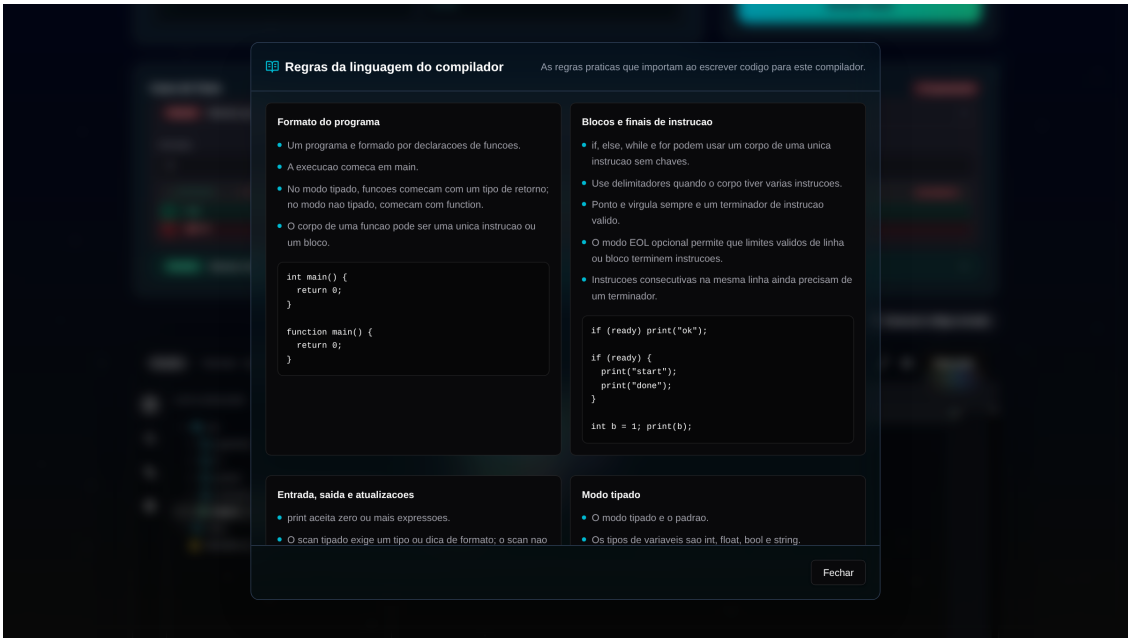
A Figura 4.10 mostra a parte inicial da página, com o título do projeto, a descrição, os autores, a área e a primeira imagem da galeria.

4.10 Apresentação das Telas da Plataforma

Com o intuito de tornar tangíveis as decisões arquiteturais, os *contexts* de estado, os *hooks* e os componentes discutidos ao longo das seções anteriores, esta seção preserva as capturas das principais telas produzidas nas etapas anteriores. Elas documentam a evolução dos fluxos e não uma reprodução integral da aparência da revisão de 16 de setembro de 2026. As



(a) Exemplo da linguagem ativa.



(b) Regras do compilador.

Figura 4.9 – Diálogos de ajuda da IDE com a linguagem Didática em Português ativa.

Fonte: elaborado pelo autor.

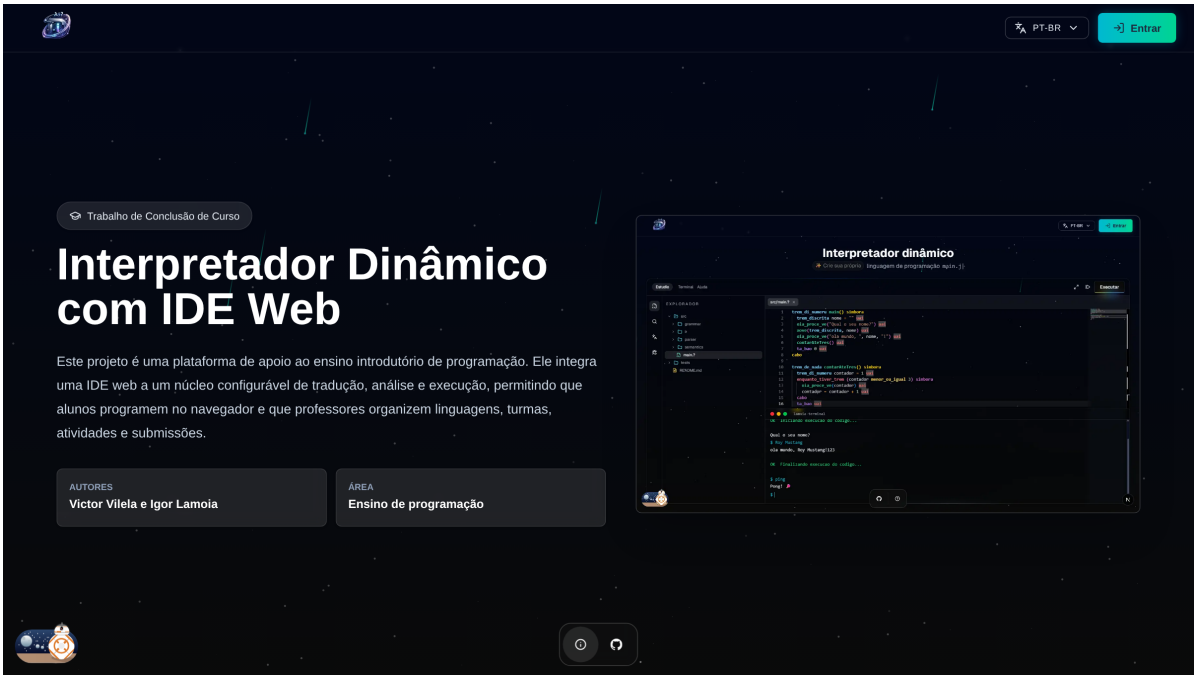


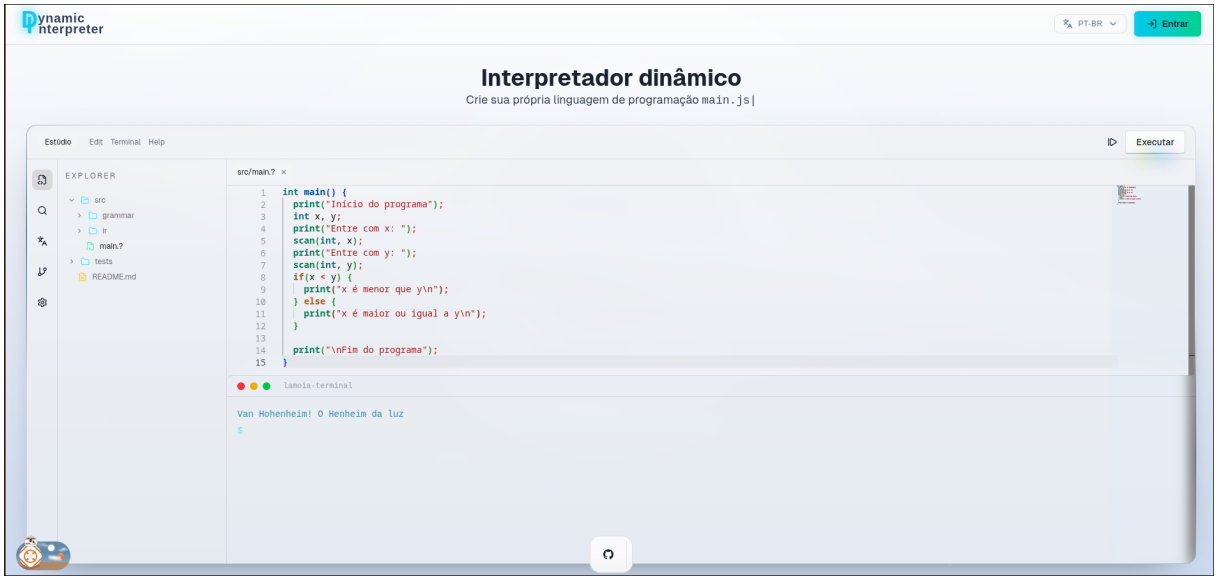
Figura 4.10 – Página Sobre.

Fonte: elaborado pelo autor.

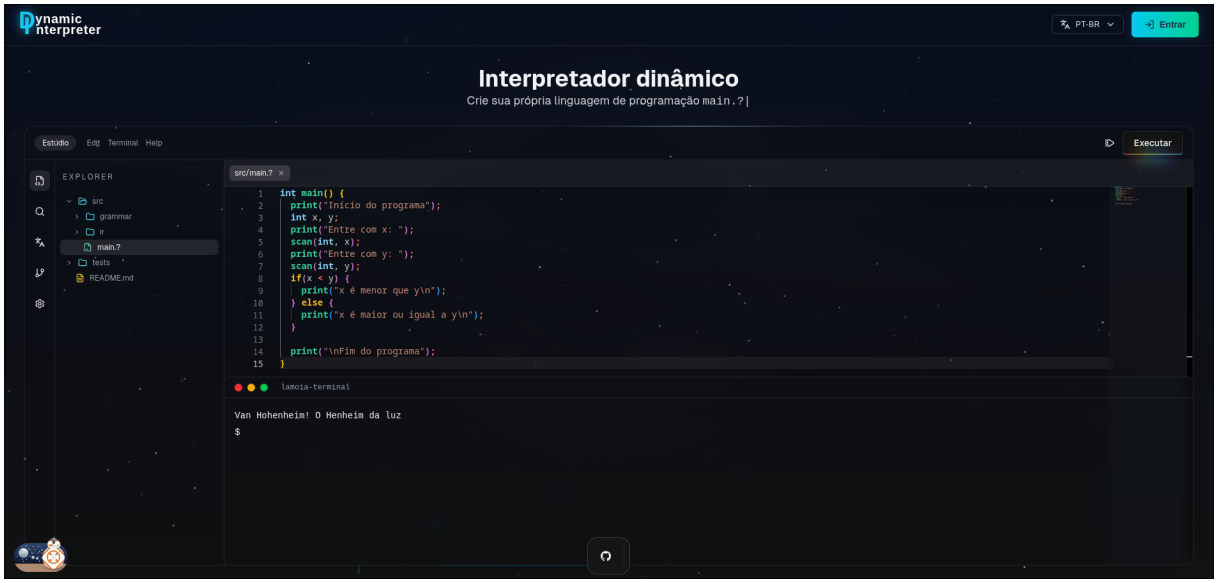
alterações recentes de seleção de linguagem, edição e resolução de exercícios são descritas nas seções anteriores; as capturas da verificação técnica do assistente têm data e escopo próprios. A organização adotada não segue a ordem em que as telas foram construídas, mas as funcionalidades especificadas na Tabela 3.2: cada bloco a seguir reúne as telas que materializam um conjunto de requisitos, cujos identificadores são indicados no próprio título. Dessa forma, é possível verificar diretamente quais requisitos já dispõem de interface implementada ao término desta etapa.

Acesso à plataforma e identidade visual (RF01)

A página inicial materializa a identidade visual da plataforma e funciona como porta de entrada para os fluxos de autenticação, navegação e divulgação dos recursos disponíveis. A composição segue o sistema de temas claro e escuro discutido na Seção 4.8, com troca controlada pelo `ThemeContext`, e foi modelada com componentes da biblioteca *shadcn/ui* para garantir consistência tipográfica, contraste adequado e respeito às diretrizes de acessibilidade WAI-ARIA. As Figuras 4.11a e 4.11b apresentam, lado a lado, as duas variações de tema, evidenciando o cuidado com a paridade visual entre as versões clara e escura.



(a) Tema claro.



(b) Tema escuro.

Figura 4.11 – Página inicial da plataforma em suas duas variações de tema.

Fonte: elaborado pelo autor.

Consulta de turmas, listas e exercícios pelo aluno (RF15 e RF18)

O painel do aluno consolida, em uma única tela, todas as turmas em que o estudante está matriculado, as listas de exercícios pendentes e o histórico recente de submissões. A tela foi modelada para reduzir o número de cliques necessários até o exercício seguinte, oferecendo cartões interativos que conduzem o aluno diretamente à composição da IDE com a linguagem dinâmica já configurada pelo professor. A Figura 4.12 ilustra a disposição geral desse painel.

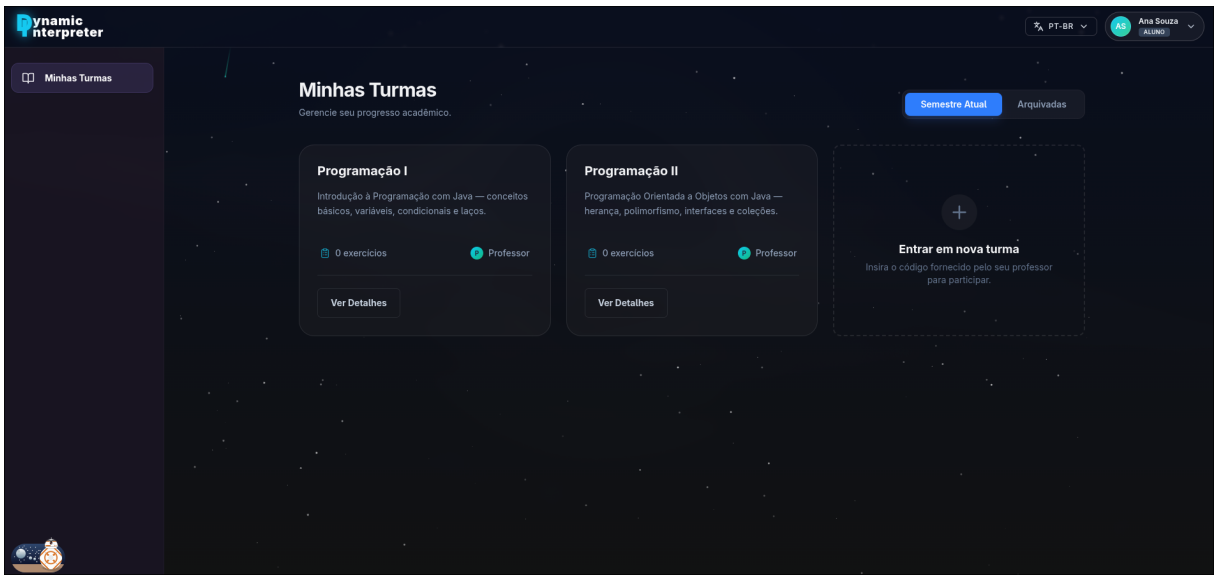


Figura 4.12 – Painel principal do aluno, com acesso às turmas, listas e exercícios.

Fonte: elaborado pelo autor.

Ao selecionar uma turma específica, o aluno é conduzido à tela de detalhamento, na qual visualiza informações da disciplina, listas publicadas pelo professor e seu progresso individual em cada exercício. A Figura 4.13 apresenta essa visão, enquanto a Figura 4.14 detalha a estrutura interna de uma lista de exercícios, na qual cada item exibe o respectivo estado (não iniciado, em andamento ou submetido) e a configuração da linguagem dinâmica vinculada ao enunciado.

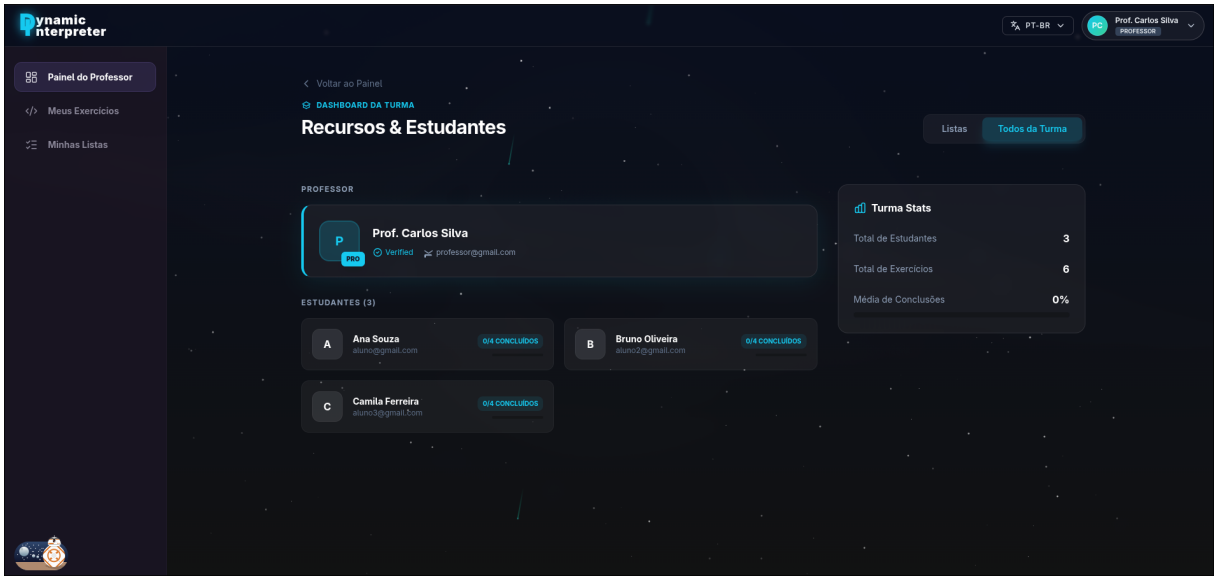


Figura 4.13 – Detalhamento de uma turma do aluno.

Fonte: elaborado pelo autor.

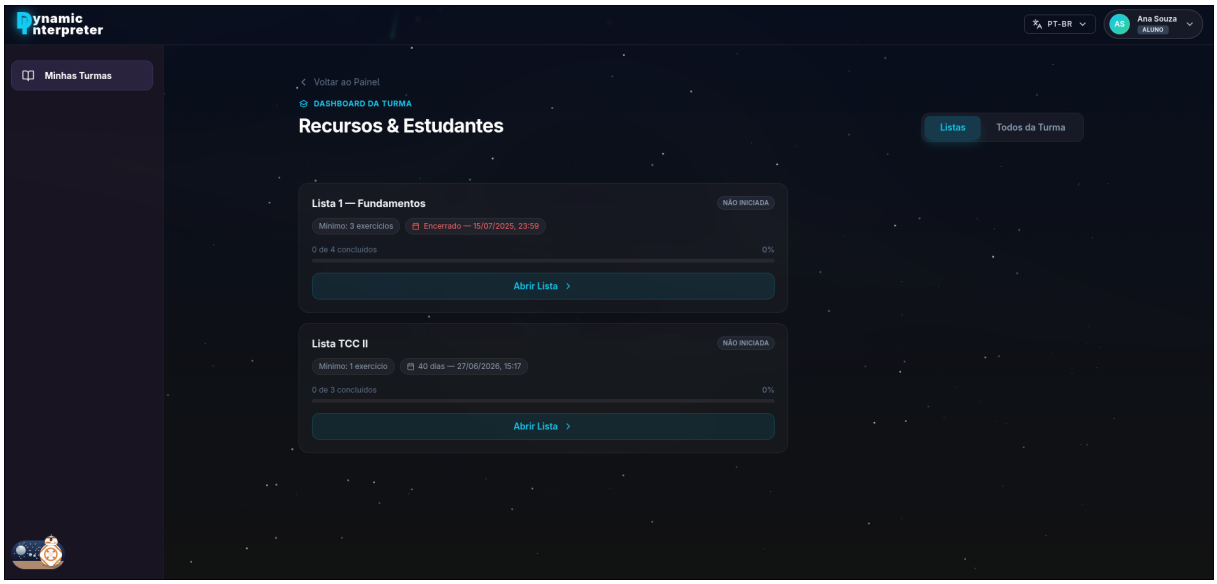


Figura 4.14 – Lista de exercícios sob a ótica do aluno.

Fonte: elaborado pelo autor.

Gestão de turmas e de seus participantes (RF15)

O painel do professor segue a mesma metáfora de organização do painel do aluno, mas amplia o conjunto de ações disponíveis, incorporando criação e gestão de turmas, listas de exercícios e enunciados. A Figura 4.15 apresenta a visão consolidada do professor, com acesso direto às turmas sob sua responsabilidade. Ao selecionar uma turma, são exibidas as listas associadas, conforme a Figura 4.16, e o conjunto de listas pode ser inspecionado em maior detalhe pela visão dedicada da Figura 4.17.

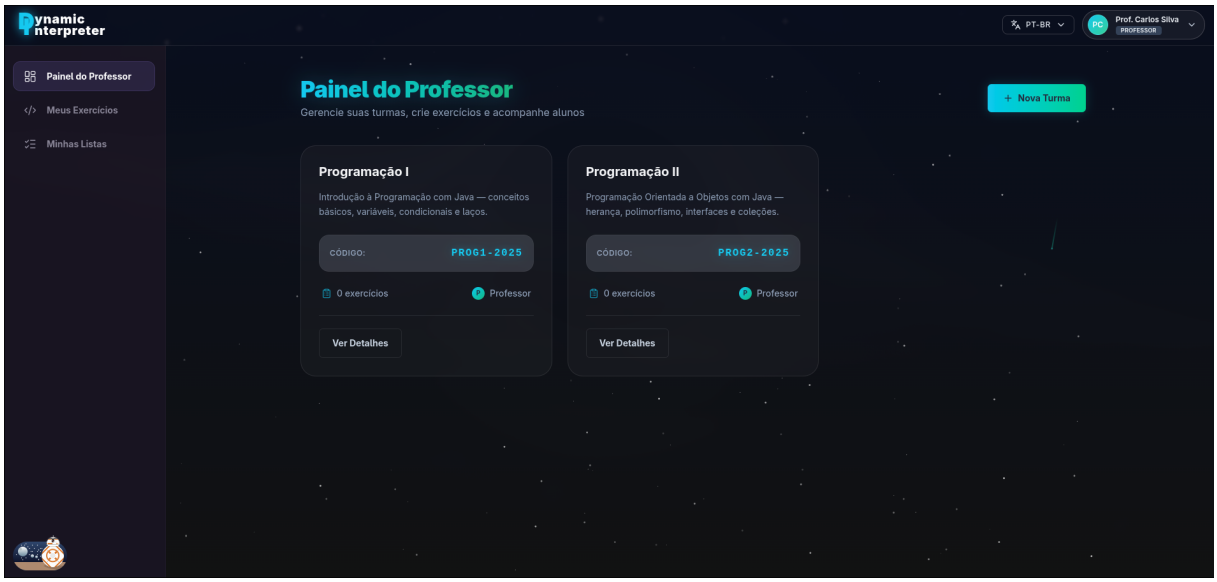


Figura 4.15 – Painel principal do professor.

Fonte: elaborado pelo autor.

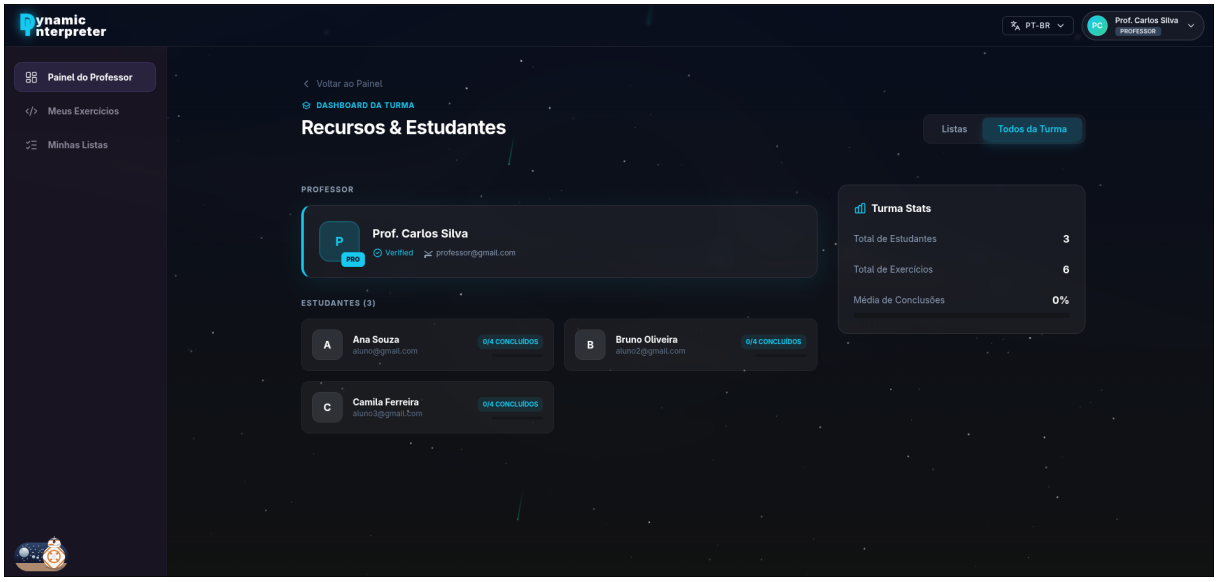


Figura 4.16 – Detalhamento de uma turma do professor.

Fonte: elaborado pelo autor.

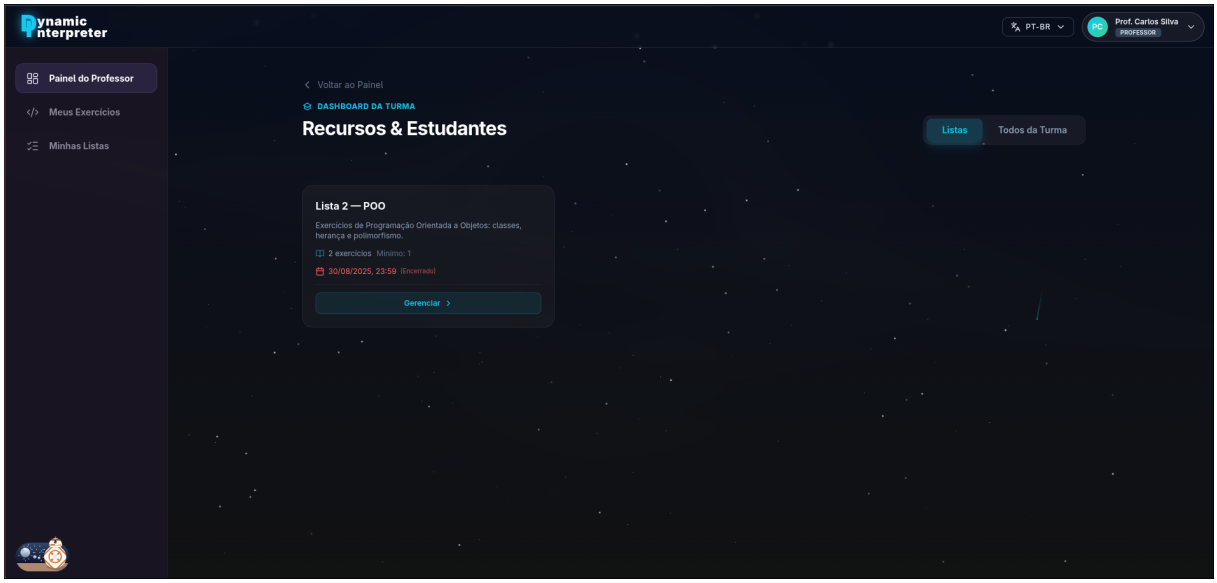
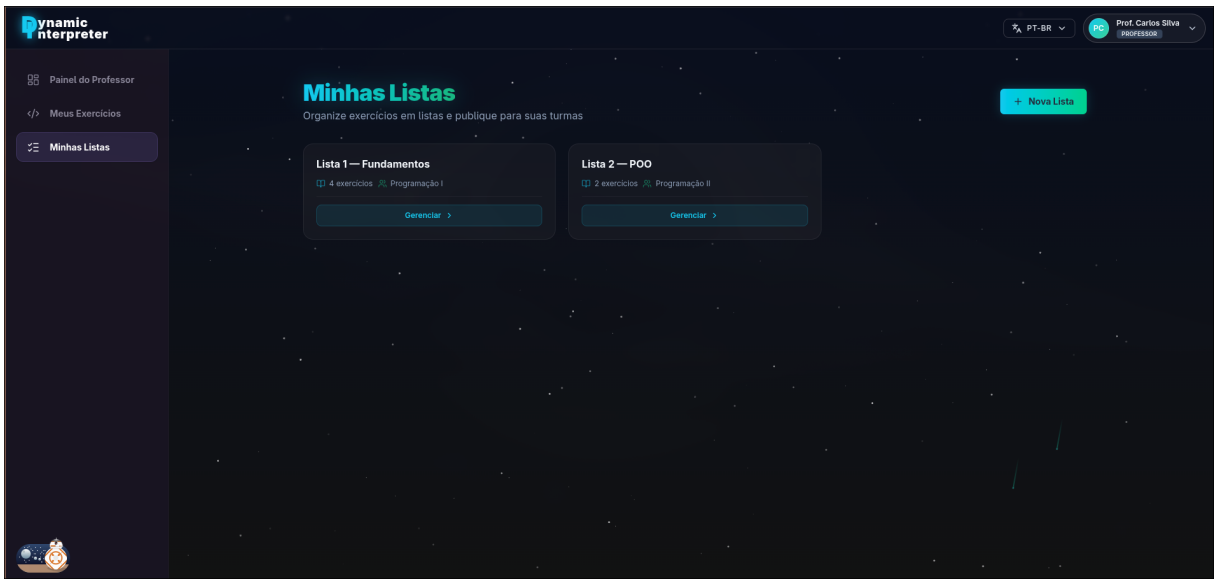


Figura 4.17 – Listas de exercícios vinculadas à turma.

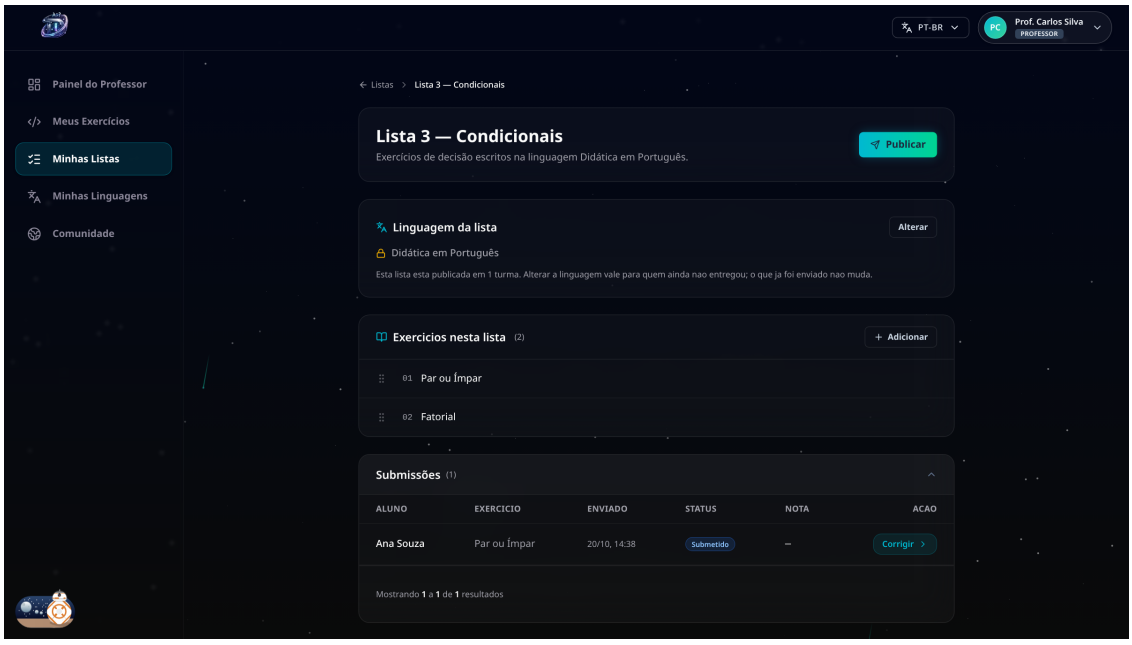
Fonte: elaborado pelo autor.

Gestão de enunciados, listas e submissões (RF16 a RF20)

A criação e a manutenção de listas pelo professor ocorrem em telas dedicadas que isolam cada operação em um fluxo independente. As Figuras 4.18 e 4.19 documentam as quatro etapas centrais desse processo: a primeira reúne a visualização das listas existentes e o detalhe de uma lista, com a linguagem obrigatória, os exercícios vinculados e as submissões recebidas; a segunda, a vinculação de exercícios à lista por meio do utilitário de busca e seleção e a publicação da lista para a turma escolhida.



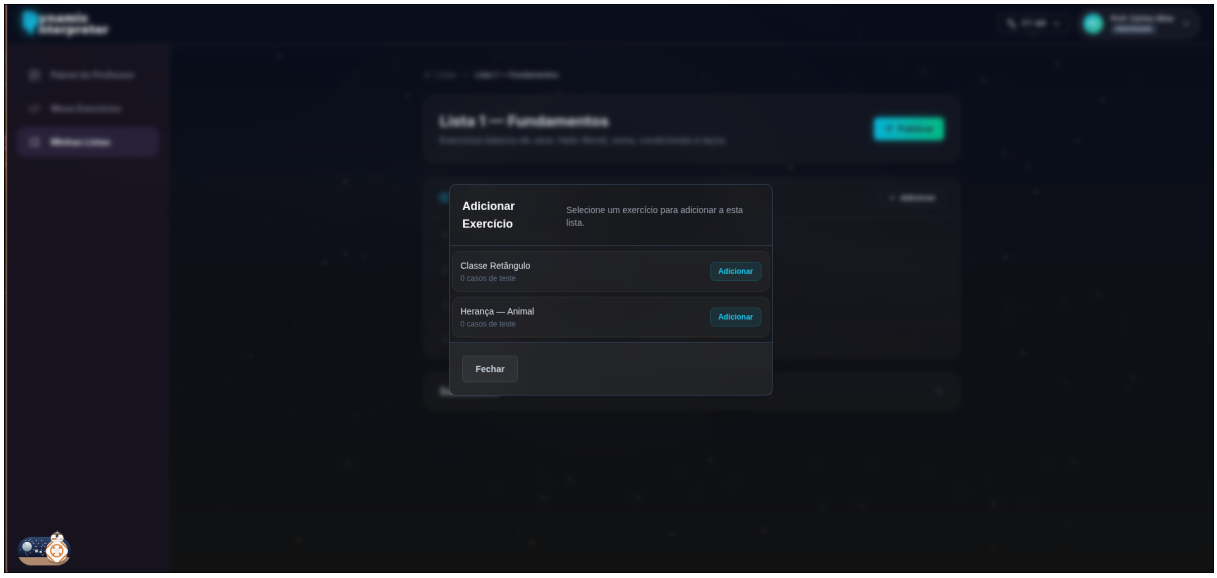
(a) Listagem das listas existentes.



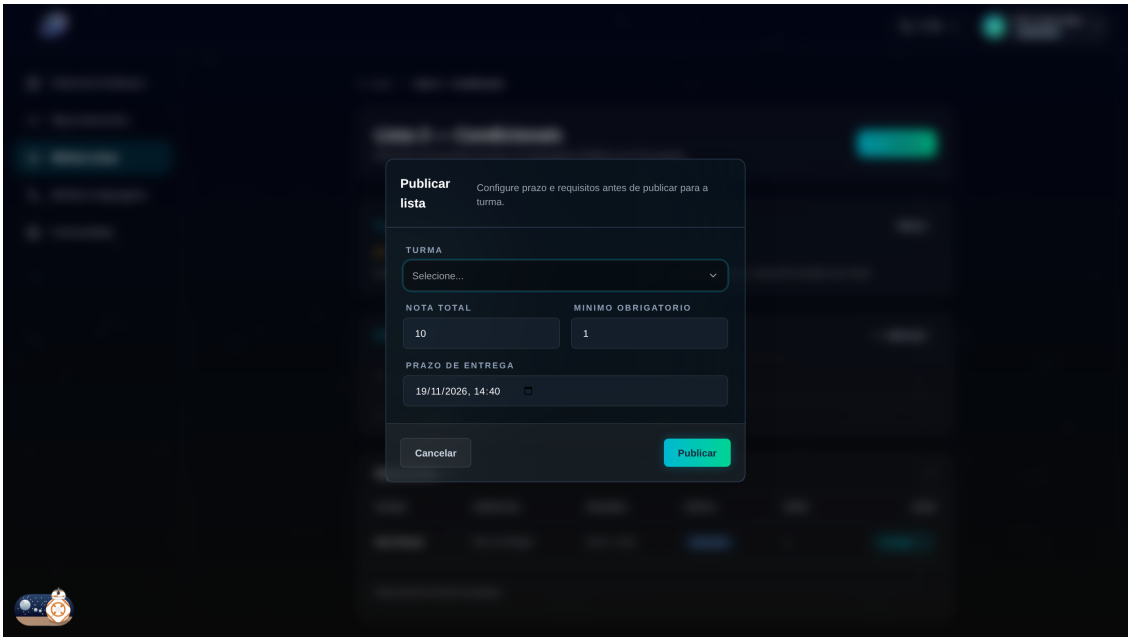
(b) Detalhe da lista, com a linguagem obrigatória, os exercícios e as submissões.

Figura 4.18 – Preparo de uma lista de exercícios pelo professor: listagem das listas existentes e detalhe de uma lista.

Fonte: elaborado pelo autor.



(a) Vinculação de exercícios à lista.

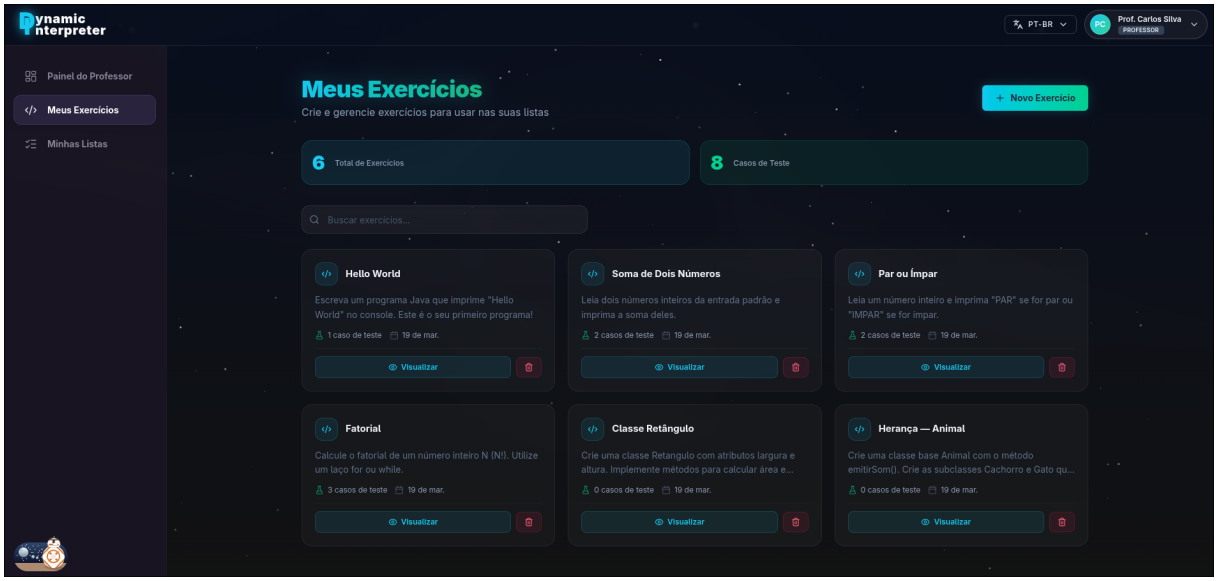


(b) Publicação da lista para a turma.

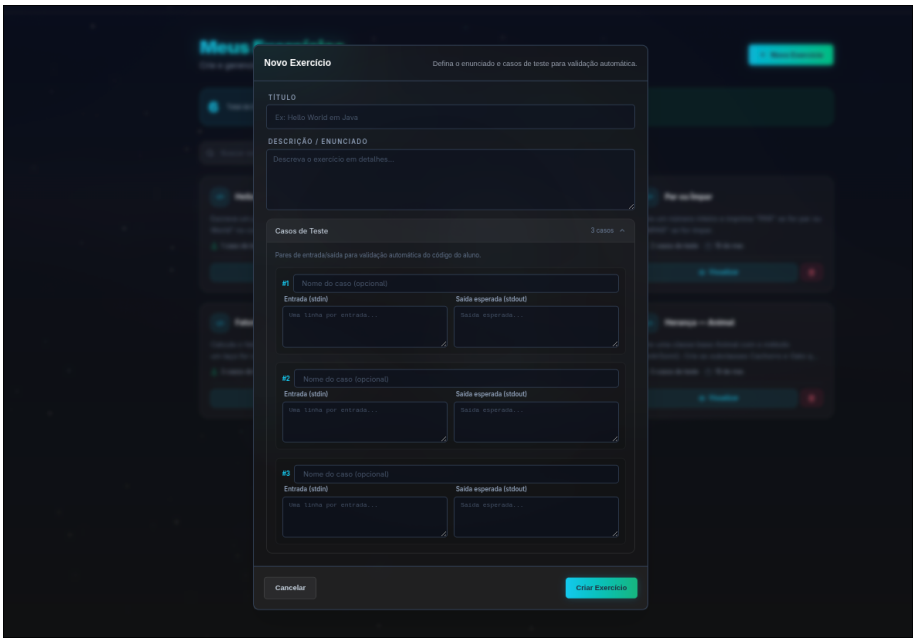
Figura 4.19 – Conclusão da lista de exercícios: vinculação dos enunciados e publicação para a turma.

Fonte: elaborado pelo autor.

A gestão dos enunciados individuais ocorre em um conjunto análogo de telas, ilustrado pelas Figuras 4.20 e 4.21. A tela de listagem permite a busca, a filtragem e a ordenação dos enunciados; a tela de criação centraliza, em um único formulário, a descrição do problema, os casos de teste com entradas e saídas esperadas e a vinculação a uma configuração de linguagem que delimita o vocabulário disponível ao aluno; a tela de detalhamento mostra a composição final do exercício; e a tela de submissões consolida todas as resoluções enviadas pelos alunos, com os respectivos vereditos do subsistema de avaliação automatizada discutido na Seção 4.7.



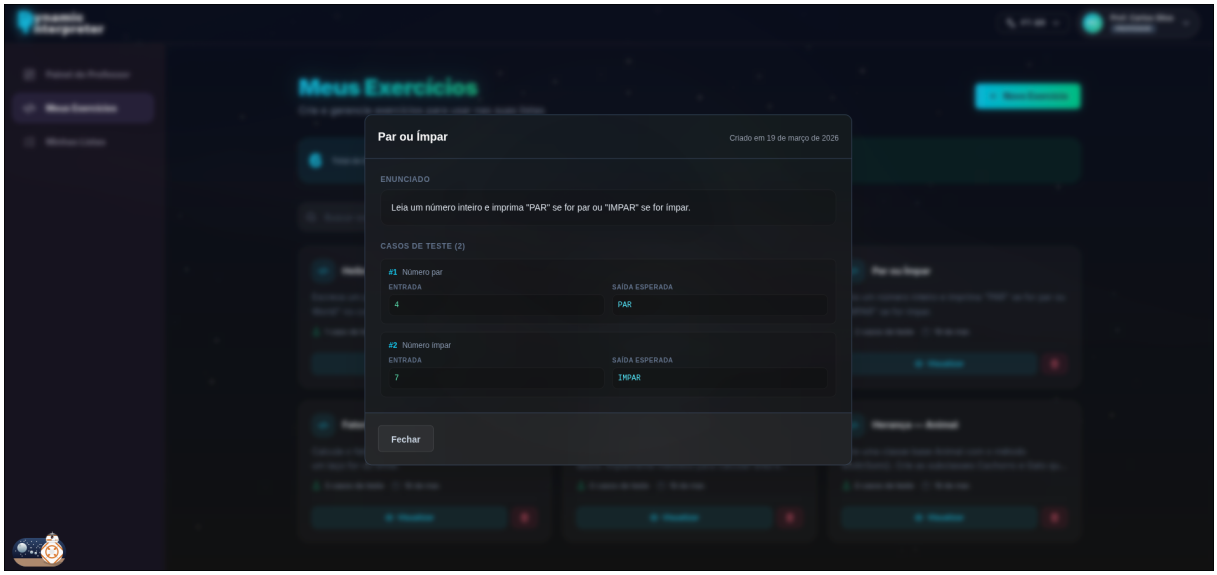
(a) Listagem dos enunciados existentes.



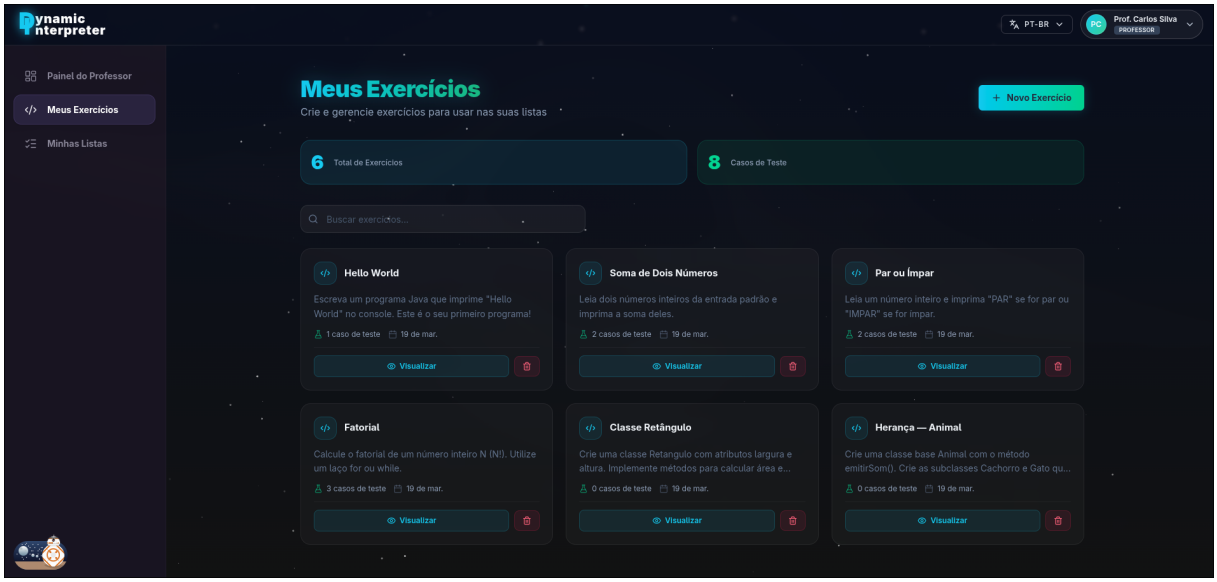
(b) Criação de um novo enunciado.

Figura 4.20 – Criação de enunciados pelo professor: listagem dos exercícios existentes e formulário de cadastro.

Fonte: elaborado pelo autor.



(a) Detalhamento de um exercício.



(b) Submissões dos alunos para o exercício.

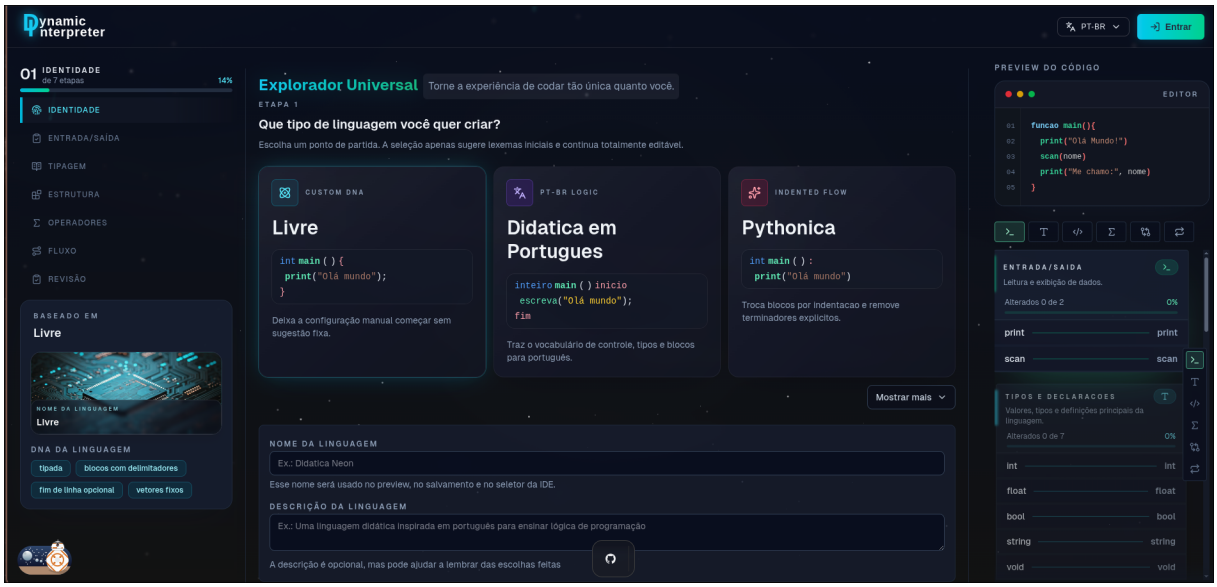
Figura 4.21 – Acompanhamento dos enunciados pelo professor: detalhamento do exercício e submissões recebidas.

Fonte: elaborado pelo autor.

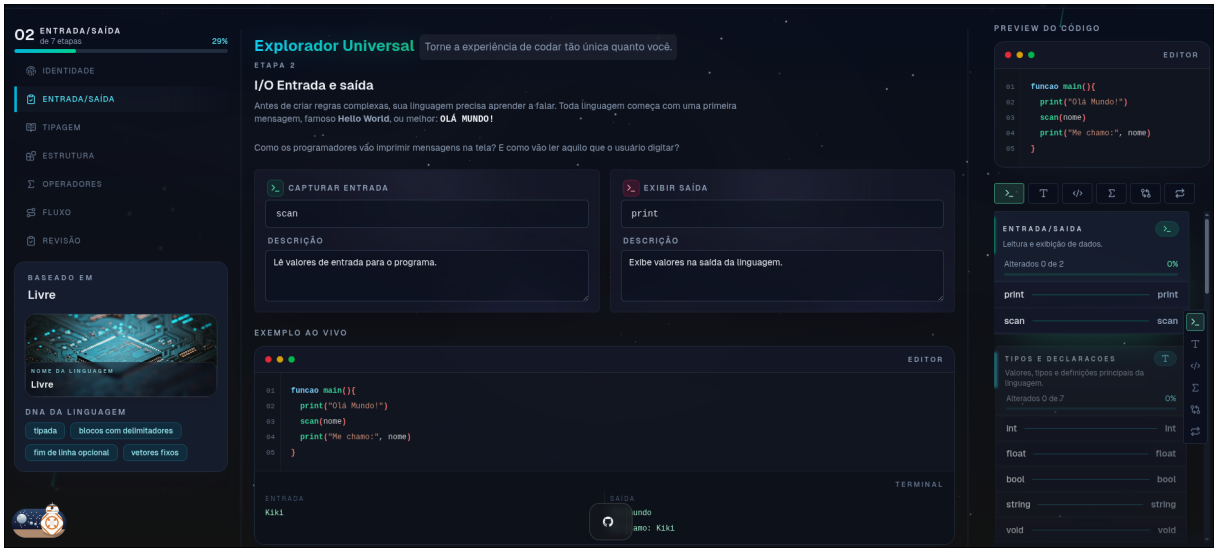
Personalização da linguagem pelo estudante (RF03 a RF06)

A funcionalidade central da plataforma, que corresponde à personalização dinâmica do vocabulário e das regras estruturais da linguagem, foi materializada em um assistente progressivo cuja versão atual contém sete etapas: identidade, entrada/saída, tipagem, estrutura, operadores, fluxo e revisão. Esse *wizard*, discutido na Seção 4.3, conduz o usuário por escolhas independentes, validadas isoladamente, com geração de *preview* em tempo real do código resultante. As capturas a seguir registram a organização anterior em seis etapas, preservada como histórico da implementação; não representam a sequência atual. A Figura 4.6 mostra o assistente

na versão verificada nesta revisão.



(a) Etapa 1: definição do nome da linguagem e da identidade da customização.

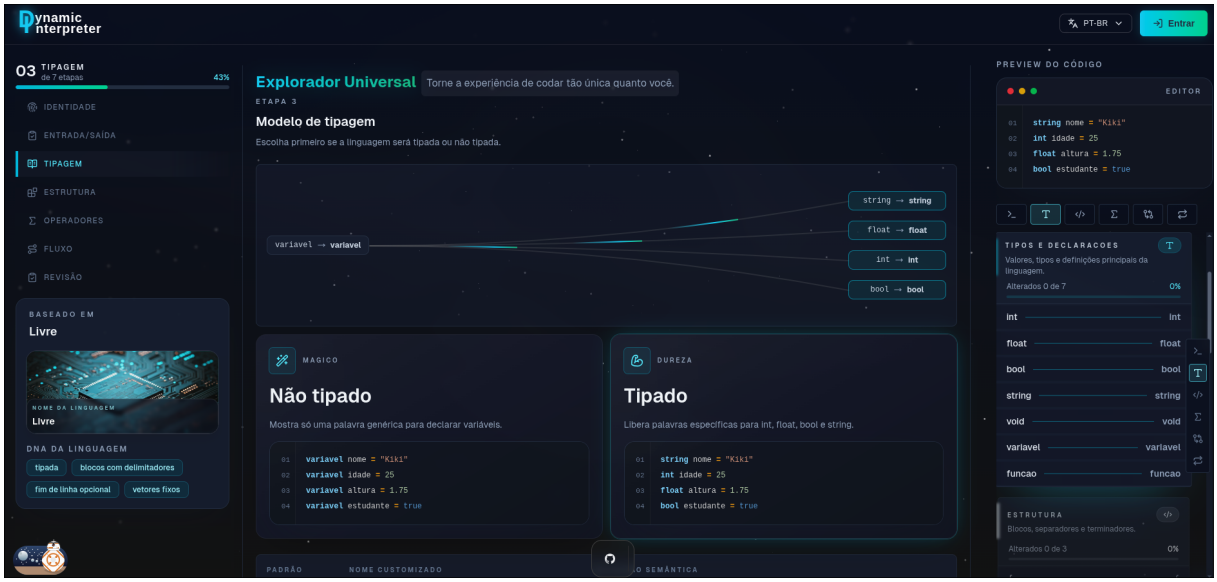


(b) Etapa 2: configuração das palavras-chave de tipos primitivos.

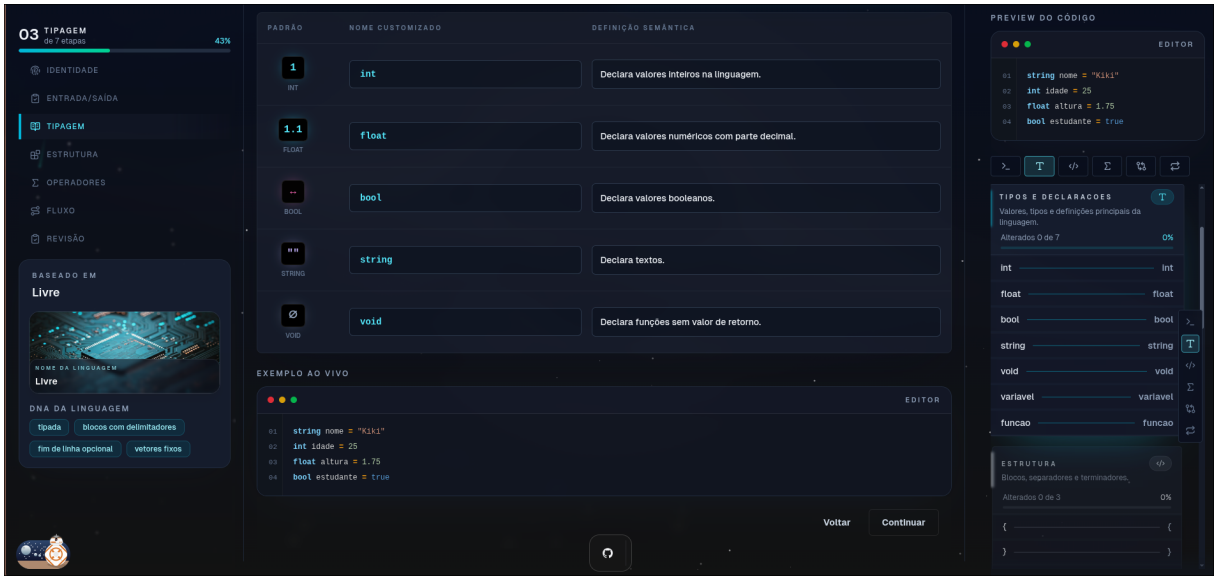
Figura 4.22 – Etapas 1 e 2 do assistente de personalização da linguagem dinâmica.

Fonte: elaborado pelo autor.

A terceira etapa concentra-se na personalização das palavras-chave de controle de fluxo, oferecendo ao usuário a possibilidade de redefinir comandos como `if`, `else`, `for`, `while`, `break`, `continue` e `return`. A Figura 4.23 apresenta as duas variações dessa etapa, contemplando a configuração das palavras-chave básicas e a configuração estendida com construtores como `switch` e `case`.



(a) Etapa 3.1: palavras-chave básicas de controle de fluxo.

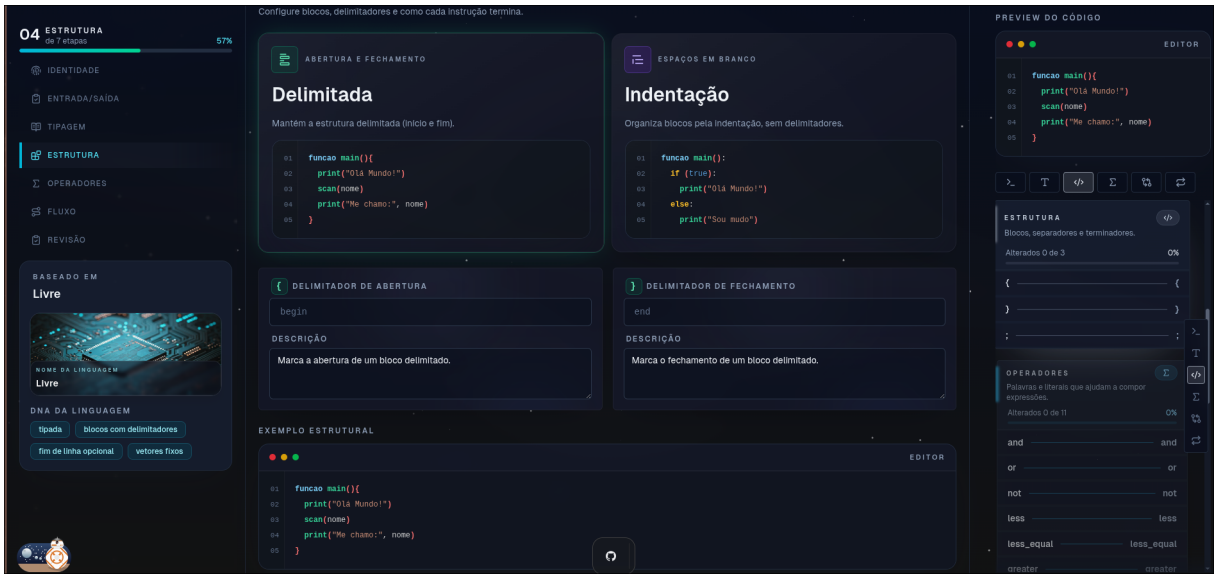


(b) Etapa 3.2: construtores estendidos de controle de fluxo.

Figura 4.23 – Etapa 3 do assistente, dedicada às palavras-chave de controle de fluxo.

Fonte: elaborado pelo autor.

A quarta etapa habilita a substituição de operadores lógicos e relacionais por palavras de uso natural, como uma alternativa de escrita a ser investigada, motivada pela discussão de escolhas sintáticas de Stefik e Siebert (2013). A Figura 4.24 ilustra as duas variações dessa etapa, contemplando, primeiro, a definição de operadores lógicos como `e`, `ou` e `não` e, em seguida, a redefinição dos operadores relacionais por meio do mesmo princípio de palavras significativas.



(a) Etapa 4.1: operadores lógicos em forma de palavra.



(b) Etapa 4.2: operadores relacionais em forma de palavra.

Figura 4.24 – Etapa 4 do assistente, dedicada à substituição de operadores por palavras.

Fonte: elaborado pelo autor.

A quinta etapa, apresentada na Figura 4.25, é responsável pela configuração dos literais booleanos, oferecendo ao usuário a oportunidade de adotar termos como `verdadeiro` e `falso` em substituição aos canônicos `true` e `false`. A finalização do assistente ocorre na etapa seis, na qual o usuário define os delimitadores de bloco (chaves, palavras-chave ou indentação) e o terminador de instrução, conforme ilustrado pela Figura 4.26. A Figura 4.27 apresenta, por fim, a pré-visualização consolidada da linguagem personalizada, exibida ao término do assistente.

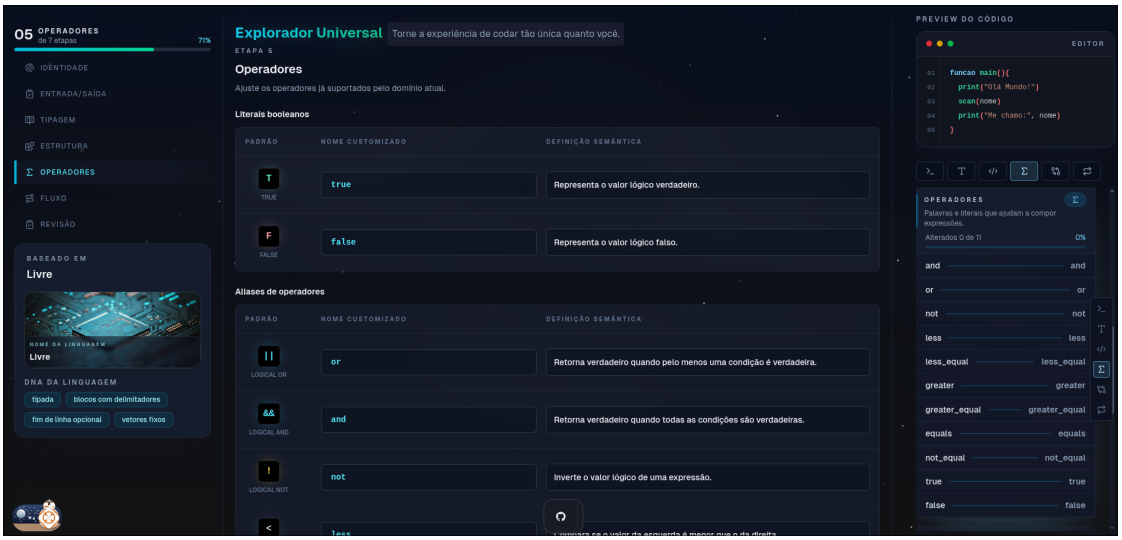
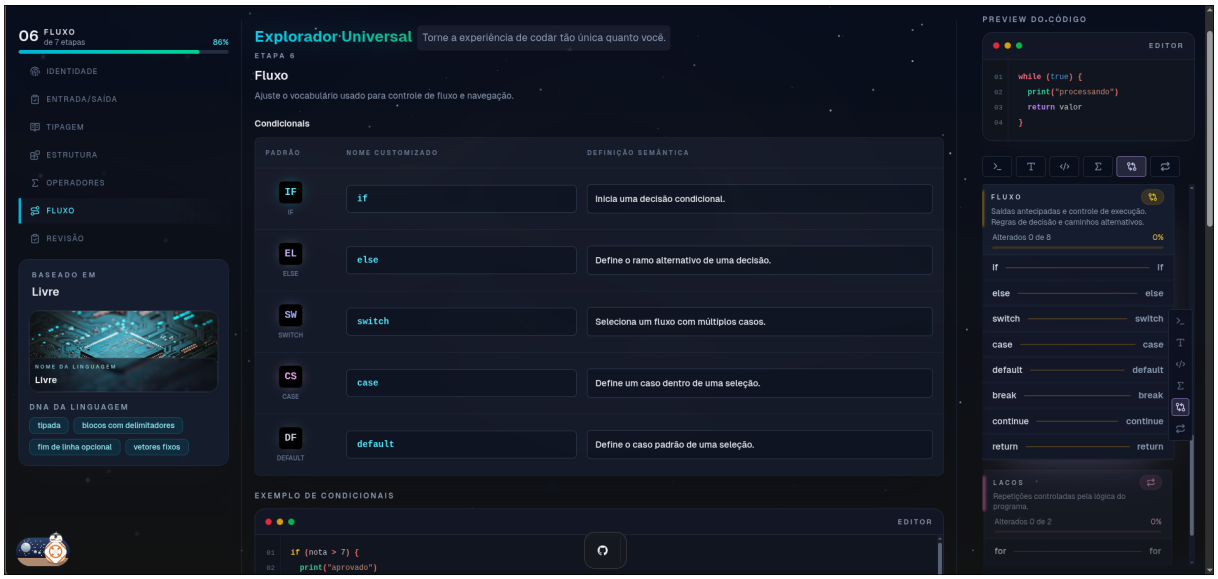
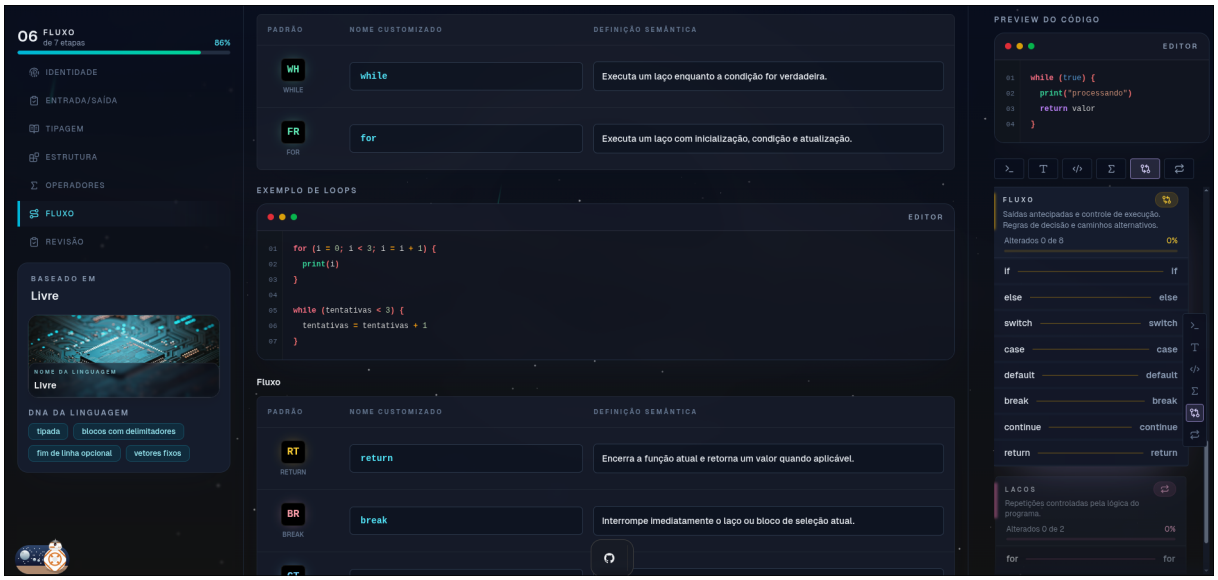


Figura 4.25 – Etapa 5: personalização dos literais booleanos.

Fonte: elaborado pelo autor.



(a) Delimitadores de bloco.



(b) Terminador de instrução.

Figura 4.26 – Etapa 6 do assistente: definição dos delimitadores de bloco e do terminador de instrução.

Fonte: elaborado pelo autor.

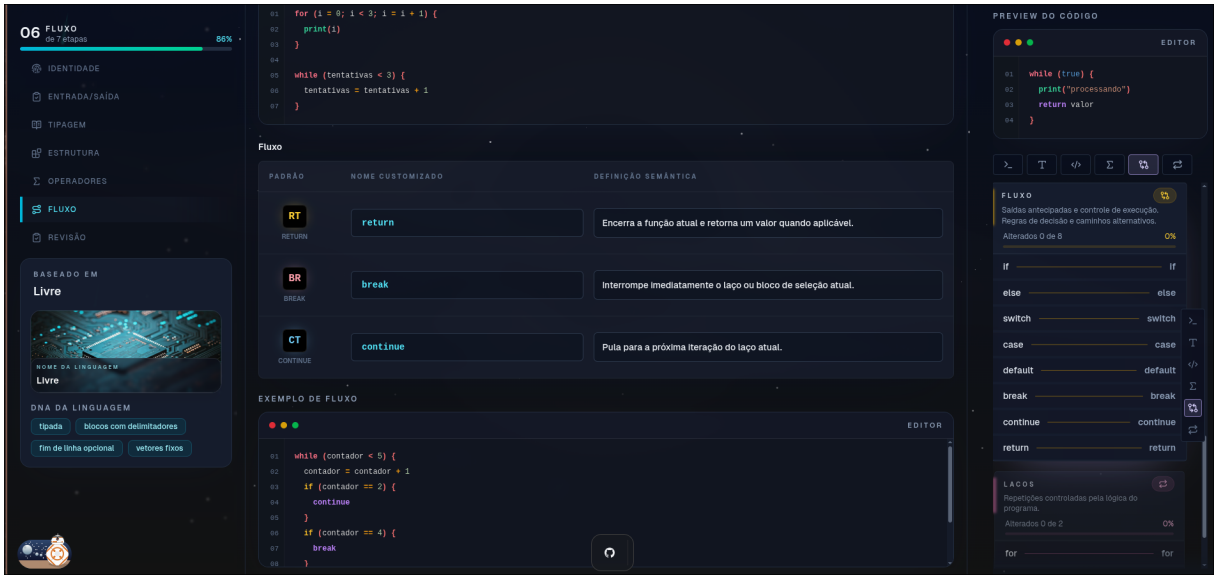


Figura 4.27 – Pré-visualização final da linguagem personalizada, exibida ao término do assistente.

Fonte: elaborado pelo autor.

O conjunto de telas apresentado consolida, sob o ponto de vista visual, as decisões arquiteturais e de modelagem discutidas ao longo deste capítulo. A organização das informações, a coerência entre os perfis de aluno e professor, a granularidade do assistente de personalização e o cuidado com a paridade entre os temas claro e escuro evidenciam o esforço consciente em alinhar a experiência de uso da plataforma aos princípios de UX e de *design* inclusivo discutidos no referencial teórico.

4.11 Estratégia de Testes Automatizados

A validação contínua da plataforma foi sustentada por uma suíte de testes automatizados construída com o *framework* Vitest, complementada por *jsdom* para simulação do ambiente de navegador em testes que envolvem componentes React. A organização dos testes refletiu a topologia interna do projeto: testes unitários acompanham diretamente os módulos correspondentes em diretórios `__tests__`, enquanto testes de integração foram concentrados em arquivos com sufixos `.spec.ts` ou `.spec.tsx` próximos aos componentes que validam (QUEIROZ; SILVA, 2026b).

Os testes disponíveis verificam construções como configuração da linguagem, persistência, componentes do assistente, depuração e submissões. Eles oferecem evidência do comportamento coberto e apoio à detecção de regressões. A presença de uma suíte, por si só, não comprova que o ciclo teste antes da implementação tenha sido seguido em todos

os incrementos. A pirâmide apresentada na Seção 2.2.2 orienta a distinção entre verificações isoladas e integração; não se apresenta aqui uma medição da cobertura total do sistema (VOCKE, 2018; QUEIROZ; SILVA, 2026b).

A revisão anterior, de 16 de setembro, registrou 139 aprovações em 28 arquivos selecionados pelo comando de testes da IDE, além de seis verificações executadas separadamente. Após os merges, a configuração passou a descobrir arquivos `.spec.ts` e `.spec.tsx` em toda a árvore de fontes, mantendo oito exclusões explícitas de testes do assistente ainda pendentes de atualização. Em 19 de setembro de 2026, a execução local do comando `npm test` do pacote aprovou 274 testes em 54 arquivos (QUEIROZ, 2026b). Em 19 de outubro, o mesmo comando aprovou 312 testes em 60 arquivos, sem falhas, com as mesmas oito exclusões (QUEIROZ, 2026a). Entre os arquivos novos estão os testes do menu da IDE, do painel de linguagem, dos arquivos iniciais e do cartão de turma do professor. A ampliação documenta o conjunto executado, sem representar a execução dos arquivos excluídos ou uma medição de cobertura total (QUEIROZ; SILVA, 2026b). O pacote E2E não foi executado nesta revisão de outubro, porque depende dos serviços em execução; o resultado mais recente dele é o de 19 de setembro, descrito na Subseção 4.11.3.

A preparação do ambiente de testes passou a incluir uma implementação de armazenamento em memória quando o armazenamento global não fornece as operações esperadas, além de substitutos para APIs de rolagem e consulta de mídia ausentes no ambiente simulado. A nova execução não exigiu o ajuste manual de armazenamento registrado na revisão anterior. Esses substitutos viabilizam os testes dos componentes, mas não reproduzem o cálculo de layout de um navegador real (QUEIROZ; SILVA, 2026b).

Para tornar concreta essa estratégia, o Trecho de Código 4.8 apresenta um recorte de dois casos que verificam a persistência da customização, construção sobre a qual repousa a continuidade da linguagem escolhida pelo usuário entre recarregamentos da página. Os auxiliares e algumas asserções foram omitidos para abreviar a listagem; o arquivo original contém a verificação completa. O primeiro caso verifica a normalização do nome da linguagem em uma chave de armazenamento; o segundo, que a gravação permite recuperar a linguagem pelo identificador (QUEIROZ; SILVA, 2026b).

```
describe("keyword-language-storage", () => {
  beforeEach(() => { localStorage.clear(); });

  it("slugifies the language name for localStorage keys", () => {
    expect(slugifyLanguageName("  Minha Linguagem++  "))
      .toBe("minha-linguagem");
    expect(slugifyLanguageName("JAVA   MM")).toBe("java-mm");
  });

  it("persists the saved language in the registry, active key, "
   + "and runtime customization", () => {
    const savedLanguage = createSavedLanguage();

    saveSavedKeywordLanguage(savedLanguage);

    expect(loadSavedKeywordLanguage(savedLanguage.slug))
      .toEqual(savedLanguage);
  });
});
```

Listing 4.8 – Casos de teste da persistência da linguagem personalizada (`keyword-language-storage.spec.ts`).

4.11.1 Testes de ponta a ponta da interface

As integrações de 18 de setembro de 2026 acrescentaram o pacote `packages/e2e` e ampliaram os cenários automatizados dos fluxos educacionais e de gestão de linguagens. A revisão documental de 19 de setembro examinou esse código na versão `3ffae01`, que incorpora os merges `9b1e506` e `c9b8178`. A implementação utiliza Playwright Test, cuja versão fixada no arquivo de dependências é 1.63.0. O pacote reúne 31 casos em dez arquivos e configura somente o Chromium, com perfil de computador, idioma português do Brasil e fuso de São Paulo (QUEIROZ; SILVA, 2026c; Microsoft, 2026c).

O ambiente de integração utiliza o front-end Next.js, o serviço FastAPI e o PostgreSQL definidos na composição Docker local. As rotinas de preparação (*fixtures*) criam contas e dados pela API, com identificadores aleatórios, e podem autenticar o navegador mediante inserção do cookie de sessão. Os casos específicos de autenticação verificam o formulário de entrada e o cadastro pela interface; os demais podem iniciar com uma sessão preparada. Portanto, nem todo caso repete a jornada desde o login, e dois testes de preparação e um de saúde do serviço verificam diretamente a API (QUEIROZ; SILVA, 2026c; QUEIROZ; SILVA, 2026a).

A Tabela 4.3 descreve os casos existentes, agrupados pela finalidade. Suas quantidades representam cenários implementados, sem constituir uma porcentagem de cobertura dos

requisitos, do código ou das combinações possíveis de interação (QUEIROZ; SILVA, 2026c).

Tabela 4.3 – Escopo dos casos automatizados do pacote E2E.

Grupo	**Casos**	**Comportamentos verificados**
Autenticação	4	Login válido, senha incorreta, cadastro e proteção de rota.
IDE e compilador	4	Execução e saída, tokens, código intermediário e erro de sintaxe.
Catálogo comunitário	3	Listagem das cinco linguagens do seed, importação e ativação no analisador léxico.
Acervo de linguagens	4	Duplicação, exclusão, publicação e despublicação, além da restrição do botão por perfil.
Assistente de linguagem	3	Criação, edição sem duplicação e rejeição de nome já utilizado.
Jornada do aluno	3	Entrada em turma e submissões de programas corretos e incorretos.
Jornada do professor	3	Criação de turma, exercício com caso de teste e publicação de lista.
Correção	2	Persistência de nota e comentário e apresentação da nota ao aluno.
Preparação e sessão	3	Contas independentes, montagem de atividade pela API e autenticação por cookie.
Disponibilidade básica	2	Saúde do serviço e carregamento da estrutura inicial da IDE.
Total	**31**	Casos do pacote, incluindo verificações de infraestrutura.

Fonte: elaboração própria a partir dos arquivos de teste de Queiroz e Silva (2026c).

Os testes utilizam localizadores por papel e nome, além de identificadores explícitos para editor, terminal e resultados de submissão. O auxiliar do Monaco aguarda sua montagem por até 45 segundos e insere o programa diretamente no modelo do editor; a execução e a inspeção dos resultados prosseguem pela interface. Esse procedimento delimita a evidência: verifica a integração com o programa preparado, mas não simula sua digitação caractere a caractere. Nos testes de correção, recarregar a página e conferir novamente nota e comentário verifica a recuperação dos dados persistidos, além da mensagem imediata de sucesso (QUEIROZ; SILVA, 2026c; Microsoft, 2026a).

4.11.2 Integração contínua e diagnóstico de falhas

O fluxo versionado no GitHub Actions executa testes em solicitações de integração e em envios à branch `main`. O job `unit` executa Vitest no compilador e na IDE, além de pytest no back-end. O job `e2e` instala o Chromium, constrói e inicia os serviços locais, aguarda suas respostas e executa o Playwright. O job de implantação depende da aprovação de ambos e

só é acionado por envio à `main`. Essa configuração estabelece uma condição para o deploy automatizado; o arquivo não demonstra que as regras de proteção da branch obriguem a aprovação antes de qualquer merge (QUEIROZ; SILVA, 2026a).

No CI, a configuração utiliza dois processos de execução, permite duas novas tentativas após uma falha e ativa `failOnFlakyTests`. O Playwright classifica como intermitente (*flaky*) o caso que falha inicialmente e passa em uma nova tentativa; a opção adotada faz essa condição reprovar a execução. As repetições, portanto, auxiliam o diagnóstico sem converter a instabilidade em aprovação. São configurados rastros de execução na primeira repetição, capturas de tela em falhas e retenção de vídeos de falhas; o workflow disponibiliza os relatórios e resultados por sete dias (QUEIROZ; SILVA, 2026c; QUEIROZ; SILVA, 2026a; Microsoft, 2026d; Microsoft, 2026e).

As mesmas integrações incluem correções relacionadas aos fluxos exercitados: a resolução de endereço interno do serviço no código de servidor do Next.js passou a ser compartilhada pela edição de linguagens e validação de submissões; o componente de formulário passou a transferir identificação e atributos de acessibilidade ao controle filho por meio de `Slot`. Esses registros documentam alterações concretas no front-end, sem equivaler a uma nova auditoria completa de acessibilidade (QUEIROZ; SILVA, 2026b).

4.11.3 Resultado da execução local

Em 19 de setembro de 2026, os 31 casos foram aprovados na primeira tentativa de cada caso, sem falhas, ocorrências intermitentes ou casos ignorados, em aproximadamente 24 segundos. A execução utilizou Node.js 25.2.1, Playwright 1.63.0 e as opções de CI da configuração versionada, com dois processos, contra os serviços Docker locais já ativos. O relatório estruturado registra os arquivos, cenários e resultados. Também foi aprovada a verificação de tipos do pacote E2E. Esses resultados são locais e não constituem evidência de uma execução específica do GitHub Actions nem de uma reconstrução das imagens dos serviços nesta revisão (QUEIROZ, 2026b).

Antes dessa execução completa, uma tentativa foi interrompida pela ausência do executável Chromium esperado pelo Playwright: os 28 casos que precisavam do navegador falharam na inicialização e os três casos diretos de API passaram. Após instalar o navegador, a suíte foi repetida integralmente e produziu os resultados acima. O registro distingue, assim,

uma falha de preparação do ambiente dos comportamentos funcionais exercitados na execução concluída (QUEIROZ, 2026b).

4.12 Proposta de Protocolo de Avaliação

Esta seção descreve o instrumento de avaliação da plataforma, chamado de PA-01, e explica o que ele permite concluir. O instrumento é um só e é compartilhado com o texto do coautor: o mesmo formulário vai para o mesmo grupo de respondentes, e cada trabalho analisa os itens que interessam ao seu tema. Aqui interessam os itens sobre a interface, o assistente de personalização e os fluxos do ambiente virtual de aprendizagem. O protocolo ainda não foi aplicado, e nenhum resultado de participantes aparece neste documento.

4.12.1 Quem responde e o que é possível concluir

A avaliação será feita por um painel de especialistas, formado por professores e profissionais de computação. Não é um estudo com os alunos, que são o público-alvo da plataforma. Pede-se ao respondente que avalie se a ferramenta é adequada para um aluno em primeiro contato com programação, e não que relate a própria experiência de aprendizagem.

Essa escolha limita o que o trabalho pode afirmar, e por isso ela é explicada antes da descrição do instrumento. Com um painel de especialistas é possível dizer que profissionais da área acham plausível que a personalização da sintaxe reduza a barreira sintática. Não é possível dizer que ela reduz. A pergunta feita na introdução continua sem resposta empírica, e respondê-la exige medir o desempenho de estudantes, como descrito na Subseção 4.12.6.

Em troca, o painel dá acesso a avaliações que um aluno iniciante não teria como fazer: se o subconjunto da linguagem é adequado a uma disciplina introdutória, como a plataforma se compara às ferramentas discutidas no Capítulo 2 e quais riscos pedagógicos a proposta traz. Há ainda um motivo ligado ao tema deste trabalho. Personalizar a linguagem é uma tarefa de autoria: alguém precisa decidir quais palavras substituem quais e verificar se o vocabulário final faz sentido. Na escola, quem costuma fazer isso é o professor, que prepara o ambiente antes da aula. Perguntar a professores se o assistente é claro e se o trabalho de preparação compensa é, portanto, perguntar a quem de fato usa essa parte da plataforma.

4.12.2 Composição do instrumento

O formulário é respondido pelo próprio participante, distribuído pela internet e criado por um programa, para que sua estrutura possa ser gerada de novo sempre que necessário. São dez páginas, oito delas com grades de itens em escala, como mostra a Tabela 4.4. Três blocos vêm de instrumentos já consolidados na literatura de avaliação de interfaces e os outros foram escritos para este projeto.

Tabela 4.4 – Blocos do instrumento PA-01, procedência e número de itens.

Bloco	**Procedência**	**Itens**
Consentimento livre e esclarecido	Elaborado para o projeto	1
Perfil profissional	Elaborado para o projeto	4
Uso em sala (condicional)	Elaborado para o projeto	8
Como a plataforma foi usada	Elaborado para o projeto	4
Avaliação dos recursos	Elaborado para o projeto	10
Facilidade de uso	SUS (BROOKE, 1996)	10
Impressão geral	UEQ-S (SCHREPP; HINDERKS; THOMASCHEWSKI, 2017)	8
Potencial pedagógico	Elaborado para o projeto	8
Riscos e limitações	Elaborado para o projeto	4
Qualidade técnica	Elaborado para o projeto	7
Utilidade e intenção de uso	TAM (DAVIS, 1989)	6
Comentários abertos	Elaborado para o projeto	5

Fonte: elaborado pelo autor.

Os instrumentos já conhecidos foram usados porque têm valores de referência publicados. Com eles é possível comparar o resultado obtido com o de outros produtos já avaliados, em vez de apenas descrever a opinião dos respondentes. Os blocos próprios foram escritos porque nenhum instrumento pronto pergunta sobre o que esta plataforma propõe. Antes do bloco de avaliação dos recursos, o formulário mostra o mesmo programa em duas versões: uma na configuração padrão, com `int`, `for`, `if` e delimitadores simbólicos, e outra personalizada, com `numero_inteiro`, `para`, `se`, operadores escritos por extenso e os delimitadores `inicio` e `fim`. Assim o respondente avalia algo concreto, e não uma ideia abstrata de personalização.

4.12.3 Itens sobre a interface e a personalização

O bloco de avaliação dos recursos tem a opção “não usei” em cada linha, o que separa quem discorda de quem não chegou a experimentar o recurso. As afirmações são:

- criar ou ajustar uma linguagem no assistente foi simples;
- ficou claro em que etapa o usuário estava;
- o exemplo de código ajudou a entender o efeito de cada alteração;
- escrever programas com a linguagem configurada foi confortável;
- as mensagens de erro ajudaram a identificar o que corrigir;
- resolver e enviar exercícios foi simples;
- importar uma linguagem do acervo da comunidade foi simples.

O bloco termina com três itens sobre a utilidade da tabela de *tokens*, do código intermediário e da execução passo a passo.

Os três primeiros itens correspondem a decisões de interface descritas na Seção 4.3: dividir a personalização em etapas, indicar em qual delas o usuário está e mostrar uma pré-visualização do código. Esses itens verificam escolhas que foram adotadas como hipótese durante o desenvolvimento.

Quem declara que leciona, ou já lecionou, disciplina que envolva programação responde a um bloco extra sobre o ambiente virtual de aprendizagem. Ele pergunta:

- se foi simples criar uma turma e receber alunos pelo código de acesso;
- se foi simples criar um exercício com casos de teste;
- se travar uma linguagem em um exercício ajuda a planejar as aulas;
- se a correção automática ajuda a acompanhar a turma;
- se o respondente conseguiria preparar uma aula sem apoio técnico;
- se o trabalho de preparar a linguagem compensa o ganho em sala.

Uma questão de múltipla escolha pergunta em que momento de uma disciplina introdutória a plataforma seria mais útil, e uma das alternativas diz que não há espaço para ela na disciplina. Essa alternativa foi mantida de propósito, pelo mesmo motivo que justifica o bloco de riscos.

O bloco de potencial pedagógico, respondido em terceira pessoa, transforma em afirmações as premissas que sustentam a plataforma:

- deixar o aluno definir as palavras-chave tende a reduzir a barreira sintática no primeiro contato;
- usar palavras em português no lugar de termos em inglês facilita entender o código;
- a personalização tende a reduzir os erros de sintaxe de iniciantes;
- mostrar os erros no vocabulário escolhido pelo próprio aluno facilita a correção;
- o conjunto de elementos configuráveis é adequado ao propósito da ferramenta.

4.12.4 Escalas usadas e cálculo dos escores

A facilidade de uso usa a escala de Brooke (1996), com dez afirmações que alternam entre positivas e negativas. Essa alternância faz parte do instrumento e não deve ser corrigida. Para calcular o escore, subtrai-se 1 do valor marcado nos itens ímpares, subtrai-se de 5 o valor marcado nos itens pares, somam-se os resultados e multiplica-se o total por 2,5. O resultado vai de 0 a 100 e não é uma porcentagem. A média de referência fica perto de 68 pontos, e a interpretação segue Bangor, Kortum e Miller (2009) e Sauro e Lewis (2012). A impressão geral usa a versão curta do questionário de Schrepp, Hinderks e Thomaschewski (2017), com oito pares de adjetivos opostos em sete pontos; subtrai-se 4 do valor bruto e separam-se a dimensão pragmática e a hedônica. A utilidade e a intenção de uso vêm do modelo de Davis (1989), adaptado ao público de professores.

O bloco de riscos funciona ao contrário dos outros: nele, concordar significa apontar um problema. As afirmações são:

- aprender em uma sintaxe personalizada pode dificultar a migração para uma linguagem de mercado;
- alunos da mesma turma usando sintaxes diferentes dificultaria o trabalho do professor;
- a liberdade de personalizar pode desviar a atenção do raciocínio algorítmico;
- o tempo gasto configurando a linguagem não compensa o ganho de aprendizagem.

Os escores desse bloco serão relatados separadamente e não podem ser somados aos do bloco de potencial pedagógico, porque as duas medidas apontam em sentidos opostos e uma anularia

a outra. O bloco foi incluído de propósito: se o questionário trouxesse apenas afirmações favoráveis, a aprovação obtida diria muito pouco.

4.12.5 Participantes, procedimento e questões éticas

Os participantes são convidados diretamente e não são identificados. O formulário registra titulação, área de atuação, ferramentas de ensino que a pessoa já conhece e tempo de experiência com programação. Em uma avaliação feita por especialistas, esses dados fazem parte do resultado e serão apresentados junto com os escores. Um item pede que o respondente marque o que fez na plataforma e o que chegou a personalizar. Com isso é possível saber se uma avaliação negativa veio de quem usou o assistente até o fim ou de quem nem chegou a abri-lo.

Sobre o número de participantes: de cinco a oito pessoas já revelam a maior parte dos problemas de usabilidade e são suficientes para analisar as respostas abertas. Para apresentar o escore SUS com intervalo de confiança são necessários vinte respondentes ou mais. Com menos que isso, o resultado será apresentado como indicativo, sem afirmar que existe diferença entre subgrupos.

O instrumento foi escrito como pesquisa de opinião pública com participantes não identificados, situação prevista no parágrafo único do artigo 1º da Resolução CNS n.º 510 de 2016 (SAÚDE, 2016). O termo de consentimento segue a Resolução CNS n.º 466 de 2012 (SAÚDE, 2012). Esse enquadramento não vale só porque os pesquisadores o declararam: ele depende de uma manifestação escrita do comitê de ética da instituição, que ainda não foi obtida. Esta proposta não é uma autorização nem uma aprovação ética, e nenhuma resposta pode ser coletada antes desse encaminhamento.

4.12.6 Limites de interpretação e continuidade

Além do limite já citado sobre o público, três ressalvas afetam a leitura dos resultados. A primeira é sobre o instrumento: no formulário, os pares de adjetivos do UEQ-S estão sempre com o polo negativo à esquerda, enquanto o instrumento original inverte alguns pares para evitar que o respondente concorde por hábito. Se a comparação com os valores de referência for importante para o argumento, essa ordem deve ser conferida antes da coleta. A segunda é sobre a redação em português dos itens do SUS, que precisa ser comparada com uma tradução validada para que o escore possa ser comparado com a média histórica. A terceira é sobre a análise: escores altos

dados por especialistas não comprovam aprendizagem, redução de carga cognitiva nem aumento de engajamento.

Investigar a hipótese pedagógica é a etapa seguinte e exige outro desenho de estudo: comparar, na mesma IDE, o vocabulário padrão e o vocabulário personalizado, com tarefas equivalentes de entrada e saída, seleção e repetição. A ordem das condições deve ser alternada entre os participantes para reduzir o efeito de prática, e os enunciados, os casos de teste, o tempo disponível e os critérios de classificação dos erros precisam ser definidos antes da coleta, separando erros de escrita de falhas de lógica e de dificuldades de navegação. Com isso seria possível descrever os acertos, o tempo até a primeira execução correta e a frequência de erros em cada condição. Ainda assim, uma diferença nessas medidas não comprova, sozinha, que houve aprendizagem duradoura.

4.12.7 Continuidade da verificação técnica

Antes da avaliação com participantes, é preciso corrigir as barreiras registradas na Seção 4.8 e ampliar os testes de teclado para os fluxos de salvar linguagem, editar código, usar o terminal e enviar exercícios. A verificação deve cobrir foco visível, entrada e saída de diálogos, nomes dos controles, anúncio de erros e compatibilidade com leitor de tela, registrando qual sistema operacional, navegador e tecnologia assistiva foram usados. Também devem entrar os temas claro e escuro e as larguras de 320, 390, 768 e 1440 pixels CSS. Os critérios WCAG apresentados no referencial orientam essa inspeção, mas declarar conformidade exigiria examinar as páginas e os processos completos (World Wide Web Consortium, 2024; World Wide Web Consortium, 2026).

5 RESULTADOS DA AVALIAÇÃO

Este capítulo é reservado à análise dos dados do instrumento descrito na Seção 4.12. Até o fechamento desta revisão a coleta não tinha sido feita: o formulário está escrito e revisado, mas não foi distribuído, e o encaminhamento ao comitê de ética da instituição continua pendente. Por isso não há resultados de participantes aqui.

As seções abaixo definem desde já o que será relatado e com quais critérios. Se essa definição fosse feita depois de ver os dados, seria possível escolher o recorte que deixasse a plataforma em melhor situação; defini-la agora evita essa escolha. Os critérios de cálculo dos escores estão na Seção 4.12 e não são repetidos aqui.

5.1 Composição do Painel

5.2 Assistente de Personalização da Linguagem

5.3 Uso do Ambiente Virtual de Aprendizagem

5.4 Usabilidade e Experiência de Uso

5.5 Potencial Pedagógico e Intenção de Uso

5.6 Riscos e Objeções Levantados pelo Painel

5.7 Respostas Abertas

5.8 Síntese e Confronto com os Objetivos

6 CONSIDERAÇÕES FINAIS

Este capítulo retoma os objetivos, os resultados técnicos apresentados nesta revisão para o TCC II e as limitações que ainda condicionam a avaliação da plataforma.

6.1 Retomada dos Objetivos

O objetivo geral foi projetar, implementar e integrar uma IDE web ao núcleo de tradução e execução configurável. Os fluxos apresentados no Capítulo 4 documentam o assistente de personalização, a edição de código, a execução e depuração, a gestão de linguagens e os recursos educacionais de turmas e exercícios.

A especificação dos elementos configuráveis está consolidada na Tabela 3.1 e na descrição das variantes de análise. A implementação do assistente e dos recursos de edição e execução é apresentada por trechos de código e capturas de tela. A integração utiliza o pacote `compiler` no navegador e, para a validação de submissões, no servidor Next.js, conforme a Figura 4.1.

A revisão incorpora quatro linguagens predefinidas, seleção temporária de linguagem obrigatória, edição conjunta de exercícios e casos de teste e a composição da área de resolução em componentes. A disponibilização técnica permite executar os fluxos locais no navegador. Na revisão de 19 de setembro de 2026, foram aprovados 274 testes da IDE em 54 arquivos; também passaram os 31 casos do pacote E2E, sem ocorrências intermitentes (QUEIROZ, 2026b). A Seção 4.8 registra uma verificação delimitada da interface. Os resultados não permitem considerar concluídos os requisitos de acessibilidade, contraste e internacionalização integral, nem demonstram a disponibilidade de uma implantação pública específica.

A revisão de 19 de outubro de 2026 acrescenta os tutoriais guiados, os diálogos de exemplo da linguagem e de regras do compilador, a página Sobre e a correção de submissões em uma IDE carregada com o código e a linguagem do aluno. A tradução da interface avançou de 56 para 98 componentes, mas a área de resolução de exercícios continua com textos fixos. A suíte da IDE aprovou 312 testes em 60 arquivos; o pacote E2E não foi executado de novo nesta revisão (QUEIROZ, 2026a).

O objetivo de propor um protocolo de avaliação foi atendido pela Seção 4.12, que descreve quem responde, os blocos de itens, as escalas usadas e o cálculo dos escores. Para aplicá-lo ainda falta definir a amostra e obter o encaminhamento de ética da instituição. A

avaliação não foi realizada, e o Capítulo 5 registra essa pendência.

6.2 Limitações

A principal limitação é a ausência de dados de estudantes. O artefato permite investigar a personalização da linguagem, mas ainda não responde empiricamente em que medida essa personalização reduz a barreira sintática. Não se apresentam resultados de aprendizagem, redução de carga cognitiva ou aumento de engajamento.

A verificação de acessibilidade abrange um recorte do assistente, em um navegador, com testes automatizados e uma transição por teclado. Os problemas encontrados exigem ajustes na aplicação; faltam a inspeção dos processos completos e a verificação com tecnologias assistivas. O uso de bibliotecas de componentes não sustenta uma presunção de conformidade.

A configuração da IDE ainda exclui oito arquivos de testes do assistente. A suíte de ponta a ponta utiliza somente Chromium com perfil de computador e prepara parte dos cenários pela API; seus resultados não abrangem todos os navegadores, dispositivos ou sequências de interação. O fluxo versionado passou a condicionar o deploy automatizado à aprovação dos testes, embora suas definições não comprovem regras de proteção de branch (QUEIROZ; SILVA, 2026b; QUEIROZ; SILVA, 2026c; QUEIROZ; SILVA, 2026a). Também não foram conduzidas medições de desempenho nem auditoria de segurança. A configuração do token em *cookie* acessível por JavaScript e a ausência de isolamento de processo na avaliação de programas são aspectos que exigem análise técnica específica antes de ampliar o cenário de uso.

A linguagem oferece construções imperativas e variantes de tipagem e de arranjos implementadas no núcleo. Não permite acrescentar gramáticas arbitrárias, orientação a objetos ou modularização de um programa em múltiplos arquivos de código-fonte. O sistema de arquivos virtual da IDE organiza documentos, sem representar, por si só, um mecanismo de módulos da linguagem.

6.3 Encaminhamentos

Os próximos passos técnicos são corrigir os problemas de acessibilidade documentados, verificar os fluxos completos de teclado e leitor de tela, revisar os textos ainda não internacionalizados e atualizar os oito arquivos excluídos da execução de testes. Os diálogos de ajuda também podem passar a mostrar as regras no vocabulário da linguagem ativa, e o

registro de tutoriais concluídos ainda não é usado pela interface. A ampliação dos cenários E2E e dos navegadores exercitados constitui outra possibilidade, preservando a prioridade de jornadas relevantes e os testes de menor granularidade (QUEIROZ; SILVA, 2026b; QUEIROZ; SILVA, 2026c; VOCKE, 2018). As capturas históricas da implementação devem ser atualizadas conforme a interface se estabilize para a apresentação final.

No plano metodológico, a continuidade consiste em revisar e aplicar o protocolo proposto, depois de definir participantes, instrumentos e procedimentos institucionais. Só então será possível discutir resultados de uso e investigar a contribuição pedagógica da personalização. O compartilhamento de linguagens, já implementado no catálogo comunitário e na vinculação a exercícios e listas, poderá ser estudado como recurso didático em turmas; a exportação para uso fora da plataforma permanece outro possível desdobramento.

REFERÊNCIAS

ADAPTWORKS. **Agile: Desenvolvimento de Software com Entregas Frequentes e Foco no Valor de Negócio**. São Paulo: Casa do Código, 2012.

AHO, A. V.; SETHI, R.; ULLMAN, J. D. **Compiladores**: princípios, técnicas e ferramentas. Rio de Janeiro: LTC, 1995. Tradução de Daniel de Ariosto Pinto.

ALCANTARA, F. **Tabela de Símbolos**. 2024. Disponível em: <https://frankalcantara.com/lf/11-tabelaSimbolos.html>. Acesso em: 30 mar. 2026.

ANICHE, M. **Test-Driven Development: Teste e Design no Mundo Real**. São Paulo: Casa do Código, 2012.

BANGOR, A.; KORTUM, P.; MILLER, J. Determining what individual sus scores mean: adding an adjective rating scale. **Journal of Usability Studies**, v. 4, n. 3, p. 114–123, 2009.

BECK, K.; ANDRES, C. **Extreme Programming Explained: Embrace Change**. 2. ed. Boston: Addison-Wesley, 2004.

Beecrowd. **FAQs: Problems**. 2026. Disponível em: <https://judge.beecrowd.com/en/faqs/about/problems>. Acesso em: 12 set. 2026.

BROOKE, J. Sus: a quick and dirty usability scale. In: JORDAN, P. W.; THOMAS, B.; WEERDMEESTER, B. A.; MCCLELLAND, I. L. (Ed.). **Usability Evaluation in Industry**. London: Taylor & Francis, 1996. p. 189–194.

COOPER, K. D.; TORCZON, L. **Engineering a Compiler**. 2. ed. Waltham: Elsevier / Morgan Kaufmann, 2012.

DAVIS, F. D. Perceived usefulness, perceived ease of use, and user acceptance of information technology. **MIS Quarterly**, v. 13, n. 3, p. 319–340, 1989.

FASTAPI. **Recursos e Documentação Oficial do FastAPI**. 2026. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 12 set. 2026.

GADELHA, D. **Portugol Webstudio: IDE online para o Portugol**. 2026. Disponível em: <https://github.com/dgadelha/Portugol-Webstudio>. Acesso em: 12 set. 2026.

LADNER, R. E. **Learn About the New Quorum Studio**. Computer Science Teachers Association (CSTA), 2019. Disponível em: <https://csteachers.org/learn-about-the-new-quorum-studio/>. Acesso em: 12 set. 2026.

MARTIN, R. C. **Código Limpo: Habilidades Práticas do Agile Software**. Rio de Janeiro: Alta Books, 2009.

MARTIN, R. C. **The Clean Coder: A Code of Conduct for Professional Programmers**. Upper Saddle River: Prentice Hall, 2011.

MARTIN, R. C. **Arquitetura Limpa: O Guia do Artesão para Estrutura e Design de Software**. Rio de Janeiro: Alta Books, 2019.

Microsoft. **Playwright: Best Practices**. 2026. Disponível em: <https://playwright.dev/docs/best-practices>. Acesso em: 19 set. 2026.

Microsoft. **Playwright: Continuous Integration**. 2026. Disponível em: <https://playwright.dev/docs/ci>. Acesso em: 19 set. 2026.

Microsoft. **Playwright: Installation**. 2026. Disponível em: <https://playwright.dev/docs/intro>. Acesso em: 19 set. 2026.

Microsoft. **Playwright: Retries**. 2026. Disponível em: <https://playwright.dev/docs/test-retries>. Acesso em: 19 set. 2026.

Microsoft. **Playwright: TestConfig**. 2026. Disponível em: <https://playwright.dev/docs/api/class-testconfig>. Acesso em: 19 set. 2026.

NORMAN, D.; NIELSEN, J. **The Definition of User Experience (UX)**. 1998. Disponível em: <https://www.nngroup.com/articles/definition-user-experience/>. Acesso em: 12 set. 2026.

NYSTROM, R. **Crafting Interpreters**. [S.l.]: Genever Benning, 2021. ISBN 978-0990582939.

PRICE, A. M. A.; TOSCANI, S. S. **Implementação de Linguagens de Programação: Compiladores**. Porto Alegre: Sagra Luzzatto, 2001.

QUEIROZ, I. L. **Registro de verificação local dos testes da IDE: 19 de outubro de 2026**. 2026. Disponível em: <https://github.com/igorlamoia/dynamic-interpreter/blob/escrita-tcc/qualificacao/igor/verificacao/testes-2026-10-19.json>. Acesso em: 19 out. 2026.

QUEIROZ, I. L. **Registro de verificação local dos testes da IDE e do pacote E2E: 19 de setembro de 2026**. 2026. Disponível em: <https://github.com/igorlamoia/dynamic-interpreter/blob/escrita-tcc/qualificacao/igor/verificacao/testes-2026-09-19.json>. Acesso em: 19 set. 2026.

QUEIROZ, I. L.; SILVA, V. d. S. V. d. **Dynamic Interpreter: fluxo de integração contínua e implantação**. 2026. Disponível em: <https://github.com/igorlamoia/dynamic-interpreter/tree/3ffae0101cf1c6aacc85dfbadce4edf123a4ad4c/.github/workflows/deploy.yml>. Acesso em: 19 set. 2026.

QUEIROZ, I. L.; SILVA, V. d. S. V. d. **Dynamic Interpreter: implementação e testes da IDE**. 2026. Disponível em: <https://github.com/igorlamoia/dynamic-interpreter/tree/3ffae0101cf1c6aacc85dfbadce4edf123a4ad4c/packages/ide>. Acesso em: 19 set. 2026.

QUEIROZ, I. L.; SILVA, V. d. S. V. d. **Dynamic Interpreter: suíte de testes E2E**. 2026. Disponível em: <https://github.com/igorlamoia/dynamic-interpreter/tree/3ffae0101cf1c6aacc85dfbadce4edf123a4ad4c/packages/e2e>. Acesso em: 19 set. 2026.

QUORUM. **The Quorum Programming Language: Documentation and Curriculum**. 2025. Disponível em: <https://quorumlanguage.com/>. Acesso em: 12 set. 2026.

Radix. **Accessibility: Radix Primitives**. 2026. Disponível em: <https://www.radix-ui.com/primitives/docs/overview/accessibility>. Acesso em: 12 set. 2026.

SAÚDE, B. C. N. de. **Resolução n. 466, de 12 de dezembro de 2012**. Brasília: [s.n.], 2012. Diário Oficial da União.

SAÚDE, B. C. N. de. **Resolução n. 510, de 7 de abril de 2016**. Brasília: [s.n.], 2016. Diário Oficial da União.

SAURO, J.; LEWIS, J. R. **Quantifying the User Experience**: practical statistics for user research. Waltham: Morgan Kaufmann, 2012.

SCHREPP, M.; HINDERKS, A.; THOMASCHEWSKI, J. Design and evaluation of a short version of the user experience questionnaire (ueq-s). **International Journal of Interactive Multimedia and Artificial Intelligence**, v. 4, n. 6, p. 103–108, 2017.

SILVA, M. L. d.; PEREIRA, S. S. L.; LIMA, D. R. **Laila: um gerador de analisador léxico e sintático online baseado em FLEX e Bison**. 2021. Instituto Federal do Ceará, Campus Aracati. Disponível em: <https://gestaoaracati.ifce.edu.br/attachments/download/2699/tcc.pdf>. Acesso em: 12 set. 2026.

SILVEIRA, P.; SILVEIRA, G.; LOPES, S.; MOREIRA, G.; STEPPAT, N.; KUNG, F.; WEBBER, J. **Introdução à arquitetura e design de software: uma visão sobre a plataforma Java**. São Paulo: Casa do Código, 2012.

SOUZA, D. M.; BATISTA, M. H. d. S.; BARBOSA, E. F. Problemas e dificuldades no ensino e na aprendizagem de programação: um mapeamento sistemático. **Revista Brasileira de Informática na Educação**, v. 24, n. 1, p. 39–52, 2016.

STEFIK, A.; SIEBERT, S. An empirical investigation into programming language syntax. **ACM Transactions on Computing Education**, ACM, v. 13, n. 4, 2013.

UEYAMA, R. **chibicc: A small C compiler**. 2020. Disponível em: <https://github.com/rui314/chibicc>. Acesso em: 12 set. 2026.

UNIVALI. **Portugol Studio - LITE**. Laboratório de Inovação Tecnológica na Educação, 2026. Disponível em: <https://univali-lite.github.io/Portugol-Studio/>. Acesso em: 12 set. 2026.

Vercel. **API Routes**. 2026. Disponível em: <https://nextjs.org/docs/pages/building-your-application/routing/api-routes>. Acesso em: 12 set. 2026.

Vercel. **Pages Router**. 2026. Disponível em: <https://nextjs.org/docs/pages>. Acesso em: 12 set. 2026.

VOCKE, H. **The Practical Test Pyramid**. 2018. Disponível em: <https://martinfowler.com/articles/practical-test-pyramid.html>. Acesso em: 19 set. 2026.

W3C. **Accessible Rich Internet Applications (WAI-ARIA) 1.2**. [S.l.], 2023. Disponível em: <https://www.w3.org/TR/wai-aria-1.2/>. Acesso em: 12 set. 2026.

WILDT, D.; MOURA, D.; LACERDA, G.; HELM, R. **eXtreme Programming: práticas para o dia a dia no desenvolvimento ágil de software**. São Paulo: Casa do Código, 2015.

World Wide Web Consortium. **Web Content Accessibility Guidelines (WCAG) 2.2**. 2024. Disponível em: <https://www.w3.org/TR/WCAG22/>. Acesso em: 12 set. 2026.

World Wide Web Consortium. **ARIA Authoring Practices Guide: Read Me First**. 2026. Disponível em: <https://www.w3.org/WAI/ARIA/apg/practices/read-me-first/>. Acesso em: 12 set. 2026.

A APÊNDICE A – PLANO DE TRABALHO: DIVISÃO DE RESPONSABILIDADES

Este apêndice detalha a divisão de responsabilidades entre os dois autores, conforme a pré-proposta aprovada e o plano de trabalho entregue à Coordenação do Curso de Engenharia de Computação do Centro Federal de Educação Tecnológica de Minas Gerais, Unidade Leopoldina.

A.1 Atividades Desenvolvidas em Regime Colaborativo

As atividades listadas a seguir foram conduzidas de forma conjunta por Igor Lamoia Queiroz e Victor de Souza Vilela da Silva, com participação efetiva de ambos os integrantes nas etapas de especificação, implementação, integração e revisão:

- Revisão bibliográfica sobre interpretadores, compiladores e ferramentas educacionais relacionadas;
- Definição dos requisitos funcionais e não funcionais e da arquitetura geral do sistema;
- Participação na implementação e validação do núcleo do compilador, incluindo análise léxica, análise sintática, geração e execução de código intermediário;
- Especificação dos mecanismos de configuração das variantes de linguagem implementadas e integração desses mecanismos à experiência de personalização da IDE;
- Integração do núcleo do interpretador com a plataforma web e definição dos contratos de comunicação entre os módulos;
- Revisão mútua de código e tomada de decisões arquiteturais nas etapas de refatoração e evolução do sistema.

A.2 Eixos de Aprofundamento Individual

A partir da base técnica construída em conjunto, cada autor concentrou seu aprofundamento acadêmico em eixos específicos, conforme estabelecido na pré-proposta.

A.2.1 Igor Lamoia Queiroz (este documento)

- Detalhamento da arquitetura da aplicação web (Next.js, Pages Router) e da integração com o Monaco Editor para edição de código com realce de sintaxe configurável;

- Documentação da customização dinâmica de *tokens* e sintaxe por meio do assistente de personalização da linguagem (*wizard*), incluindo o modelo de etapas, a lógica de validação preventiva e o mecanismo de pré-visualização em tempo real;
- Análise da experiência de uso da plataforma, dos contextos React que governam o estado da aplicação e das mediações pedagógicas oferecidas pela interface ao estudante iniciante;
- Descrição da execução das análises léxica e sintática com o código produzido na IDE e da apresentação do código intermediário no terminal integrado;
- Especificação do protocolo de avaliação da experiência de uso, com os instrumentos de coleta, as escalas de usabilidade e o cálculo dos escores. O protocolo é dirigido a um painel de especialistas e ainda não foi aplicado.

A.2.2 Victor de Souza Vilela da Silva (documento complementar)

- Detalhamento das fases de análise léxica e do sistema de varredura por escâneres especializados (*factory pattern*);
- Descrição aprofundada da análise sintática por descida recursiva e da correspondência entre as produções do analisador e a gramática formal da linguagem Java--;
- Documentação da geração de código de três endereços: instrução a instrução, tabela de temporários e tabela de rótulos;
- Análise do funcionamento do interpretador, da tabela de símbolos e do despacho das chamadas de sistema de entrada e saída;
- Desenvolvimento do *back-end* da plataforma em FastAPI, incluindo a modelagem das entidades de domínio, a implementação das rotas REST, a persistência com SQLAlchemy assíncrono e o suporte a autenticação, turmas, exercícios, listas e submissões;
- Verificação técnica do núcleo de tradução, aferindo o comportamento do interpretador frente a programas de teste elaborados para cobrir as construções da linguagem.